DESARROLLO MÓVIL · UNIDAD 1 · PRÁCTICA

Acceso a Cafeto

Guía de estudio para entender, reconstruir y memorizar el Inicio de sesión, el Registro y la Verificación de correo de la app de clientes de Cafeto Las Lomitas, escrita en React Native con Expo y Firebase.

Pantallas **Login · Registro · Verificación** Archivos **9**

Expo SDK **57** · React Native **0.86** Actualizada **14/10/2026**

1

Qué vas a construir

La pantalla replica la vista **1.1 Login inicial** del diseño de Cafeto (carpeta `Vistas-Móvil-Cafeto`) y además incorpora los estados de las vistas 1.2 (error), 1.3 (modo claro, campo enfocado con ayuda y botón deshabilitado), 1.5 (ingresando, con la pantalla simplificada) y 1.6 (mensaje de demasiados intentos). Además incluye el **Registro** en dos pasos (vistas 2.1 a 2.6) y la **Verificación de correo** (vistas 2.4 y 2.8). Ningún archivo tiene comentarios, como pide la consigna: toda la explicación está en esta guía.

Archivos del proyecto que explica esta guía:

- `screens/Login.js`, `screens/Registro.js` y `screens/Verificacion.js`: las tres pantallas.
- `src/componentes/CampoTexto.js`: el campo de formulario reutilizable.
- `src/aviso/AvisoContext.js`: las ventanas de aviso con los colores de Cafeto.
- `src/tema/temas.js` y `src/tema/TemaContext.js`: los colores y el tema compartido.
- `App.js` y `navigation/Navigation.js`: el arranque y la navegación.

Lo que hace cada elemento de la pantalla de Login:

ELEMENTO	COMPORTAMIENTO
Seguir sin cuenta	Inicia una sesión anónima en Firebase (`signInAnonymously`). La app pasa a Inicio. Requiere habilitar el proveedor *Anónimo* en Firebase; si no está habilitado, avisa con un mensaje claro.
Botón de contraste	Alterna entre el tema oscuro (por defecto, como el diseño) y el claro. Cambian colores, logo y barra de estado, y la elección se mantiene en Registro y Verificación.

ELEMENTO	COMPORTAMIENTO
Correo y contraseña	Borde ámbar al enfocar, borde rojo y mensaje debajo si hay error. Al enfocar el correo aparece una ayuda. El ojo muestra u oculta la contraseña.
Recordarme	Si está marcado, guarda el correo en el teléfono con AsyncStorage y lo completa solo la próxima vez.
¿Olvidaste tu contraseña?	Envía el correo de recuperación de Firebase (`sendPasswordResetEmail`) a la dirección escrita.
Iniciar sesión	Gris y deshabilitado hasta que los dos campos tengan algo. Al tocarlo valida, guarda la preferencia de Recordarme y llama a `signInWithEmailAndPassword`. Mientras espera muestra «Ingresando…», el aviso «Un momento, estamos verificando tus datos.» y oculta los otros ingresos.
Google / Apple	Muestran «Próximamente». El ingreso con esos proveedores necesita configuración nativa que el proyecto todavía no tiene.
Registrate	Abre la pantalla de Registro (sección 9).

¿Cómo vuelve la app a la pantalla de Inicio después de entrar? No lo hace el Login. En `Navigation.js` ya había un observador, `onAuthStateChanged`, que cambia las pantallas del stack cuando Firebase detecta una sesión. Por eso el Login solo tiene que autenticar; nunca llama a `navigation.navigate('Home')`.

2

Preparar el proyecto

Antes de escribir las pantallas hicieron falta estos cambios fuera de ellas. Hacelos en este orden.

1 Instalar las fuentes del diseño

El título «Hola de nuevo» usa Caveat y el resto de la pantalla usa Inter. Con la terminal de VS Code abierta en la carpeta `sistema-cafeto-movil`:

```
Terminal                                                PowerShell o CMD
npx expo install expo-font @expo-google-fonts/caveat @expo-google-fonts/inter
```

Para el selector de fecha del Registro:

```
Terminal                                                PowerShell o CMD
npx expo install @react-native-community/datetimepicker
```

Se usa `npx expo install` y no `npm install` porque Expo elige la versión compatible con el SDK 57. Con `npm install` podrías terminar con una versión que no corre en Expo Go.

2 Revisar app.json

Al instalar `expo-font` y el selector de fecha, Expo agregó solo el bloque `plugins`. No tenés que tocarlo, pero conviene saber que existe:

app.json — 28 líneas

```
{
  "expo": {
    "name": "login-signup",
    "slug": "login-signup",
    "version": "1.0.0",
    "orientation": "portrait",
    "icon": "./assets/icon.png",
    "userInterfaceStyle": "light",
    "ios": {
      "supportsTablet": true
    },
    "android": {
      "adaptiveIcon": {
        "backgroundColor": "#E6F4FE",
        "foregroundImage": "./assets/android-icon-foreground.png",
        "backgroundImage": "./assets/android-icon-background.png",
        "monochromeImage": "./assets/android-icon-monochrome.png"
      }
    },
    "web": {
      "favicon": "./assets/favicon.png"
    },
```

```
    "plugins": [
      "expo-font",
      "@react-native-community/datetimepicker"
    ]
  }
}
```

3 Navegación y tema compartido

`navigation/Navigation.js` ahora oculta el encabezado gris de React Navigation, suma las pantallas de Registro y Verificación y decide cuál mostrar según el estado de la cuenta. `App.js` envuelve todo en el proveedor del tema. Las dos cosas se explican en la sección 3.

4 Nombre de la carpeta

La carpeta del proyecto se llama `sistema-cafeto-movil`. Abrila así en VS Code, y ejecutá `npm start` desde adentro.

5 Reponer assets/logo.png

`Home.js` y `SignUp.js` piden `require('../assets/logo.png')`, pero ese archivo no estaba en el repositorio y la app no compilaba. Se copió el isologotipo positivo de Cafeto con ese nombre. No se tocó el código de esas pantallas.

3

Base compartida: tema y navegación

Al sumar el Registro apareció un problema: el tema claro u oscuro era un estado *dentro* del Login. Si elegías el modo claro y tocabas «Registrate», la pantalla nueva arrancaba en oscuro, porque cada pantalla tenía su propio estado. La solución es sacar el tema de las pantallas y ponerlo en un lugar que todas puedan leer: un **contexto** de React.

Esta sección explica las tres piezas que sostienen a todas las pantallas. Conviene estudiarla antes que el Login, porque el Login ya las usa.

3.1 El contexto del tema

src/tema/TemaContext.js — líneas 1–25

```
import React, { createContext, useContext, useMemo, useState } from 'react';
import { isotipos, logos, temas } from './temas';

const TemaContext = createContext(null);

export function TemaProvider({ children }) {
  const [tema, setTema] = useState('oscuro');

  const valor = useMemo(
    () => ({
      tema,
      colores: temas[tema],
      logo: logos[tema],
      isotipo: isotipos[tema],
      cambiarTema: () => setTema((actual) => (actual === 'oscuro' ? 'claro' : 'oscuro')),
    }),
    [tema]
  );

  return <TemaContext.Provider value={valor}>{children}</TemaContext.Provider>;
}

export function useTema() {
  return useContext(TemaContext);
}
```

Qué hace. Crea un contexto, un componente `TemaProvider` que guarda el tema elegido y un hook `useTema()` para leerlo desde cualquier pantalla o componente.

> POR QUÉ ASÍ
>
> **Qué es un contexto.** Normalmente un dato baja de padre a hijo por props. Si el tema tuviera que llegar del `App` al `CampoTexto` pasando por la navegación y la pantalla, habría que escribir `tema={tema}` en cada nivel. El contexto evita ese «pasamanos»: el Provider publica un valor y cualquier componente de adentro lo lee con `useContext`.
>
> `createContext(null)`. El `null` es el valor que recibiría un componente que no esté dentro del Provider. No debería pasar, porque el Provider envuelve toda la app (3.2).

`children`. Es la prop especial que contiene lo que se escribe entre las etiquetas de apertura y cierre. Gracias a ella, `<TemaProvider><Navigation /></TemaProvider>` dibuja la navegación adentro.

Qué publica. El nombre del tema, la paleta, el logo y el isotipo ya elegidos, y la función para cambiarlo. Así las pantallas no repiten `temas[tema]` ni `logos[tema]`: piden lo que necesitan con `const { colores, logo } = useTema()`.

`useMemo` **en el valor.** Cada vez que el Provider se dibuja, un objeto escrito a mano sería un objeto *nuevo*, y React avisaría a todos los que lo leen aunque nada haya cambiado. Con `useMemo(..., [tema])` el objeto solo se recrea cuando cambia el tema.

`setTema((actual) ⇒ ...)`. Es la forma *funcional* del setter: recibe el valor actual y devuelve el nuevo. Se usa cuando el estado nuevo depende del anterior, porque garantiza leer el valor más reciente.

El hook `useTema`. Envuelve `useContext(TemaContext)` para que las pantallas no tengan que importar el contexto. Si mañana cambia cómo se guarda el tema, solo se toca este archivo.

3.2 App.js: los Providers envuelven todo

App.js líneas 1–15

```
1  import React from 'react';
2  import 'react-native-gesture-handler';
3  import Navigation from './navigation/Navigation';
4  import { TemaProvider } from './src/tema/TemaContext';
5  import { AvisoProvider } from './src/aviso/AvisoContext';
6
7  export default function App() {
8    return (
9      <TemaProvider>
10       <AvisoProvider>
11         <Navigation />
12       </AvisoProvider>
13     </TemaProvider>
14   );
15 }
```

POR QUÉ ASÍ

Los Providers van lo más arriba posible, por encima de `Navigation`, para que todas las pantallas queden adentro. Si estuvieran dentro de una pantalla, las demás no los verían.

El orden importa. `AvisoProvider` va *dentro* de `TemaProvider` porque usa `useTema()` para pintar la ventana. Al revés, no encontraría el tema.

Los paréntesis después de `return` permiten escribir el JSX en varias líneas. Sin ellos, JavaScript termina la instrucción en el salto de línea y la función devolvería `undefined`.

3.3 Navigation.js: quién ve qué pantalla

navigation/Navigation.js líneas 1-13

```
1  import React, { useEffect, useState } from 'react';
2  import { ActivityIndicator, StyleSheet, Text, View } from 'react-native';
3  import { NavigationContainer } from '@react-navigation/native';
4  import { createStackNavigator } from '@react-navigation/stack';
5  import { onIdTokenChanged } from 'firebase/auth';
6  import { auth, isFirebaseConfigured } from '../src/config/firebaseConfig';
7  import Login from '../screens/Login';
8  import Registro from '../screens/Registro';
9  import Verificacion from '../screens/Verificacion';
10 import Home from '../screens/Home';
11
12 const Stack = createStackNavigator();
13
```

POR QUÉ ASÍ

Se reemplazó `SignUp` por las dos pantallas nuevas, `Registro` y `Verificacion`. El archivo `screens/SignUp.js` del profesor ya no hace falta y se borró del proyecto.

navigation/Navigation.js líneas 14-24

```
14 export default function Navigation() {
15   const [user, setUser] = useState(undefined);
16   const [verificado, setVerificado] = useState(false);
17
18   useEffect(() => {
19     if (!auth) return;
20     return onIdTokenChanged(auth, (usuario) => {
21       setUser(usuario);
22       setVerificado(Boolean(usuario && (usuario.emailVerified || usuario.isAnonymous)));
23     });
24   }, []);
```

Qué hace. Escucha a Firebase y guarda dos datos: el usuario (o `null` si no hay sesión) y si está **verificado**. Cuenta como verificado quien confirmó su correo o quien entró como invitado.

POR QUÉ ASÍ

`onIdTokenChanged` en lugar de `onAuthStateChanged`. Los dos avisan cuando alguien entra o sale. Pero cuando el usuario confirma su correo, Firebase no avisa nada: el usuario sigue siendo el mismo. `onIdTokenChanged` además avisa cuando se renueva el *token* de la sesión, y la pantalla de Verificación lo renueva a propósito (10.8) para que la navegación se entere del cambio.

Por qué un estado aparte para `verificado`. Firebase entrega siempre el *mismo objeto* de usuario. Si solo se hiciera `setUser(usuario)`, React compararía el objeto nuevo con el anterior, vería que es el mismo y no volvería a dibujar. Un booleano que pasa de `false` a `true` sí cambia, y fuerza el redibujo.

`Boolean(...)` convierte el resultado en `true` o `false`. Sin él, cuando no hay usuario la expresión valdría `null`.

`usuario.isAnonymous`. El invitado («Seguir sin cuenta») no tiene correo que verificar. Sin esta condición quedaría atrapado en la pantalla de Verificación.

navigation/Navigation.js líneas 26-39

```
26   if (!isFirebaseConfigured) {
27     return (
28       <View style={styles.center}>
29         <Text style={styles.message}>
30           Falta configurar Firebase. Copia .env.example como .env y completa los datos de tu
proyecto.
31         </Text>
32       </View>
33     );
34   }
35
36   if (user === undefined) {
37     return <View style={styles.center}><ActivityIndicator size="large" /></View>;
38   }
39
```

POR QUÉ ASÍ

Son los dos controles que ya traía el proyecto: el aviso si falta el archivo `.env` y el indicador mientras Firebase todavía no respondió (`undefined` significa «no sé todavía»; `null` significa «sé que no hay sesión»).

navigation/Navigation.js líneas 40-61

```
40   return (
41     <NavigationContainer>
42       <Stack.Navigator screenOptions={{ headerShown: false }}>
43         {!user ? (
44           <>
45             <Stack.Screen name="Login" component={Login} />
46             <Stack.Screen name="Registro" component={Registro} />
47           </>
48         ) : !verificado ? (
49           <Stack.Screen name="Verificacion" component={Verificacion} />
50         ) : (
51           <Stack.Screen name="Home" component={Home} options={{ headerShown: true, title: 'Inicio' }}
/>
52         )}
53       </Stack.Navigator>
54     </NavigationContainer>
55   );
56 }
57
58 const styles = StyleSheet.create({
59   center: { flex: 1, justifyContent: 'center', alignItems: 'center', padding: 24 },
60   message: { textAlign: 'center', fontSize: 16 },
61 });
```

Qué hace. Muestra uno de tres grupos de pantallas:

SITUACIÓN	PANTALLAS DISPONIBLES
Sin sesión	Login y Registro

SITUACIÓN	PANTALLAS DISPONIBLES
Con sesión, correo sin verificar	Verificación
Con sesión verificada o invitado	Home

POR QUÉ ASÍ

Un ternario encadenado. `!user ? A : !verificado ? B : C` se lee como «si no hay usuario, A; si no, si no está verificado, B; si no, C». Para tres casos es claro; con más, convendría una función aparte.

Las pantallas que no corresponden directamente no existen. Un usuario sin verificar no puede «volver» al Login con el botón atrás, porque el Login no está en el stack. Es más seguro que esconder botones.

`screenOptions={{ headerShown: false }}` en el Navigator oculta el encabezado de todas las pantallas de una vez. Home lo vuelve a mostrar con su propio `options`, porque todavía usa el diseño del profesor.

Cómo funciona el registro sin navegar. Cuando el Registro crea la cuenta, Firebase inicia la sesión solo. El listener recibe un usuario con `emailVerified` en `false` y la navegación cambia a Verificación sin que el Registro llame a `navigate`. Es el mismo mecanismo que ya usaba el Login.

3.4 Avisos con el estilo de Cafeto

Al principio, los mensajes («Revisá tu correo», «Sin conexión», «Próximamente») usaban `Alert.alert` de React Native, que muestra la ventana gris del sistema operativo. Esa ventana no se puede pintar: ignora el tema y los colores de la marca. `src/aviso/AvisoContext.js` la reemplaza por una propia, con el estilo de las tarjetas del diseño (`.alert.warn` y `.alert.ok` en `cafeto-app.html`).

TIPO	COLOR	CUÁNDO SE USA
`'error'`	Rojo	Algo falló: sin conexión, no se pudo crear la cuenta, no se pudo enviar el correo.
`'advertencia'`	Ámbar	Hay que prestar atención o esperar: función todavía no disponible, demasiados pedidos seguidos, confirmar algo que borra datos.
`'exito'`	Verde	Salió bien: se envió el correo, se reenvió el enlace.

Desde cualquier pantalla se usa así: `avisar('exito', 'Revisá tu correo', 'Te mandamos un enlace...')`. Es el mismo orden que tenía `Alert.alert`, con el tipo adelante.

src/aviso/AvisoContext.js líneas 1–17

```
import React, { createContext, useContext, useMemo, useState } from 'react';
import { Modal, Pressable, StyleSheet, Text, View } from 'react-native';
import { Feather } from '@expo/vector-icons';
import { useTema } from '../tema/TemaContext';

const AvisoContext = createContext(null);

const ICONOS = {
  error: 'alert-circle',
  advertencia: 'alert-triangle',
  exito: 'check-circle',
};

const ESTILOS_BOTON = {
  cancelar: ['botonCancelar', 'textoCancelar'],
  destructivo: ['botonDestructivo', 'textoDestructivo'],
};
```

POR QUÉ ASÍ

Tablas en lugar de `if`. `ICONOS` dice qué ícono lleva cada tipo y `ESTILOS_BOTON` qué par de estilos (caja y texto) lleva cada clase de botón. Para elegir alcanza con `ICONOS[aviso.tipo]`; agregar un tipo nuevo es sumar una línea.

Es el mismo patrón de contexto que el tema (3.1): un contexto, un Provider y un hook.

src/aviso/AvisoContext.js — líneas 19–42

```
export function AvisoProvider({ children }) {
  const { colores } = useTema();
  const estilos = useMemo(() => crearEstilos(colores), [colores]);
  const [aviso, setAviso] = useState(null);

  const valor = useMemo(
    () => ({
      avisar: (tipo, titulo, mensaje, botones = [{ texto: 'Entendido' }]) => {
        setAviso({ tipo, titulo, mensaje, botones });
      },
    }),
    []
  );

  const colorDelIcono = {
    error: colores.peligro,
    advertencia: colores.marca,
    exito: colores.exito,
  };

  const tocarBoton = (boton) => {
    setAviso(null);
    if (boton.alPresionar) boton.alPresionar();
  };
```

Qué hace. Guarda el aviso que se está mostrando (o `null` si no hay ninguno) y publica una sola función, `avisar`, que lo carga.

POR QUÉ ASÍ

Un estado que es un objeto o `null`. Con un solo dato se sabe si hay que mostrar la ventana (`aviso ≠= null`) y qué mostrar en ella. No hace falta un booleano `visible` aparte que pueda quedar desincronizado.

Botones por defecto. Si quien avisa no pasa botones, aparece uno solo, «Entendido». Para una confirmación se pasan dos, como hace la Verificación (10.5).

`useMemo(..., [])` crea la función `avisar` una sola vez. Como solo usa `setAviso`, que React garantiza que no cambia, no depende de nada.

`tocarBoton` primero cierra el aviso y después ejecuta la acción del botón, si tiene. Así, si la acción muestra otro aviso, el nuevo reemplaza al anterior en vez de quedar tapado.

src/aviso/AvisoContext.js líneas 44-85

```
44   return (
45     <AvisoContext.Provider value={valor}>
46       {children}
47       <Modal
48         visible={aviso ≠= null}
49         transparent
50         animationType="fade"
51         statusBarTranslucent
52         onRequestClose={() ⇒ setAviso(null)}
53       >
54         <View style={estilos.velo}>
55           {aviso && (
56             <View style={[estilos.tarjeta, estilos[aviso.tipo]]}>
57               <View style={estilos.cabecera}>
58                 <Feather name={ICONOS[aviso.tipo]} size={22} color={colorDelIcono[aviso.tipo]} />
59                 <Text style={estilos.titulo}>{aviso.titulo}</Text>
60               </View>
61               <Text style={estilos.mensaje}>{aviso.mensaje}</Text>
62               <View style={estilos.botones}>
63                 {aviso.botones.map((boton) ⇒ {
64                   const [estiloBoton, estiloTexto] = ESTILOS_BOTON[boton.estilo] ?? [
65                     'botonPrincipal',
66                     'textoPrincipal',
67                   ];
68                   return (
69                     <Pressable
70                       key={boton.texto}
71                       style={({ pressed }) ⇒ [estilos.boton, estilos[estiloBoton], pressed &&
estilos.presionado]}
72                       onPress={() ⇒ tocarBoton(boton)}
73                     >
74                       <Text style={[estilos.textoBoton, estilos[estiloTexto]]}>{boton.texto}</Text>
75                     </Pressable>
76                   );
77                 })}
78               </View>
79             </View>
80           )}
81         </View>
82       </Modal>
83     </AvisoContext.Provider>
84   );
85 }
```

POR QUÉ ASÍ

`{children}` **y después el** `Modal`**.** El Provider dibuja toda la app y, encima, la ventana. `Modal` es el componente de React Native para mostrar algo por encima de todo, incluso de la navegación.

Props del Modal. `transparent` deja ver la pantalla detrás, oscurecida por el `velo`. `animationType="fade"` la hace aparecer de a poco. `statusBarTranslucent` hace que en Android el velo cubra también la barra de estado. `onRequestClose` es obligatorio en Android: se ejecuta con el botón atrás del teléfono y cierra el aviso.

`estilos[aviso.tipo]`**.** Los estilos `error`, `advertencia` y `exito` se llaman igual que los tipos, así que el tipo elige directamente su fondo y su borde.

Desestructurar un arreglo con valor por defecto. `const [estiloBoton, estiloTexto] = ESTILOS_BOTON[boton.estilo] ?? [...]` toma los dos nombres de estilo del botón; si el botón no tiene `estilo`, usa el principal (ámbar).

`key={boton.texto}`**.** Como en las barras de fortaleza (9.20), cada elemento de una lista necesita una clave única. Dentro de un aviso no hay dos botones con el mismo texto.

src/aviso/AvisoContext.js líneas 87–89

```
87 export function useAviso() {
88   return useContext(AvisoContext);
89 }
```

src/aviso/AvisoContext.js líneas 91–175

```
91 function crearEstilos(c) {
92   return StyleSheet.create({
93     velo: {
94       flex: 1,
95       justifyContent: 'center',
96       paddingHorizontal: 28,
97       backgroundColor: c.velo,
98     },
99     tarjeta: {
100      padding: 20,
101      borderRadius: 18,
102      borderWidth: 1,
103    },
104    error: {
105      backgroundColor: c.fondoPeligro,
106      borderColor: c.bordePeligro,
107    },
108    advertencia: {
109      backgroundColor: c.fondoAdvertencia,
110      borderColor: c.bordeAdvertencia,
111    },
112    exito: {
113      backgroundColor: c.fondoExito,
114      borderColor: c.bordeExito,
115    },
116    cabecera: {
117      flexDirection: 'row',
118      alignItems: 'center',
119      gap: 10,
120    },
121    titulo: {
```

```
      flex: 1,
      fontFamily: 'Inter_600SemiBold',
      fontSize: 16,
      color: c.texto,
    },
    mensaje: {
      fontFamily: 'Inter_400Regular',
      fontSize: 14.5,
      lineHeight: 21,
      color: c.textoSecundario,
      marginTop: 8,
      marginLeft: 32,
    },
    botones: {
      flexDirection: 'row',
      justifyContent: 'flex-end',
      gap: 10,
      marginTop: 20,
    },
    boton: {
      height: 44,
      paddingHorizontal: 18,
      borderRadius: 12,
      justifyContent: 'center',
      alignItems: 'center',
    },
    botonPrincipal: {
      backgroundColor: c.marca,
    },
    botonCancelar: {
      borderWidth: 1,
      borderColor: c.borde,
    },
    botonDestructivo: {
      backgroundColor: c.peligro,
    },
    presionado: {
      opacity: 0.85,
    },
    textoBoton: {
      fontFamily: 'Inter_600SemiBold',
      fontSize: 15,
    },
    textoPrincipal: {
      color: c.textoSobreMarca,
    },
    textoCancelar: {
      color: c.texto,
    },
    textoDestructivo: {
      color: '#FFFFFF',
    },
  });
}
```

POR QUÉ ASÍ

El velo usa un color con transparencia (`rgba`): oscurece la pantalla de fondo sin taparla del todo. `justifyContent: 'center'` pone la tarjeta en el medio.

Los colores salen de la paleta (5.3), donde se sumaron `bordePeligro` , `bordeExito` , `fondoAdvertencia` , `bordeAdvertencia` y `velo` , con un valor para cada tema. Los bordes oscuros son los del diseño: `#6B4A52` para error y `#2F6B55` para éxito.

> `mensaje` tiene `marginLeft: 32` para alinearse con el título y no con el ícono (22 de ícono más 10 de separación).

4

Login: orden de construcción desde una hoja en blanco

El archivo final se lee de arriba hacia abajo, pero no se escribe en ese orden. Se construye de afuera hacia adentro: primero algo que se vea en pantalla, después el aspecto y por último el comportamiento. Así cada paso se puede probar con `npm start` antes de seguir. Usá esta secuencia cuando practiques.

1 Esqueleto que compile

Creá `screens/Login.js` con `import React from 'react'`, `import { Text, View } from 'react-native'` y un `export default function Login({ navigation })` que devuelva un `<View>` con un `<Text>`. Guardá y mirá el teléfono. Si ves el texto, la conexión con la navegación funciona.

2 Paleta y función de estilos

Escribí `src/tema/temas.js` copiando los colores del diseño, y al final del Login la función `crearEstilos(c)` con solo `pantalla` y `contenido`. Para probar, importá `temas` y agregá `const colores = temas.oscuro` y `const estilos = crearEstilos(colores)`. Ya tenés el fondo índigo.

3 Contenedores de pantalla

Envolvé todo en `SafeAreaView` → `KeyboardAvoidingView` → `ScrollView` (líneas 176–187) y agregá `StatusBar`. Desde acá todo lo que agregues va dentro del `ScrollView`.

4 Parte estática de arriba hacia abajo

Barra superior, logo con «LAS LOMITAS», título y subtítulo, etiquetas y campos (todavía sin estado), fila de Recordarme, botón principal, separador, botones sociales y pie. Por cada pieza que agregues en JSX, agregá su estilo en `crearEstilos` y guardá. Es el momento de comparar con la captura del diseño.

5 Fuentes

Importá `useFonts` y las cuatro variantes (líneas 18–19), llamá al hook (29–34) y agregá la salida temprana con el indicador (168–174). Después sumá `fontFamily` a cada estilo de texto.

6 Estado de los campos

Declará `correo`, `contrasena`, `mostrarContrasena` y `campoActivo`, y conectalos a los `TextInput` con `value`, `onChangeText`, `onFocus` y `onBlur`. Probá que el borde se ponga ámbar y que el ojo funcione.

7 Tema claro

Escribí `TemaContext.js`, envolvé la app con el proveedor en `App.js` (sección 3) y reemplazá `temas.oscuro` por `useTema()` y el `useMemo` de la línea 46. Tocá el botón de contraste para probarlo.

8 Validación y errores

Agregá `errorCorreo` y `errorContrasena`, la constante `FORMATO_CORREO`, `validarFormulario`, los manejadores `cambiarCorreo` / `cambiarContrasena` y los bloques condicionales de error en el JSX. Probá enviar el formulario vacío.

9 Firebase

Importá `auth` y las tres funciones de Firebase. Escribí `mostrarErrorDeFirebase`, después `iniciarSesion` con el estado `cargando`, y por último `recuperarContrasena` y `seguirSinCuenta`.

10 Recordarme

Importá AsyncStorage, agregá el estado `recordarme`, la constante de la clave, `guardarPreferenciaRecordarme` y el `useEffect` que lee el correo guardado al abrir la pantalla.

11 Detalles finales

`avisarProximamente` para Google y Apple, `hitSlop` en el ojo, la opacidad al presionar y `onSubmitEditing` para que la tecla «Ir» del teclado inicie sesión.

12 Estados de las vistas 1.3 y 1.5

Agregá `formularioCompleto` junto a `colores`, el estilo gris del botón y su `disabled`. Después la ayuda debajo del correo. Por último el ternario de `cargando` que cambia separador y botones sociales por el aviso, el espaciador y la condición del texto legal. Para ver «Ingresando…» con calma, probá con el wifi lento o apagado.

5

Login: importaciones, datos y lógica

5.1 Importaciones

screens/Login.js líneas 1–23

```
import React, { useEffect, useMemo, useState } from 'react';
import {
  ActivityIndicator,
  Image,
  KeyboardAvoidingView,
  Platform,
  Pressable,
  ScrollView,
  StyleSheet,
  Text,
  TextInput,
  View,
} from 'react-native';
import { SafeAreaView } from 'react-native-safe-area-context';
import { StatusBar } from 'expo-status-bar';
import { Feather, FontAwesome, Ionicons } from '@expo/vector-icons';
import { useFonts, Caveat_700Bold } from '@expo-google-fonts/caveat';
import { Inter_400Regular, Inter_500Medium, Inter_600SemiBold } from '@expo-google-fonts/inter';
import AsyncStorage from '@react-native-async-storage/async-storage';
import { sendPasswordResetEmail, signInAnonymously, signInWithEmailAndPassword } from
'firebase/auth';
import { auth } from '../src/config/firebaseConfig';
import { useTema } from '../src/tema/TemaContext';
import { useAviso } from '../src/aviso/AvisoContext';
```

Qué hace. Trae todo lo que el archivo usa. Se agrupan de lo más general a lo más propio: React, React Native, librerías de Expo, librerías externas y al final los módulos del propio proyecto (`../src/config/firebaseConfig`).

POR QUÉ ASÍ

Llaves en lugar de import por defecto. `react-native` exporta muchos componentes con nombre, así que se piden por nombre. Solo `React`, `AsyncStorage` y el componente `Login` usan export por defecto.

Lista vertical y ordenada alfabéticamente. Cuando el import de `react-native` tiene doce nombres, una línea por nombre permite ver rápido si falta o sobra algo.

SafeAreaView de react-native-safe-area-context y no de react-native. El de React Native solo funciona en iOS y está desaconsejado. El de la librería respeta la muesca y la barra de estado también en Android.

Pressable en lugar de TouchableOpacity. El proyecto original usaba TouchableOpacity. Pressable es la API moderna: su `style` puede ser una función que recibe `{ pressed }`, lo que da control total del aspecto presionado. Se usa en los botones principales.

> **Tres familias de íconos.** Feather tiene los íconos de línea fina del diseño (sobre, candado, ojo). Ionicons tiene `contrast`, el círculo partido del botón de tema. FontAwesome tiene los logos de Google y Apple.

5.2 Constantes del módulo

screens/Login.js — líneas 25–26

```
25 const CLAVE_CORREO_RECORDADO = 'cafeto:correoRecordado';
26 const FORMATO_CORREO = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
```

Qué hace. `CLAVE_CORREO_RECORDADO` es el nombre con el que se guarda el correo en el teléfono. `FORMATO_CORREO` es una expresión regular: «algo sin espacios ni @, una @, algo, un punto, algo».

> **POR QUÉ ASÍ**
>
> Van fuera del componente porque nunca cambian. Si estuvieran adentro, se volverían a crear en cada render. Las mayúsculas con guion bajo son la convención para constantes fijas. El prefijo `cafeto:` evita chocar con claves de otras librerías que también usan AsyncStorage.
>
> La expresión regular no pretende validar todo correo posible; eso lo hace Firebase. Solo frena errores evidentes sin gastar una llamada a la red.

5.3 Paleta de temas (src/tema/temas.js)

src/tema/temas.js — líneas 1–54

```
1  export const temas = {
2    oscuro: {
3      fondo: '#121128',
4      superficie: '#1B1B3A',
5      superficieActiva: '#22223F',
6      borde: '#2E2E5C',
7      divisor: '#2E2E52',
8      texto: '#E2E8F0',
9      textoSecundario: '#A6B0C3',
10     textoTenue: '#6E7899',
11     titulo: '#FDE1A5',
12     marca: '#FAB41F',
13     marcaSuave: '#2A2410',
14     textoSobreMarca: '#161616',
15     peligro: '#F0564A',
16     fondoPeligro: '#2A1B22',
17     exito: '#35D97C',
18     fondoExito: '#152A24',
19     bordeExito: '#2F6B55',
20     bordePeligro: '#6B4A52',
21     fondoAdvertencia: '#2A2410',
22     bordeAdvertencia: '#6B5A2A',
23     velo: 'rgba(8, 8, 20, 0.72)',
24     fondoApple: '#000000',
25     bordeApple: '#2E2E5C',
26     barraEstado: 'light',
27   },
28   claro: {
```

```
    fondo: '#FFFFFF',
    superficie: '#F4F1EA',
    superficieActiva: '#FFFFFF',
    borde: '#E6E2D8',
    divisor: '#EAECF1',
    texto: '#161616',
    textoSecundario: '#5B6078',
    textoTenue: '#8A8F9C',
    titulo: '#161616',
    marca: '#FAB41F',
    marcaSuave: '#FFF3D6',
    textoSobreMarca: '#161616',
    peligro: '#F0564A',
    fondoPeligro: '#FDECEC',
    exito: '#1F9D57',
    fondoExito: '#E9F8F0',
    bordeExito: '#A8DFC0',
    bordePeligro: '#F3B8B3',
    fondoAdvertencia: '#FFF3D6',
    bordeAdvertencia: '#F2D38A',
    velo: 'rgba(22, 22, 22, 0.45)',
    fondoApple: '#000000',
    bordeApple: '#000000',
    barraEstado: 'dark',
  },
};
```

Dónde está. La paleta nació dentro del Login y se mudó a `src/tema/temas.js` cuando la necesitaron también el Registro y la Verificación. En el archivo se agregaron colores que el Login no usa: `marcaSuave`, `fondoPeligro`, `exito` y `fondoExito`.

Qué hace. Un objeto con dos paletas, `oscuro` y `claro`, con las mismas claves. Los valores salen de los tokens del diseño (`--bg`, `--surface`, `--brand`, etc. de `cafeto-app.html`). `barraEstado` indica si los íconos del reloj y la batería se dibujan claros u oscuros.

> **POR QUÉ ASÍ**
>
> Los nombres describen *función* (`fondo`, `peligro`, `textoTenue`) y no color (`azulOscuro`). Así los estilos dicen `c.peligro` y funcionan con cualquier paleta. Si un día cambia el rojo, se cambia en un solo lugar.
>
> Las dos paletas tienen exactamente las mismas claves. Es la regla que hace posible `temas[tema]` en el contexto (3.1): el resto del código no sabe ni le importa cuál está activa.
>
> `export const` permite importar cada objeto por nombre desde otro archivo: `import { temas } from './temas'`.
>
> Alternativa descartada: `useColorScheme()` de React Native para seguir el tema del sistema. En este proyecto `app.json` fija `"userInterfaceStyle": "light"`, así que siempre devolvería claro, y el diseño arranca en oscuro.

5.4 Logos e isotipos por tema

src/tema/temas.js líneas 56–64

```
56 export const logos = {
57   oscuro: require('../../assets/cafeto-isologotipo-negativo@512.png'),
58   claro: require('../../assets/cafeto-isologotipo-positivo@512.png'),
59 };
60
61 export const isotipos = {
62   oscuro: require('../../assets/cafeto-isotipo-blanco@512.png'),
63   claro: require('../../assets/cafeto-isotipo-negro@512.png'),
64 };
```

Qué hace. Asocia cada tema con su versión del isologotipo: «negativo» (contorno blanco) sobre fondo oscuro, «positivo» (contorno negro) sobre fondo claro, como indica el manual de marca. `isotipos` hace lo mismo con el dibujo sin el rotulado, que usan los encabezados de Registro y Verificación. Las rutas empiezan con `../../` porque el archivo está dos carpetas adentro (`src/tema`).

> POR QUÉ ASÍ
>
> Metro, el empaquetador de React Native, necesita ver cada `require` con la ruta escrita de forma literal para incluir la imagen en la app. Por eso **no** se puede hacer `require(`../assets/logo-${tema}.png`)`. La solución estándar es un objeto con todos los `require` posibles y elegir uno con la clave.
>
> Se usan los PNG y no los SVG porque `<Image>` no dibuja SVG. Para SVG haría falta instalar `react-native-svg`.

5.5 Firma del componente y carga de fuentes

screens/Login.js líneas 28–34

```
28 export default function Login({ navigation, route }) {
29   const [fuentesCargadas] = useFonts({
30     Caveat_700Bold,
31     Inter_400Regular,
32     Inter_500Medium,
33     Inter_600SemiBold,
34   });
```

Qué hace. Declara el componente y recibe `navigation` y `route`, que React Navigation le pasa a toda pantalla del stack. `route` trae los parámetros con los que se abrió la pantalla. `useFonts` empieza a cargar las cuatro tipografías y devuelve un arreglo cuyo primer elemento es `true` cuando terminan.

> POR QUÉ ASÍ
>
> `{ navigation, route }` desestructura las props: es más corto que escribir `props.navigation` cada vez.
>
> Se escribe `{ Caveat_700Bold, ... }` sin `: valor` porque en JavaScript `{ Caveat_700Bold }` equivale a `{ Caveat_700Bold: Caveat_700Bold }`. La clave es el

nombre que después usás en `fontFamily`.

Cada pantalla (Login, Registro y Verificación) carga las fuentes que usa, para que cada archivo se entienda solo. `expo-font` recuerda las que ya cargó, así que la segunda pantalla no las vuelve a descargar. En una app más grande convendría cargarlas una sola vez en `App.js`.

5.6 Estados

screens/Login.js — líneas 36-43

```
36  const [correo, setCorreo] = useState('');
37  const [contrasena, setContrasena] = useState('');
38  const [mostrarContrasena, setMostrarContrasena] = useState(false);
39  const [recordarme, setRecordarme] = useState(false);
40  const [campoActivo, setCampoActivo] = useState(null);
41  const [errorCorreo, setErrorCorreo] = useState('');
42  const [errorContrasena, setErrorContrasena] = useState('');
43  const [cargando, setCargando] = useState(false);
```

Qué hace. Cada `useState` guarda un dato que, al cambiar, vuelve a dibujar la pantalla.

ESTADO	VALOR INICIAL	PARA QUÉ
`correo`, `contrasena`	`''`	Lo que el usuario escribe.
`mostrarContrasena`	`false`	Si el campo muestra los caracteres o puntos.
`recordarme`	`false`	Si la casilla está marcada.
`campoActivo`	`null`	`'correo'`, `'contrasena'` o ninguno; pinta el borde ámbar.
`errorCorreo`, `errorContrasena`	`''`	Mensaje de error de cada campo. Vacío significa sin error.
`cargando`	`false`	Si hay una operación con Firebase en curso.

POR QUÉ ASÍ

Un estado por dato en lugar de un objeto `formulario` con todo junto. Con estados separados, cada setter cambia solo lo suyo y no hace falta copiar el resto con `...formulario`.

Un único `campoActivo` en lugar de dos booleanos `correoEnfocado` y `contrasenaEnfocada`: solo un campo puede tener el foco a la vez, y un valor único hace imposible el estado inválido «los dos enfocados».

Errores como texto y no como booleano. El mismo dato sirve para decidir si se pinta en rojo (`=/= ''`) y para mostrar el mensaje.

> Se escribe `contrasena` sin ñ: los identificadores con caracteres no ASCII funcionan en JavaScript pero traen problemas con teclados, búsquedas y algunas herramientas.

5.7 Valores derivados

screens/Login.js — líneas 45-48

```
45  const { colores, logo, cambiarTema } = useTema();
46  const { avisar } = useAviso();
47  const estilos = useMemo(() => crearEstilos(colores), [colores]);
48  const formularioCompleto = correo.trim() !== '' && contrasena !== '';
```

Qué hace. `useTema()` entrega la paleta activa, el logo del tema y la función para cambiarlo (3.1). `useAviso()` entrega `avisar`, la función que muestra las ventanas de aviso (3.4). `estilos` es el resultado de pasar esa paleta a `crearEstilos`. `formularioCompleto` vale `true` cuando el correo (sin espacios) y la contraseña tienen algo escrito; decide si el botón principal está activo.

> POR QUÉ ASÍ
>
> El tema ya no es un estado del Login: vive en el contexto, y el Login solo lo lee. Así, si lo cambiás acá, el Registro lo recibe igual. Regla útil: un dato que comparten varias pantallas vive arriba de todas ellas.
>
> `useMemo` guarda el resultado y solo vuelve a llamar a `crearEstilos` cuando cambia `colores`. Sin él, cada letra que escribís en un campo volvería a crear los 48 estilos. Funciona porque el contexto entrega siempre el mismo objeto `temas.oscuro` mientras no cambie el tema, entonces React detecta que no cambió.
>
> `formularioCompleto` sigue la misma regla: se calcula en cada render a partir de `correo` y `contrasena`. No necesita `useMemo` porque comparar dos textos es instantáneo; memorizar algo tan barato agrega código sin ganar nada.

5.8 Completar el correo al abrir

screens/Login.js — líneas 50-64

```
50  useEffect(() => {
51    AsyncStorage.getItem(CLAVE_CORREO_RECORDADO).then((correoGuardado) => {
52      if (correoGuardado) {
53        setCorreo((actual) => actual || correoGuardado);
54        setRecordarme(true);
55      }
56    });
57  }, []);
58
59  useEffect(() => {
60    if (route.params?.correo) {
61      setCorreo(route.params.correo);
62      setErrorCorreo('');
63    }
64  }, [route.params?.correo]);
```

Qué hace. El primer efecto, al montarse la pantalla, busca en el almacenamiento del teléfono si hay un correo guardado; si lo hay, lo pone en el campo y marca la casilla. El segundo completa el correo que manda el Registro cuando ese correo ya tiene cuenta (9.15).

> POR QUÉ ASÍ
>
> El arreglo vacío `[]` significa «ejecutar una sola vez, al montar». Sin él, se ejecutaría después de cada render y pisaría lo que el usuario escribe.
>
> Se usa `.then()` en vez de `async/await` porque la función que recibe `useEffect` no puede ser `async`: React espera que devuelva nada o una función de limpieza, y una función async devuelve una promesa.
>
> `setCorreo((actual) ⇒ actual || correoGuardado)`. AsyncStorage tarda un instante en responder. Si para entonces el campo ya tiene el correo que mandó el Registro, se respeta ese; el guardado solo se usa si el campo sigue vacío.
>
> `route.params?.correo` usa `?.` porque cuando el Login se abre normalmente no hay parámetros y `route.params` es `undefined`. Está en las dependencias del efecto: si llega un correo nuevo, el efecto vuelve a correr.
>
> AsyncStorage guarda texto simple, sin cifrar. Por eso se guarda el correo y **nunca** la contraseña. La sesión en sí ya la conserva Firebase con la persistencia configurada en `firebaseConfig.js`.

5.9 Manejadores simples

screens/Login.js — líneas 66–74

```
66   const cambiarCorreo = (texto) ⇒ {
67     setCorreo(texto);
68     setErrorCorreo('');
69   };
70
71   const cambiarContrasena = (texto) ⇒ {
72     setContrasena(texto);
73     setErrorContrasena('');
74   };
```

Qué hace. `cambiarCorreo` y `cambiarContrasena` guardan el texto y además borran el error de ese campo. `cambiarTema` ya no está acá: viene del contexto.

> POR QUÉ ASÍ
>
> Si en `onChangeText` pusieras `setCorreo` directamente, el mensaje rojo quedaría visible mientras el usuario corrige. Borrar el error al escribir es lo que se espera de un buen formulario.
>
> Se declaran con `const nombre = () ⇒ {}`, igual que en el resto del proyecto. Dentro de un componente no hay diferencia práctica con `function nombre() {}`; lo importante es ser consistente.

5.10 Validar el formulario

screens/Login.js líneas 76-93

```
76   const validarFormulario = () => {
77     let esValido = true;
78
79     if (!correo.trim()) {
80       setErrorCorreo('Ingresá tu correo electrónico.');
81       esValido = false;
82     } else if (!FORMATO_CORREO.test(correo.trim())) {
83       setErrorCorreo('Revisá el formato del correo.');
84       esValido = false;
85     }
86
87     if (!contrasena) {
88       setErrorContrasena('Ingresá tu contraseña.');
89       esValido = false;
90     }
91
92     return esValido;
93   };
```

Qué hace. Revisa los dos campos, carga el mensaje que corresponda en cada uno y devuelve `true` solo si no encontró problemas.

> POR QUÉ ASÍ
>
> No usa `return` apenas encuentra el primer error. Usa la bandera `esValido` y sigue revisando, para mostrar los errores de los **dos** campos a la vez. Así el usuario no corrige uno, envía y recién ahí descubre el otro.
>
> `correo.trim()` quita los espacios del principio y del final, que el teclado del teléfono a veces agrega al autocompletar. A la contraseña no se le aplica `trim()` porque un espacio puede ser parte de ella.
>
> Los textos usan voseo («Ingresá», «Revisá») para coincidir con el diseño («¿No tenés cuenta?»).
>
> ¿Para qué revisar campos vacíos si el botón ya está deshabilitado? Porque hay otra forma de enviar: la tecla «Ir» del teclado llama a `iniciarSesion` desde el campo de contraseña (`onSubmitEditing`), aunque el botón esté gris. La validación es la red de seguridad para cualquier camino.

5.11 Guardar la preferencia de Recordarme

screens/Login.js líneas 95-101

```
95   const guardarPreferenciaRecordarme = async () => {
96     if (recordarme) {
97       await AsyncStorage.setItem(CLAVE_CORREO_RECORDADO, correo.trim());
98     } else {
99       await AsyncStorage.removeItem(CLAVE_CORREO_RECORDADO);
100    }
101  };
```

Qué hace. Si la casilla está marcada, guarda el correo; si no, borra cualquier correo guardado antes.

> POR QUÉ ASÍ
>
> La rama `else` importa: si alguien marcó Recordarme ayer y hoy lo desmarca, esperaría que el teléfono olvide su correo. Sin `removeItem` el correo viejo seguiría apareciendo.

5.12 Traducir los errores de Firebase

screens/Login.js — líneas 103–127

```
103   const mostrarErrorDeFirebase = (error) => {
104     switch (error.code) {
105       case 'auth/invalid-email':
106         setErrorCorreo('El formato del correo no es válido.');
107         break;
108       case 'auth/invalid-credential':
109       case 'auth/wrong-password':
110       case 'auth/user-not-found':
111         setErrorContrasena('La contraseña no coincide con ese correo.');
112         break;
113       case 'auth/too-many-requests':
114         setErrorContrasena('Demasiados intentos. Esperá unos minutos y probá de nuevo.');
115         break;
116       case 'auth/network-request-failed':
117         avisar('error', 'Sin conexión', 'Revisá el wifi o los datos del teléfono.');
118         break;
119       case 'auth/configuration-not-found':
120       case 'auth/admin-restricted-operation':
121       case 'auth/operation-not-allowed':
122         avisar('advertencia', 'No disponible', 'Este método de acceso no está habilitado en
Firebase.');
123         break;
124       default:
125         avisar('error', 'Error', `No pudimos iniciar sesión.\n\nCódigo: ${error.code ??
'desconocido'}`);
126     }
127   };
```

Qué hace. Recibe el error que lanza Firebase y, según su `code`, muestra el mensaje debajo del campo correspondiente o en una alerta.

> POR QUÉ ASÍ
>
> **Error en el campo o en alerta.** Si el problema es de un dato que el usuario escribió (correo inválido, contraseña incorrecta, demasiados intentos), el mensaje va pegado a ese campo, como en las vistas 1.2 y 1.6. Si no depende de lo que escribió (sin conexión, proveedor deshabilitado), va en una ventana con `avisar` (3.4): roja si es un error y ámbar si es una advertencia.
>
> **Casos agrupados.** Escribir varios `case` seguidos sin `break` hace que compartan el mismo código. Las versiones nuevas de Firebase devuelven `auth/invalid-credential` tanto para contraseña incorrecta como para usuario inexistente, a propósito: no revelar si un correo está registrado. Por eso el mensaje dice «no coincide con ese correo» sin decir cuál de los dos falló.

`auth/configuration-not-found` comparte el mensaje de «método no habilitado»: significa que en la consola de Firebase todavía no se activó Authentication (9.13).

Una sola función para los tres llamados a Firebase (ingresar, recuperar, invitado). Sin ella, el mismo `switch` estaría copiado tres veces.

`error.code ?? 'desconocido'`: el operador `??` usa el valor de la derecha solo si el de la izquierda es `null` o `undefined`.

5.13 Iniciar sesión

screens/Login.js líneas 129–140

```
129   const iniciarSesion = async () => {
130     if (!validarFormulario()) return;
131
132     setCargando(true);
133     try {
134       await guardarPreferenciaRecordarme();
135       await signInWithEmailAndPassword(auth, correo.trim(), contrasena);
136     } catch (error) {
137       mostrarErrorDeFirebase(error);
138       setCargando(false);
139     }
140   };
```

Qué hace. Si la validación falla, corta. Si pasa, activa `cargando`, guarda la preferencia de Recordarme y pide a Firebase que inicie sesión.

POR QUÉ ASÍ

`setCargando(false)` solo en el `catch`. Si el ingreso sale bien, el observador de `Navigation.js` reemplaza esta pantalla por Inicio y el Login se desmonta. Poner `setCargando(false)` en un `finally` intentaría actualizar un componente que ya no existe.

`async/await` con `try/catch` se lee de arriba hacia abajo como código normal. La alternativa con `.then().catch()` funciona igual pero anida más.

La preferencia se guarda *antes* del ingreso porque después la pantalla ya no existe.

5.14 Recuperar contraseña

screens/Login.js líneas 142–154

```
142   const recuperarContrasena = async () => {
143     if (!FORMATO_CORREO.test(correo.trim())) {
144       setErrorCorreo('Escribí tu correo y te mandamos un enlace para elegir una nueva.');
145       return;
146     }
147
148     try {
149       await sendPasswordResetEmail(auth, correo.trim());
150       avisar('exito', 'Revisá tu correo', `Te mandamos un enlace a ${correo.trim()} para que elijas
una contraseña nueva.`);
151     } catch (error) {
```

```
152       mostrarErrorDeFirebase(error);
153     }
154   };
```

Qué hace. Si el correo escrito no tiene formato válido, le pide al usuario que lo escriba. Si es válido, Firebase envía el enlace de restablecimiento y la app lo confirma.

> POR QUÉ ASÍ
>
> Reutiliza el campo de correo que ya está en pantalla en lugar de abrir otra vista. Es la forma más corta de hacerlo funcional sin crear la pantalla 1.4, que queda como siguiente paso.

5.15 Seguir sin cuenta y proveedores pendientes

screens/Login.js — líneas 156-168

```
156   const seguirSinCuenta = async () => {
157     setCargando(true);
158     try {
159       await signInAnonymously(auth);
160     } catch (error) {
161       mostrarErrorDeFirebase(error);
162       setCargando(false);
163     }
164   };
165
166   const avisarProximamente = (proveedor) => {
167     avisar('advertencia', 'Próximamente', `El ingreso con ${proveedor} todavía no está
disponible.`);
168   };
```

Qué hace. `seguirSinCuenta` crea una sesión anónima. `avisarProximamente` recibe el nombre del proveedor y arma el mensaje.

> POR QUÉ ASÍ
>
> La sesión anónima de Firebase es la forma estándar de dejar usar la app sin registro: el usuario recibe un identificador y más adelante esa cuenta se puede convertir en una normal. Como pasa por el mismo observador, la app muestra Inicio sin código extra.
>
> `avisarProximamente` recibe un parámetro para que una sola función sirva a los dos botones. En el JSX se llama con una flecha, `() => avisarProximamente('Google')`, porque `onPress={avisarProximamente('Google')}` la ejecutaría al dibujar la pantalla y no al tocar.

5.16 Esperar las fuentes

screens/Login.js — líneas 170-176

```
170   if (!fuentesCargadas) {
171     return (
```

```
172        <View style={[estilos.pantalla, estilos.centrado]}>
173          <ActivityIndicator size="large" color={colores.marca} />
174        </View>
175      );
176    }
```

Qué hace. Mientras las fuentes no cargaron, muestra solo un indicador centrado sobre el fondo del tema.

> **POR QUÉ ASÍ**
>
> Si se dibujara la pantalla antes, aparecería un instante con la fuente del sistema y después «saltaría» a Caveat e Inter.
>
> Este `if` está **después** de todos los hooks. Es una regla de React: los hooks se llaman siempre, en el mismo orden, en cada render. Un `return` antes de un `useState` haría que en algunos renders ese hook no se ejecute, y React lanza un error.
>
> Un arreglo de estilos, `[estilos.pantalla, estilos.centrado]`, combina los dos; si se repite una propiedad, gana el último.

6

Login: la interfaz (JSX)

6.1 Contenedores de la pantalla

screens/Login.js líneas 178–189

```
178   return (
179     <SafeAreaView style={estilos.pantalla}>
180       <StatusBar style={colores.barraEstado} />
181       <KeyboardAvoidingView
182         style={estilos.pantalla}
183         behavior={Platform.OS === 'ios' ? 'padding' : undefined}
184       >
185         <ScrollView
186           contentContainerStyle={estilos.contenido}
187           keyboardShouldPersistTaps="handled"
188           showsVerticalScrollIndicator={false}
189         >
```

Qué hace. Arma cuatro capas: zona segura, barra de estado, ajuste por teclado y desplazamiento.

> POR QUÉ ASÍ
>
> **SafeAreaView** evita que el contenido quede debajo de la muesca o del reloj.
>
> **StatusBar** de `expo-status-bar` cambia el color de los íconos del sistema: claros sobre fondo oscuro y al revés.
>
> **KeyboardAvoidingView** con `padding` solo en iOS: iOS no mueve nada por su cuenta cuando aparece el teclado, así que hay que agregar el espacio a mano. En Android se deja `undefined` y el sistema ajusta la ventana. Si en tu teléfono Android el teclado tapa el campo, probá cambiar `undefined` por `'height'`.
>
> **ScrollView y no View.** La pantalla mide más que un teléfono chico, y con el teclado abierto mucho más. Con un View el botón quedaría fuera de alcance. Se descarta FlatList porque es para listas largas de datos repetidos, no para un formulario.
>
> `contentContainerStyle` (no `style`) es el que aplica al contenido interno del ScrollView, donde va el padding. `keyboardShouldPersistTaps="handled"` permite tocar un botón con el teclado abierto al primer toque; sin eso, el primer toque solo cierra el teclado.

6.2 Barra superior

screens/Login.js líneas 190–198

```
190           <View style={estilos.barraSuperior}>
191             <Pressable style={estilos.chipInvitado} onPress={seguirSinCuenta} disabled={cargando}>
192               <Text style={estilos.textoChip}>Seguir sin cuenta</Text>
193               <Feather name="chevron-right" size={16} color={colores.textoSecundario} />
194             </Pressable>
```

```
195            <Pressable style={estilos.botonTema} onPress={cambiarTema}>
196              <Ionicons name="contrast" size={20} color={colores.texto} />
197            </Pressable>
198          </View>
```

Qué hace. A la izquierda, el chip «Seguir sin cuenta»; a la derecha, el botón de tema, que llama a `cambiarTema` del contexto.

> POR QUÉ ASÍ
>
> Los dos quedan en los extremos por `justifyContent: 'space-between'` en `barraSuperior`. No hace falta calcular márgenes.
>
> `disabled={cargando}` evita que alguien toque «Seguir sin cuenta» mientras ya se está iniciando sesión, lo que dispararía dos operaciones a la vez.

6.3 Marca, título y subtítulo

screens/Login.js — líneas 200–206

```
200          <View style={estilos.marca}>
201            <Image source={logo} style={estilos.logo} resizeMode="contain" />
202            <Text style={estilos.lomitas}>LAS LOMITAS</Text>
203          </View>
204
205          <Text style={estilos.titulo}>Hola de nuevo</Text>
206          <Text style={estilos.subtitulo}>Qué bueno verte.</Text>
```

Qué hace. Muestra el logo del tema activo (`logo`, que entrega el contexto), la leyenda «LAS LOMITAS», el título manuscrito y la bienvenida.

> POR QUÉ ASÍ
>
> `resizeMode="contain"` hace que la imagen entre completa en el cuadro de 92 × 95 sin deformarse. Esa medida mantiene la proporción del PNG original (512 × 527).
>
> «LAS LOMITAS» es texto y no parte de la imagen, porque en el diseño va con letra separada (`letterSpacing`) y cambia de color con el tema.
>
> El título y el subtítulo quedan centrados por `textAlign: 'center'` en sus estilos (7.2). Se centra el *texto* y no el contenedor: un `<Text>` dentro de una columna ya ocupa todo el ancho, así que `alignItems: 'center'` en el padre no alcanzaría para un texto de varias líneas.

6.4 Campo de correo

screens/Login.js — líneas 208–241

```
208          <Text style={estilos.etiqueta}>Correo electrónico</Text>
209          <View
210            style={[
211              estilos.campo,
212              campoActivo === 'correo' && estilos.campoActivo,
213              errorCorreo !== '' && estilos.campoConError,
```

```
            ]}
          >
            <Feather name="mail" size={20} color={colores.textoSecundario} />
            <TextInput
              style={estilos.entrada}
              placeholder="tunombre@correo.com"
              placeholderTextColor={colores.textoTenue}
              value={correo}
              onChangeText={cambiarCorreo}
              onFocus={() => setCampoActivo('correo')}
              onBlur={() => setCampoActivo(null)}
              keyboardType="email-address"
              autoCapitalize="none"
              autoCorrect={false}
              autoComplete="email"
              returnKeyType="next"
              editable={!cargando}
            />
          </View>
          {errorCorreo !== '' && (
            <View style={estilos.filaError}>
              <Feather name="alert-circle" size={14} color={colores.peligro} />
              <Text style={estilos.textoError}>{errorCorreo}</Text>
            </View>
          )}
          {campoActivo === 'correo' && errorCorreo === '' && (
            <Text style={estilos.ayuda}>Usá el correo con el que te registraste.</Text>
          )}
```

Qué hace. Un `View` con borde que contiene el ícono y el `TextInput`. Debajo, solo si hay error, una fila con ícono y mensaje; y si el campo está enfocado y sin error, la ayuda de la vista 1.3.

> POR QUÉ ASÍ
>
> **El borde va en el View, no en el TextInput.** El diseño pone el ícono *dentro* del recuadro. Un TextInput no puede contener otros componentes, así que el View dibuja la caja y el TextInput ocupa el espacio libre con `flex: 1`.
>
> **Estilos condicionales en arreglo.** `campoActivo === 'correo' && estilos.campoActivo` vale el estilo o `false`, y React Native ignora los `false` dentro de un arreglo de estilos. El error va último para que el rojo gane sobre el ámbar.
>
> **Componente controlado.** `value={correo}` más `onChangeText` hacen que el estado sea la única fuente de verdad. Por eso el `useEffect` puede completar el campo con `setCorreo`.
>
> `keyboardType="email-address"` muestra la @ en el teclado, `autoCapitalize="none"` evita la mayúscula inicial y `autoCorrect={false}` evita que el corrector cambie el correo. `autoComplete="email"` deja que el teléfono sugiera correos guardados.
>
> **Ayuda de la vista 1.3.** «Usá el correo con el que te registraste.» aparece solo con dos condiciones: el campo de correo tiene el foco y no hay error. Si hay error, el mensaje rojo ocupa ese lugar; mostrar los dos a la vez sería confuso. Usa su propio estilo, `ayuda`, en gris y no en rojo, porque es una indicación y no un problema.

`{errorCorreo !== '' && (...)}` es renderizado condicional. Se compara contra `''` en lugar de escribir `{errorCorreo && ...}` porque si `errorCorreo` vale `''`, la expresión devuelve ese texto vacío, y React Native exige que todo texto esté dentro de `<Text>`. Comparar contra `''` garantiza un booleano.

6.5 Campo de contraseña

screens/Login.js — líneas 243-280

```
243          <Text style={estilos.etiqueta}>Contraseña</Text>
244          <View
245            style={[
246              estilos.campo,
247              campoActivo === 'contrasena' && estilos.campoActivo,
248              errorContrasena !== '' && estilos.campoConError,
249            ]}
250          >
251            <Feather name="lock" size={20} color={colores.textoSecundario} />
252            <TextInput
253              style={estilos.entrada}
254              placeholder="Tu contraseña"
255              placeholderTextColor={colores.textoTenue}
256              value={contrasena}
257              onChangeText={cambiarContrasena}
258              onFocus={() => setCampoActivo('contrasena')}
259              onBlur={() => setCampoActivo(null)}
260              secureTextEntry={!mostrarContrasena}
261              autoCapitalize="none"
262              autoComplete="password"
263              returnKeyType="go"
264              onSubmitEditing={iniciarSesion}
265              editable={!cargando}
266            />
267            <Pressable onPress={() => setMostrarContrasena(!mostrarContrasena)} hitSlop={10}>
268              <Feather
269                name={mostrarContrasena ? 'eye-off' : 'eye'}
270                size={20}
271                color={colores.textoSecundario}
272              />
273            </Pressable>
274          </View>
275          {errorContrasena !== '' && (
276            <View style={estilos.filaError}>
277              <Feather name="alert-circle" size={14} color={colores.peligro} />
278              <Text style={estilos.textoError}>{errorContrasena}</Text>
279            </View>
280          )}
```

Qué hace. Igual que el correo, más el botón del ojo a la derecha.

POR QUÉ ASÍ

`secureTextEntry={!mostrarContrasena}`: el campo oculta los caracteres cuando `mostrarContrasena` es `false`. El ícono cambia entre `eye` y `eye-off` con un operador ternario.

`hitSlop={10}` agranda 10 puntos el área táctil del ojo sin cambiar su tamaño visible. Un ícono de 20 puntos es difícil de acertar con el dedo.

`returnKeyType="go"` con `onSubmitEditing={iniciarSesion}` hace que la tecla del teclado diga «Ir» e inicie sesión.

6.6 Recordarme y ¿Olvidaste tu contraseña?

screens/Login.js — líneas 282–292

```
282           <View style={estilos.filaOpciones}>
283             <Pressable style={estilos.recordarme} onPress={() => setRecordarme(!recordarme)}>
284               <View style={[estilos.casilla, recordarme && estilos.casillaMarcada]}>
285                 {recordarme && <Feather name="check" size={16} color={colores.textoSobreMarca} />}
286               </View>
287               <Text style={estilos.textoRecordarme}>Recordarme</Text>
288             </Pressable>
289             <Pressable onPress={recuperarContrasena}>
290               <Text style={estilos.enlace}>¿Olvidaste tu contraseña?</Text>
291             </Pressable>
292           </View>
```

Qué hace. Una casilla dibujada a mano que se pinta de ámbar con un tilde cuando está marcada, y el enlace de recuperación a la derecha.

> **POR QUÉ ASÍ**
>
> React Native no trae un componente de casilla multiplataforma (el viejo `CheckBox` fue retirado). Un `View` con borde más un ícono `check` resuelve el diseño sin dependencias.
>
> El `Pressable` envuelve la casilla *y* el texto, así se puede tocar cualquiera de los dos.

6.7 Botón Iniciar sesión

screens/Login.js — líneas 294–315

```
294           <Pressable
295             style={({ pressed }) => [
296               estilos.botonPrincipal,
297               !formularioCompleto && estilos.botonDeshabilitado,
298               pressed && estilos.presionado,
299             ]}
300             onPress={iniciarSesion}
301             disabled={cargando || !formularioCompleto}
302           >
303             {cargando ? (
304               <View style={estilos.filaCargando}>
305                 <ActivityIndicator color={colores.textoSobreMarca} />
306                 <Text style={estilos.textoBotonPrincipal}>Ingresando…</Text>
307               </View>
308             ) : (
309               <Text
310                 style={[estilos.textoBotonPrincipal, !formularioCompleto &&
estilos.textoBotonDeshabilitado]}
311               >
312                 Iniciar sesión
313               </Text>
314             )}
315           </Pressable>
```

Qué hace. Tiene tres aspectos. Gris y sin respuesta mientras falte el correo o la contraseña (vista 1.3). Ámbar con «Iniciar sesión» cuando los dos campos tienen algo. Ámbar con indicador y «Ingresando…» mientras `cargando` es verdadero (vista 1.5).

> **POR QUÉ ASÍ**
>
> `style` recibe una función: Pressable la llama con `{ pressed }` y devuelve un arreglo. Cada condición agrega su estilo solo si se cumple: `botonDeshabilitado` cuando el formulario está incompleto y `presionado` (opacidad 0,85) mientras el dedo está apoyado.
>
> `disabled={cargando || !formularioCompleto}` une las dos razones para no responder: ya hay una operación en curso, o todavía no hay nada que enviar. El aspecto gris es solo visual; lo que realmente bloquea el toque es `disabled`. Por eso van siempre juntos.
>
> El texto también cambia de color con `textoBotonDeshabilitado`. Si solo cambiara el fondo, quedaría texto casi negro sobre gris oscuro y no se leería en el tema oscuro.
>
> El arreglo del `style` se escribe en varias líneas porque ya tiene tres elementos. Con uno o dos, una línea alcanza; con más, una por elemento se lee mejor.

6.8 Mientras carga: aviso o alternativas de ingreso

screens/Login.js — líneas 317–343

```
317           {cargando ? (
318             <Text style={estilos.textoVerificando}>Un momento, estamos verificando tus datos.
</Text>
319           ) : (
320             <>
321               <View style={estilos.separador}>
322                 <View style={estilos.linea} />
323                 <Text style={estilos.textoSeparador}>o</Text>
324                 <View style={estilos.linea} />
325               </View>
326
327               <Pressable
328                 style={({ pressed }) => [estilos.botonSocial, estilos.botonGoogle, pressed &&
estilos.presionado]}
329                 onPress={() => avisarProximamente('Google')}
330               >
331                 <FontAwesome name="google" size={20} color="#4285F4" />
332                 <Text style={[estilos.textoBotonSocial, estilos.textoGoogle]}>Continuar con
Google</Text>
333               </Pressable>
334
335               <Pressable
336                 style={({ pressed }) => [estilos.botonSocial, estilos.botonApple, pressed &&
estilos.presionado]}
337                 onPress={() => avisarProximamente('Apple')}
338               >
339                 <FontAwesome name="apple" size={22} color="#FFFFFF" />
340                 <Text style={[estilos.textoBotonSocial, estilos.textoApple]}>Continuar con
Apple</Text>
341               </Pressable>
```

```
342            </>
343          )}
```

Qué hace. Si `cargando` es verdadero, debajo del botón aparece «Un momento, estamos verificando tus datos.». Si no, aparecen el separador «o» y los botones de Google y Apple. Nunca se ven las dos cosas a la vez.

> POR QUÉ ASÍ
>
> **Ternario y no dos `&&`.** Cuando hay dos alternativas excluyentes, `condición ? A : B` lo dice de una vez. Con `{cargando && A}` y `{!cargando && B}` tendrías la misma condición escrita dos veces.
>
> **Fragmento `<> ... </>`.** Cada rama del ternario tiene que devolver un solo elemento, y la segunda tiene tres (separador, Google y Apple). Un `<View>` serviría, pero agregaría una caja extra al diseño. El fragmento agrupa sin dibujar nada.
>
> **Por qué ocultar los otros ingresos.** En la vista 1.5 la pantalla se simplifica mientras se verifica: no tiene sentido ofrecer Google o Apple cuando ya se está entrando con correo, y así nadie inicia dos ingresos a la vez.
>
> **Separador.** Las dos líneas tienen `flex: 1`: se reparten por igual el espacio que deja la letra, en cualquier ancho de pantalla.
>
> **Botones sociales.** Comparten `botonSocial` (alto, radio, centrado) y cada uno suma su diferencia. Los colores de Google y Apple están fijos y no salen del tema, porque las marcas exigen sus colores sin importar el modo de la app.

6.9 Pie: registro y aviso legal

screens/Login.js — líneas 345-364

```
345          <View style={estilos.espaciador} />
346
347          <View style={estilos.filaRegistro}>
348            <Text style={estilos.textoPie}>¿No tenés cuenta? </Text>
349            <Pressable onPress={() => navigation.navigate('Registro')}>
350              <Text style={estilos.enlaceRegistro}>Registrate</Text>
351            </Pressable>
352          </View>
353
354          {!cargando && (
355            <Text style={estilos.legal}>
356              Al continuar aceptás los <Text style={estilos.subrayado}>Términos y Condiciones</Text>
y la{' '}
357              <Text style={estilos.subrayado}>Política de privacidad</Text>.
358            </Text>
359          )}
360        </ScrollView>
361      </KeyboardAvoidingView>
362    </SafeAreaView>
363  );
364 }
```

Qué hace. Un espaciador, el enlace a Registro y el texto legal, que se oculta mientras carga. Después se cierran los contenedores y el componente.

POR QUÉ ASÍ

Espaciador con `flex: 1`. Mientras carga desaparecen el separador y los botones sociales, y el contenido queda más corto que la pantalla. El espaciador ocupa todo el espacio sobrante y empuja «¿No tenés cuenta? Registrate» al fondo, como en la vista 1.5. Cuando el contenido es más alto que la pantalla, el espaciador mide cero y no cambia nada. Funciona porque el contenido del ScrollView tiene `flexGrow: 1` (7.1).

Textos anidados. En React Native, un `<Text>` dentro de otro hereda su estilo y fluye en la misma línea. Así se subrayan solo «Términos y Condiciones» y «Política de privacidad» sin cortar el párrafo.

`{' '}` agrega un espacio que JSX eliminaría al final de la línea.

«¿No tenés cuenta?» y «Registrate» van en una fila de dos elementos y no anidados, porque solo «Registrate» tiene que responder al toque.

7

Login: los estilos

Los estilos están en una **función** que recibe la paleta (`c`) y devuelve un `StyleSheet` . Es la pieza que permite cambiar de tema: el mismo código produce dos juegos de estilos.

> POR QUÉ UNA FUNCIÓN Y NO STYLESHEET.CREATE DIRECTO
>
> El proyecto original usaba `const styles = StyleSheet.create({...})` fuera del componente. Eso funciona con colores fijos, pero no puede leer el tema, que es un estado. Las alternativas eran escribir colores inline (`style={[estilos.campo, { borderColor: colores.borde }]}`) en cada elemento, que ensucia el JSX, o usar una librería de temas. La función más `useMemo` mantiene el JSX limpio sin agregar dependencias.
>
> Se declara con `function` al final del archivo: las declaraciones de función se «elevan» (hoisting), así que el componente la puede usar aunque esté escrita más abajo.

7.1 Base de la pantalla

screens/Login.js — líneas 366-381

```
366 function crearEstilos(c) {
367   return StyleSheet.create({
368     pantalla: {
369       flex: 1,
370       backgroundColor: c.fondo,
371     },
372     centrado: {
373       justifyContent: 'center',
374       alignItems: 'center',
375     },
376     contenido: {
377       flexGrow: 1,
378       paddingHorizontal: 28,
379       paddingTop: 12,
380       paddingBottom: 24,
381     },
```

> POR QUÉ ASÍ
>
> `flex: 1` hace que la pantalla ocupe todo el alto disponible. En el contenido del ScrollView se usa `flexGrow: 1` y no `flex: 1` : deja que el contenido crezca para llenar la pantalla pero también que sea más alto que ella y se pueda desplazar.
>
> `paddingHorizontal: 28` es el margen lateral del diseño (`--pad-x: 28px`).

7.2 Barra superior, marca y títulos

screens/Login.js líneas 382–441

```
382     barraSuperior: {
383       flexDirection: 'row',
384       justifyContent: 'space-between',
385       alignItems: 'center',
386     },
387     chipInvitado: {
388       flexDirection: 'row',
389       alignItems: 'center',
390       gap: 4,
391       height: 38,
392       paddingHorizontal: 16,
393       borderRadius: 19,
394       borderWidth: 1,
395       borderColor: c.borde,
396       backgroundColor: c.superficie,
397     },
398     textoChip: {
399       fontFamily: 'Inter_500Medium',
400       fontSize: 14,
401       color: c.textoSecundario,
402     },
403     botonTema: {
404       width: 40,
405       height: 40,
406       borderRadius: 20,
407       borderWidth: 1,
408       borderColor: c.borde,
409       backgroundColor: c.superficie,
410       justifyContent: 'center',
411       alignItems: 'center',
412     },
413     marca: {
414       alignItems: 'center',
415       marginTop: 8,
416     },
417     logo: {
418       width: 92,
419       height: 95,
420     },
421     lomitas: {
422       fontFamily: 'Inter_500Medium',
423       fontSize: 11,
424       letterSpacing: 4,
425       color: c.textoSecundario,
426       marginTop: 6,
427     },
428     titulo: {
429       fontFamily: 'Caveat_700Bold',
430       fontSize: 40,
431       color: c.titulo,
432       textAlign: 'center',
433       marginTop: 24,
434     },
435     subtitulo: {
436       fontFamily: 'Inter_400Regular',
437       fontSize: 15,
438       color: c.textoSecundario,
439       textAlign: 'center',
440       marginBottom: 8,
441     },
```

POR QUÉ ASÍ

`textAlign: 'center'` en `titulo` y `subtitulo` centra «Hola de nuevo» y «Qué bueno verte.» debajo del logo. Si quisieras volver a alinearlos a la izquierda como en la captura original, basta con borrar esa línea en los dos estilos.

`gap` separa los hijos de una fila o columna sin poner márgenes en cada uno. React Native lo admite desde la versión 0.71.

Los botones redondos usan `borderRadius` igual a la mitad del alto (chip de 38 → 19; botón de 40 → 20). Es la forma de obtener una «píldora» o un círculo.

En React Native los números de tamaño son puntos independientes de la densidad, sin unidad. `fontSize: 40` se ve del mismo tamaño físico en cualquier teléfono.

Con fuentes cargadas por `useFonts` no se usa `fontWeight`: cada peso es un archivo distinto, así que el peso se elige por nombre (`Inter_600SemiBold`). En Android, combinar `fontFamily` personalizada con `fontWeight: 'bold'` puede ignorar la fuente.

7.3 Campos y errores

screens/Login.js líneas 442–491

```
442     etiqueta: {
443       fontFamily: 'Inter_600SemiBold',
444       fontSize: 13.5,
445       color: c.texto,
446       marginTop: 16,
447       marginBottom: 8,
448     },
449     campo: {
450       flexDirection: 'row',
451       alignItems: 'center',
452       gap: 12,
453       height: 52,
454       paddingHorizontal: 16,
455       borderRadius: 14,
456       borderWidth: 1.5,
457       borderColor: c.borde,
458       backgroundColor: c.superficie,
459     },
460     campoActivo: {
461       borderColor: c.marca,
462       backgroundColor: c.superficieActiva,
463     },
464     campoConError: {
465       borderColor: c.peligro,
466     },
467     entrada: {
468       flex: 1,
469       height: '100%',
470       fontFamily: 'Inter_400Regular',
471       fontSize: 16,
472       color: c.texto,
473     },
474     filaError: {
475       flexDirection: 'row',
476       alignItems: 'center',
477       gap: 6,
```

```
478       marginTop: 6,
479     },
480     textoError: {
481       flex: 1,
482       fontFamily: 'Inter_400Regular',
483       fontSize: 12.5,
484       color: c.peligro,
485     },
486     ayuda: {
487       fontFamily: 'Inter_400Regular',
488       fontSize: 12.5,
489       color: c.textoSecundario,
490       marginTop: 6,
491     },
```

POR QUÉ ASÍ

Medidas tomadas de los tokens del diseño: alto 52 (`--h-field`), radio 14 (`--r-field`), borde 1,5 (`--bw`), texto de 16 (`--fs-input`).

`campoActivo` y `campoConError` solo tienen las propiedades que cambian. Al combinarse en arreglo con `campo` , pisan esas y conservan el resto.

`textoError` tiene `flex: 1` para que un mensaje largo haga salto de línea en lugar de salirse de la pantalla.

`ayuda` copia el tamaño y la separación de `textoError` (12,5 y 6) para que la ayuda y el error ocupen exactamente el mismo lugar y la pantalla no salte al pasar de uno a otro.

7.4 Opciones y casilla

screens/Login.js — líneas 492–527

```
492     filaOpciones: {
493       flexDirection: 'row',
494       justifyContent: 'space-between',
495       alignItems: 'center',
496       marginTop: 16,
497     },
498     recordarme: {
499       flexDirection: 'row',
500       alignItems: 'center',
501       gap: 10,
502     },
503     casilla: {
504       width: 22,
505       height: 22,
506       borderRadius: 6,
507       borderWidth: 1.5,
508       borderColor: c.borde,
509       backgroundColor: c.superficie,
510       justifyContent: 'center',
511       alignItems: 'center',
512     },
513     casillaMarcada: {
514       borderColor: c.marca,
515       backgroundColor: c.marca,
516     },
517     textoRecordarme: {
518       fontFamily: 'Inter_400Regular',
```

```
519       fontSize: 14,
520       color: c.texto,
521     },
522     enlace: {
523       fontFamily: 'Inter_600SemiBold',
524       fontSize: 14,
525       color: c.marca,
526       textDecorationLine: 'underline',
527     },
```

POR QUÉ ASÍ

`filaOpciones` repite el patrón de la barra superior: fila con `space-between`. Reconocer patrones repetidos es lo que hace que un archivo largo se vuelva fácil de recordar.

`textDecorationLine: 'underline'` marca los enlaces como en el diseño.

7.5 Botón principal

screens/Login.js líneas 528–563

```
528     botonPrincipal: {
529       height: 54,
530       borderRadius: 14,
531       backgroundColor: c.marca,
532       justifyContent: 'center',
533       alignItems: 'center',
534       marginTop: 24,
535     },
536     botonDeshabilitado: {
537       backgroundColor: c.superficie,
538       borderWidth: 1,
539       borderColor: c.borde,
540     },
541     presionado: {
542       opacity: 0.85,
543     },
544     filaCargando: {
545       flexDirection: 'row',
546       alignItems: 'center',
547       gap: 10,
548     },
549     textoBotonPrincipal: {
550       fontFamily: 'Inter_600SemiBold',
551       fontSize: 16,
552       color: c.textoSobreMarca,
553     },
554     textoBotonDeshabilitado: {
555       color: c.textoTenue,
556     },
557     textoVerificando: {
558       fontFamily: 'Inter_400Regular',
559       fontSize: 13.5,
560       color: c.textoSecundario,
561       textAlign: 'center',
562       marginTop: 12,
563     },
```

POR QUÉ ASÍ

Alto 54 (`--h-btn`), centrado en los dos ejes con `justifyContent` y `alignItems` . El texto del botón es casi negro (`textoSobreMarca`) porque el blanco sobre ámbar no alcanza el contraste mínimo de lectura.

`botonDeshabilitado` reemplaza el ámbar por el color de los campos y le suma un borde, como en las vistas 1.3 y 1.6. Solo define lo que cambia; alto, radio y centrado siguen saliendo de `botonPrincipal` .

`textoVerificando` usa el gris secundario y va centrado debajo del botón, con la misma familia que el resto de los textos chicos.

7.6 Separador, botones sociales y espaciador

screens/Login.js líneas 564-610

```
564     separador: {
565       flexDirection: 'row',
566       alignItems: 'center',
567       gap: 16,
568       marginVertical: 20,
569     },
570     linea: {
571       flex: 1,
572       height: 1,
573       backgroundColor: c.divisor,
574     },
575     textoSeparador: {
576       fontFamily: 'Inter_400Regular',
577       fontSize: 14,
578       color: c.textoSecundario,
579     },
580     botonSocial: {
581       flexDirection: 'row',
582       justifyContent: 'center',
583       alignItems: 'center',
584       gap: 10,
585       height: 52,
586       borderRadius: 14,
587       borderWidth: 1,
588       marginBottom: 12,
589     },
590     botonGoogle: {
591       backgroundColor: '#FFFFFF',
592       borderColor: '#E6E2D8',
593     },
594     botonApple: {
595       backgroundColor: c.fondoApple,
596       borderColor: c.bordeApple,
597     },
598     textoBotonSocial: {
599       fontFamily: 'Inter_500Medium',
600       fontSize: 16,
601     },
602     textoGoogle: {
603       color: '#161616',
604     },
605     textoApple: {
606       color: '#FFFFFF',
607     },
608     espaciador: {
609       flex: 1,
610     },
```

POR QUÉ ASÍ

`marginVertical: 20` es un atajo para `marginTop` y `marginBottom` iguales.

`botonSocial` tiene `marginBottom: 12` para separar los dos botones entre sí.

`espaciador` tiene una sola propiedad, `flex: 1`. Es un patrón muy común en React Native: una caja vacía que se estira para empujar lo que viene después hacia el final.

7.7 Pie

screens/Login.js líneas 611-640

```
611     filaRegistro: {
612       flexDirection: 'row',
613       justifyContent: 'center',
614       alignItems: 'center',
615       marginTop: 16,
616     },
617     textoPie: {
618       fontFamily: 'Inter_400Regular',
619       fontSize: 15,
620       color: c.textoSecundario,
621     },
622     enlaceRegistro: {
623       fontFamily: 'Inter_600SemiBold',
624       fontSize: 15,
625       color: c.marca,
626       textDecorationLine: 'underline',
627     },
628     legal: {
629       fontFamily: 'Inter_400Regular',
630       fontSize: 12.5,
631       lineHeight: 19,
632       color: c.textoTenue,
633       textAlign: 'center',
634       marginTop: 12,
635     },
636     subrayado: {
637       textDecorationLine: 'underline',
638     },
639   });
640 }
```

POR QUÉ ASÍ

`lineHeight: 19` en el texto legal da aire entre las dos líneas del párrafo.
`textAlign: 'center'` centra cada línea dentro del ancho disponible.

8

Componente CampoTexto

En el Login, cada campo son unas 30 líneas: el View con borde, el ícono, el TextInput con sus eventos, el ojo y el mensaje de error. Con dos campos se puede escribir a mano. El Registro tiene seis. Copiar el bloque seis veces haría un archivo enorme y, si después cambia el borde, habría que corregirlo en seis lugares.

La regla práctica es: **cuando un bloque se repite tres veces o más, se convierte en componente**. `src/componentes/CampoTexto.js` encapsula todo lo que tienen en común los campos, y cada uso solo dice lo que cambia.

El Login quedó escrito a mano a propósito: así podés comparar las dos formas. Como ejercicio, podés pasar sus dos campos a `CampoTexto`.

8.1 Las props del componente

src/componentes/CampoTexto.js — líneas 1–21

```
import React, { useMemo, useState } from 'react';
import { Pressable, StyleSheet, Text, TextInput, View } from 'react-native';
import { Feather } from '@expo/vector-icons';
import { useTema } from '../tema/TemaContext';

export default function CampoTexto({
  etiqueta,
  opcional = false,
  icono,
  prefijo,
  valor,
  alCambiar,
  error = '',
  mensajeExito = '',
  ayuda = '',
  ayudaAlEnfocar = '',
  valido = false,
  seguro = false,
  estiloContenedor,
  ...propsEntrada
}) {
```

Qué hace. Declara qué datos recibe el componente. Cada uno con un valor por defecto, así quien lo usa solo escribe los que necesita.

PROP	PARA QUÉ
`etiqueta`, `opcional`	El texto de arriba; si es opcional agrega «· opcional».
`icono`, `prefijo`	El ícono de Feather a la izquierda y, si hace falta, algo más antes del texto (el «AR +54» del teléfono).
`valor`, `alCambiar`	El estado que muestra y la función que lo actualiza. El campo no guarda el texto: lo guarda la pantalla.

PROP	PARA QUÉ
`error`, `mensajeExito`, `ayuda`, `ayudaAlEnfocar`	Los cuatro tipos de mensaje que pueden aparecer debajo.
`valido`	Pinta el borde verde, como en la vista 2.2.
`seguro`	Lo convierte en campo de contraseña con su botón de ojo.
`estiloContenedor`	Estilo extra para la caja externa. Lo usa la fila de nombre y apellido para que cada campo ocupe la mitad.
`...propsEntrada`	Todo lo demás (placeholder, keyboardType, maxLength…) pasa directo al TextInput.

POR QUÉ ASÍ

Valores por defecto en la desestructuración. `error = ''` significa «si no me pasan error, vale texto vacío». Así el código interno puede comparar con `≠ ''` sin preguntar antes si existe.

El parámetro *rest* (`...propsEntrada`). Junta en un objeto todas las props que no se nombraron antes. Evita declarar una por una las decenas de props que acepta un TextInput.

Nombres en español (`valor`, `alCambiar`) para distinguir las props de nuestro componente de las de React Native (`value`, `onChangeText`). Cuando ves `alCambiar`, sabés que es un componente propio.

8.2 Estado interno y qué mensaje mostrar

src/componentes/CampoTexto.js líneas 22-36

```
22   const { colores } = useTema();
23   const estilos = useMemo(() ⇒ crearEstilos(colores), [colores]);
24   const [enfocado, setEnfocado] = useState(false);
25   const [mostrar, setMostrar] = useState(false);
26
27   let mensaje = null;
28   if (error ≠ '') {
29     mensaje = { texto: error, icono: 'alert-circle', estilo: estilos.textoError, color:
colores.peligro };
30   } else if (mensajeExito ≠ '') {
31     mensaje = { texto: mensajeExito, icono: 'check-circle', estilo: estilos.textoExito, color:
colores.exito };
32   } else if (enfocado && ayudaAlEnfocar ≠ '') {
33     mensaje = { texto: ayudaAlEnfocar, estilo: estilos.textoAyuda };
34   } else if (ayuda ≠ '') {
35     mensaje = { texto: ayuda, estilo: estilos.textoAyuda };
36   }
```

Qué hace. Lee la paleta del contexto, crea sus estilos y guarda dos estados propios: si el campo tiene el foco y si la contraseña está visible. Después decide cuál de los

mensajes mostrar.

> POR QUÉ ASÍ
>
> **Qué estado es del componente y cuál de la pantalla.** El texto escrito le importa a la pantalla (lo valida y lo envía), así que vive en la pantalla. El foco y el ojo solo le importan al campo, así que viven en el campo. Es la diferencia con el Login, donde `campoActivo` y `mostrarContrasena` eran estados de la pantalla.
>
> **Prioridad de los mensajes.** Solo se ve uno a la vez. El orden del `if / else if` es la prioridad: error, después éxito, después la ayuda que aparece al enfocar y por último la ayuda fija. Por eso un error siempre tapa a la ayuda.
>
> **`let` y no `const`.** `mensaje` empieza en `null` y se le asigna un valor después. `const` no permite reasignar.
>
> Se guarda el *objeto estilo* dentro de `mensaje` para que el JSX no tenga que repetir el mismo `if` para elegir el color.

8.3 La interfaz del campo

src/componentes/CampoTexto.js — líneas 38–78

```
38   return (
39     <View style={estiloContenedor}>
40       <Text style={estilos.etiqueta}>
41         {etiqueta}
42         {opcional && <Text style={estilos.opcional}> · opcional</Text>}
43       </Text>
44       <View
45         style={[
46           estilos.campo,
47           valido && estilos.campoValido,
48           enfocado && estilos.campoActivo,
49           error !== '' && estilos.campoConError,
50         ]}
51       >
52         {icono && <Feather name={icono} size={20} color={colores.textoSecundario} />}
53         {prefijo}
54         <TextInput
55           style={estilos.entrada}
56           placeholderTextColor={colores.textoTenue}
57           value={valor}
58           onChangeText={alCambiar}
59           onFocus={() => setEnfocado(true)}
60           onBlur={() => setEnfocado(false)}
61           secureTextEntry={seguro && !mostrar}
62           {...propsEntrada}
63         />
64         {seguro && (
65           <Pressable onPress={() => setMostrar(!mostrar)} hitSlop={10}>
66             <Feather name={mostrar ? 'eye-off' : 'eye'} size={20} color={colores.textoSecundario}
/>
67           </Pressable>
68         )}
69       </View>
70       {mensaje && (
71         <View style={estilos.filaMensaje}>
72           {mensaje.icono && <Feather name={mensaje.icono} size={14} color={mensaje.color} />}
73           <Text style={mensaje.estilo}>{mensaje.texto}</Text>
```

```
        </View>
      )}
    </View>
  );
}
```

POR QUÉ ASÍ

El mismo patrón del Login (6.4): un View con borde que contiene ícono, TextInput y ojo. Los estilos condicionales van en arreglo, y el error va último para que su rojo gane sobre el verde y el ámbar.

`{prefijo}` es un «hueco» para JSX: quien usa el componente puede pasar cualquier elemento y se dibuja ahí. Es la misma idea que `children`, pero con nombre.

`secureTextEntry={seguro && !mostrar}`. Oculta los caracteres solo si es un campo seguro y el ojo está cerrado.

`{...propsEntrada}` **va después** de las props fijas del TextInput. Si una se repitiera, gana la última escrita. Así quien usa el componente puede, por ejemplo, cambiar `placeholderTextColor`.

Etiqueta con texto anidado. «· opcional» es un `<Text>` dentro de otro, así comparte la línea pero con otro estilo, igual que los enlaces del aviso legal del Login.

8.4 Estilos del campo

src/componentes/CampoTexto.js — líneas 80-146

```
function crearEstilos(c) {
  return StyleSheet.create({
    etiqueta: {
      fontFamily: 'Inter_600SemiBold',
      fontSize: 13.5,
      color: c.texto,
      marginTop: 16,
      marginBottom: 8,
    },
    opcional: {
      fontFamily: 'Inter_400Regular',
      color: c.textoTenue,
    },
    campo: {
      flexDirection: 'row',
      alignItems: 'center',
      gap: 12,
      height: 52,
      paddingHorizontal: 16,
      borderRadius: 14,
      borderWidth: 1.5,
      borderColor: c.borde,
      backgroundColor: c.superficie,
    },
    campoValido: {
      borderColor: c.exito,
    },
    campoActivo: {
      borderColor: c.marca,
      backgroundColor: c.superficieActiva,
```

```
    },
    campoConError: {
      borderColor: c.peligro,
    },
    entrada: {
      flex: 1,
      height: '100%',
      fontFamily: 'Inter_400Regular',
      fontSize: 16,
      color: c.texto,
    },
    filaMensaje: {
      flexDirection: 'row',
      alignItems: 'center',
      gap: 6,
      marginTop: 6,
    },
    textoError: {
      flex: 1,
      fontFamily: 'Inter_400Regular',
      fontSize: 12.5,
      color: c.peligro,
    },
    textoExito: {
      flex: 1,
      fontFamily: 'Inter_400Regular',
      fontSize: 12.5,
      color: c.exito,
    },
    textoAyuda: {
      flex: 1,
      fontFamily: 'Inter_400Regular',
      fontSize: 12.5,
      color: c.textoSecundario,
    },
  });
}
```

POR QUÉ ASÍ

Son los mismos valores que el Login (alto 52, radio 14, borde 1,5), ahora en un solo lugar. `campoValido` solo cambia el color del borde a verde. Los tres estilos de texto comparten tamaño y separación, para que al pasar de la ayuda al error la pantalla no salte.

9

Registro

`screens/Registro.js` implementa las vistas **2.1** (paso 1), **2.2** (validación en vivo), **2.3** (paso 2), **2.5** (correo ya registrado) y **2.6** (ayuda con el teclado abierto). Es una sola pantalla que cambia de contenido según el paso, no dos pantallas.

9.1 El recorrido completo

1

Paso 1: crear la cuenta

Nombre, apellido, correo, contraseña y repetición. «Continuar» se habilita cuando están los cinco y valida todo antes de avanzar.

2

Paso 2: últimos datos

Teléfono y fecha de nacimiento (opcionales), aceptar los términos (obligatorio) y recibir promociones (opcional).

3

Crear cuenta

Firebase crea el usuario e inicia la sesión. Se guarda el nombre, se guardan los datos opcionales y se envía el correo de verificación.

4

Verificación

La navegación detecta un usuario sin verificar y muestra la pantalla de la sección 10.

Dónde queda cada dato. Firebase Authentication guarda correo, contraseña y nombre completo. Teléfono, fecha de nacimiento y la preferencia de promociones no tienen lugar en Firebase Authentication, así que por ahora se guardan en el teléfono con AsyncStorage, bajo la clave `cafeto:perfil:` más el id del usuario. Es un lugar provisorio: cuando la app se conecte a Django, esos datos se enviarán a la API y quedarán en la tabla `clientes`, que ya tiene `telefono_cliente` y `fecha_nacimiento`.

Mismas reglas que Django. Las validaciones copian las del sistema de PC (`cafeto/validaciones.py`): nombre y apellido de hasta 50 caracteres, teléfono de 7 a 15 dígitos y edad entre 14 y 100 años. Así, cuando se conecten, la app no aceptará datos que el servidor rechace.

9.2 Orden de construcción

1

Esqueleto con el paso

Creá la pantalla con el estado `paso` y un ternario `{paso === 1 ? (...) : (...)}` con un texto en cada rama. Agregala a `Navigation.js` y probá que «Registrate» la abra.

2 Encabezado y progreso

Flecha, isotipo, «PASO 1 DE 2», botón de tema y las dos barras. Con `volver` ya podés ir y venir entre pasos poniendo un botón provisorio.

3 Campos del paso 1 con CampoTexto

Primero los estados y los cinco campos sin validar. Después `errores`, `alCambiar`, `validarNombre` y `validarPaso1`.

4 Fortaleza y coincidencia

`medirFortaleza`, las cuatro barras y el mensaje verde de «Las contraseñas coinciden».

5 Paso 2

Teléfono con prefijo, selector de fecha con `calcularEdad` y `formatearFecha`, las dos casillas y `validarPaso2`.

6 Firebase

`crearCuenta` y `mostrarErrorDeRegistro`. Probá crear una cuenta y otra con el mismo correo.

7 Correo ya registrado

El bloque de la vista 2.5 con sus tres acciones, y el parámetro que recibe el Login.

9.3 Importaciones

screens/Registro.js — líneas 1–29

```
import React, { useMemo, useState } from 'react';
import {
  ActivityIndicator,
  Image,
  KeyboardAvoidingView,
  Platform,
  Pressable,
  ScrollView,
  StyleSheet,
  Text,
```

```
  View,
} from 'react-native';
import { SafeAreaView } from 'react-native-safe-area-context';
import { StatusBar } from 'expo-status-bar';
import { Feather, FontAwesome, Ionicons } from '@expo/vector-icons';
import DateTimePicker from '@react-native-community/datetimepicker';
import { useFonts, Caveat_700Bold } from '@expo-google-fonts/caveat';
import { Inter_400Regular, Inter_500Medium, Inter_600SemiBold } from '@expo-google-fonts/inter';
import AsyncStorage from '@react-native-async-storage/async-storage';
import {
  createUserWithEmailAndPassword,
  sendEmailVerification,
  sendPasswordResetEmail,
  updateProfile,
} from 'firebase/auth';
import { auth } from '../src/config/firebaseConfig';
import { useTema } from '../src/tema/TemaContext';
import { useAviso } from '../src/aviso/AvisoContext';
import CampoTexto from '../src/componentes/CampoTexto';
```

POR QUÉ ASÍ

Son casi las mismas del Login. Las nuevas: `DateTimePicker` (el calendario nativo del teléfono), cuatro funciones de Firebase para crear la cuenta, ponerle nombre, mandar la verificación y recuperar la contraseña, y nuestro `CampoTexto`. Como el import de Firebase tiene cuatro nombres, va en varias líneas.

`DateTimePicker` se instaló con `npx expo install @react-native-community/datetimepicker`. Está incluido en Expo Go, así que no hace falta compilar nada.

9.4 Constantes

screens/Registro.js — líneas 31–33

```
const FORMATO_CORREO = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
const SOLO_LETRAS = /^[A-Za-zÁÉÍÓÚÜÑáéíóúüñ' -]+$/;
const NIVELES = ['muy débil', 'débil', 'regular', 'buena', 'fuerte'];
```

POR QUÉ ASÍ

`SOLO_LETRAS` acepta letras con tilde, ñ, ü, espacios, apóstrofo y guion, para nombres como «María José», «D'Angelo» o «Pérez-Ruiz». Los corchetes `[...]` listan los caracteres permitidos y `+` pide uno o más.

`NIVELES` traduce el puntaje de la contraseña (0 a 4) a una palabra: `NIVELES[3]` es «buena».

9.5 Medir la fortaleza de la contraseña

screens/Registro.js — líneas 35–51

```
function medirFortaleza(clave) {
  const pruebas = [
    { cumple: clave.length >= 8, consejo: 'Usá 8 caracteres o más' },
    { cumple: /[a-z]/.test(clave) && /[A-Z]/.test(clave), consejo: 'Combiná mayúsculas y minúsculas'
```

```
},
    { cumple: /\d/.test(clave), consejo: 'Sumá un número' },
    { cumple: /[^A-Za-z0-9]/.test(clave), consejo: 'Sumá un símbolo' },
  ];
  const puntaje = pruebas.filter((prueba) => prueba.cumple).length;
  const pendiente = pruebas.find((prueba) => !prueba.cumple);

  return {
    puntaje,
    nivel: NIVELES[puntaje],
    consejo: pendiente ? pendiente.consejo : 'Excelente',
    segura: pruebas[0].cumple && pruebas[1].cumple && pruebas[2].cumple,
  };
}
```

Qué hace. Revisa cuatro pruebas, cuenta cuántas cumple y devuelve el puntaje, el nivel en palabras, un consejo con la primera prueba que falta y si la contraseña es lo bastante **segura** para avanzar.

POR QUÉ ASÍ

Las pruebas como lista de objetos. Cada prueba junta su condición y su consejo. Agregar una quinta prueba es sumar una línea, sin tocar el resto de la función.

`filter` y `find`. `filter` devuelve todas las pruebas que cumplen, y su `length` es el puntaje. `find` devuelve la *primera* que no cumple, o `undefined` si cumplen todas.

Segura no es lo mismo que fuerte. Para avanzar se exigen las tres primeras pruebas (8 caracteres, mayúsculas y minúsculas, un número). El símbolo suma fortaleza pero no es obligatorio, como en el diseño, donde «Cafeto2026» es «buena» y deja continuar.

Va fuera del componente porque no usa estados ni props: recibe un texto y devuelve un resultado. Una función así es fácil de probar sola.

9.6 Funciones de fecha

screens/Registro.js — líneas 53-67

```
function calcularEdad(nacimiento) {
  const hoy = new Date();
  let edad = hoy.getFullYear() - nacimiento.getFullYear();
  const yaCumplio =
    hoy.getMonth() > nacimiento.getMonth() ||
    (hoy.getMonth() === nacimiento.getMonth() && hoy.getDate() >= nacimiento.getDate());
  if (!yaCumplio) edad -= 1;
  return edad;
}

function formatearFecha(fecha) {
  const dia = String(fecha.getDate()).padStart(2, '0');
  const mes = String(fecha.getMonth() + 1).padStart(2, '0');
  return `${dia}/${mes}/${fecha.getFullYear()}`;
}
```

POR QUÉ ASÍ

Por qué no alcanza con restar años. Alguien nacido en diciembre de 2010 tiene 15 años en octubre de 2026, no 16. `yaCumplio` revisa si ya pasó el cumpleaños este año, y si no, resta uno.

`getMonth()` **empieza en 0.** Enero es 0 y diciembre es 11. Por eso `formatearFecha` suma 1 al mes, y `new Date(2000, 0, 1)` en el calendario significa 1 de enero.

`padStart(2, '0')` completa con ceros a la izquierda: el día 5 se muestra «05».

9.7 Firma y estados

screens/Registro.js — líneas 69-90

```
69 export default function Registro({ navigation }) {
70   const [fuentesCargadas] = useFonts({
71     Caveat_700Bold,
72     Inter_400Regular,
73     Inter_500Medium,
74     Inter_600SemiBold,
75   });
76
77   const [paso, setPaso] = useState(1);
78   const [nombre, setNombre] = useState('');
79   const [apellido, setApellido] = useState('');
80   const [correo, setCorreo] = useState('');
81   const [contrasena, setContrasena] = useState('');
82   const [repetir, setRepetir] = useState('');
83   const [telefono, setTelefono] = useState('');
84   const [fechaNacimiento, setFechaNacimiento] = useState(null);
85   const [mostrarCalendario, setMostrarCalendario] = useState(false);
86   const [aceptaTerminos, setAceptaTerminos] = useState(false);
87   const [aceptaPromociones, setAceptaPromociones] = useState(false);
88   const [errores, setErrores] = useState({});
89   const [correoYaRegistrado, setCorreoYaRegistrado] = useState(false);
90   const [creando, setCreando] = useState(false);
```

ESTADO	INICIAL	PARA QUÉ
`paso`	`1`	Qué parte del formulario se ve.
`nombre` a `telefono`	`''`	Lo que escribe el usuario.
`fechaNacimiento`	`null`	Un objeto `Date`, o `null` si no eligió.
`mostrarCalendario`	`false`	Si el selector de fecha está abierto.
`aceptaTerminos`, `aceptaPromociones`	`false`	Las dos casillas.
`errores`	`{}`	Un objeto con un mensaje por campo.
`correoYaRegistrado`	`false`	Activa la vista 2.5.
`creando`	`false`	Si Firebase está creando la cuenta.

POR QUÉ ASÍ

Un objeto `errores` en lugar de un estado por campo. El Login tenía `errorCorreo` y `errorContrasena`. Con ocho campos serían ocho estados más. Un objeto `{ nombre: '...', correo: '...' }` los junta, y validar es armar un objeto nuevo y guardarlo de una vez.

Los datos de los dos pasos viven en el mismo componente. Al volver del paso 2 al 1 no se pierde nada, porque el componente no se desmonta: solo cambia qué parte se dibuja.

`fechaNacimiento` como `Date` y no como texto. Con un objeto fecha se puede calcular la edad y darle formato; con un texto «05/12/2010» habría que desarmarlo primero.

9.8 Tema y valores derivados

screens/Registro.js líneas 92-99

```
92   const { tema, colores, isotipo, cambiarTema } = useTema();
93   const { avisar } = useAviso();
94   const estilos = useMemo(() => crearEstilos(colores), [colores]);
95
96   const fortaleza = medirFortaleza(contrasena);
97   const coinciden = repetir !== '' && repetir === contrasena;
98   const paso1Completo =
99     nombre.trim() !== '' && apellido.trim() !== '' && correo.trim() !== '' && contrasena !== '' &&
repetir !== '';
```

POR QUÉ ASÍ

Lee del contexto el nombre del tema (lo necesita el calendario), la paleta, el isotipo y la función para cambiarlo, y de `useAviso()` la función `avisar` (3.4). Como en el Login, `fortaleza`, `coinciden` y `paso1Completo` no son estados: se calculan en cada dibujo a partir de lo escrito.

`coinciden` exige que `repetir` no esté vacío. Si no, con los dos campos vacíos diría «Las contraseñas coinciden», que es cierto pero confuso.

9.9 Funciones para escribir en los campos

screens/Registro.js líneas 101-115

```
101  const limpiarError = (campo) => {
102    setErrores((anteriores) => ({ ...anteriores, [campo]: '' }));
103  };
104
105  const alCambiar = (campo, setter) => (texto) => {
106    setter(texto);
107    limpiarError(campo);
108    if (campo === 'correo') setCorreoYaRegistrado(false);
109  };
110
111  const validarNombre = (valor, etiqueta) => {
112    if (!valor.trim()) return `Ingresá tu ${etiqueta}.`;
113    if (!SOLO_LETRAS.test(valor.trim())) return 'Usá solo letras.';
```

```
    return '';
  };
```

Qué hace. `limpiarError` borra el mensaje de un campo. `alCambiar` *fabrica* la función que cada campo usa al escribir. `validarNombre` revisa nombre o apellido y devuelve el mensaje de error, o texto vacío si está bien.

> POR QUÉ ASÍ
>
> **Una función que devuelve una función.** `alCambiar('correo', setCorreo)` no cambia nada todavía: devuelve `(texto) ⇒ {...}`, que es lo que necesita `alCambiar` del campo. Las dos flechas seguidas, `(campo, setter) ⇒ (texto) ⇒`, son eso. Reemplaza las cinco funciones `cambiarNombre`, `cambiarApellido`... que habría que escribir a mano.
>
> `{ ...anteriores, [campo]: '' }`. Copia todos los errores que había y pisa solo el del campo indicado. Los corchetes en `[campo]` usan el *valor* de la variable como nombre de la clave: si `campo` vale `'correo'`, se escribe la clave `correo`.
>
> **Una validación que devuelve el mensaje** sirve dos veces: para el error al enviar (9.10) y para el borde verde mientras se escribe (9.18).

9.10 Validar el paso 1

screens/Registro.js — líneas 117-142

```
  const validarPaso1 = () => {
    const nuevos = {};

    const errorNombre = validarNombre(nombre, 'nombre');
    if (errorNombre) nuevos.nombre = errorNombre;

    const errorApellido = validarNombre(apellido, 'apellido');
    if (errorApellido) nuevos.apellido = errorApellido;

    if (!correo.includes('@')) {
      nuevos.correo = 'Falta la @. Ejemplo: maxi@cafeto.com.ar';
    } else if (!FORMATO_CORREO.test(correo.trim())) {
      nuevos.correo = 'Falta el dominio. Ejemplo: maxi@cafeto.com.ar';
    }

    if (!fortaleza.segura) {
      nuevos.contrasena = 'Usá 8 caracteres o más, con mayúsculas, minúsculas y un número.';
    }

    if (repetir !== contrasena) {
      nuevos.repetir = 'Las contraseñas no coinciden.';
    }

    setErrores(nuevos);
    return Object.keys(nuevos).length === 0;
  };
```

> POR QUÉ ASÍ
>
> Arma un objeto `nuevos` solo con los campos que tienen problemas, lo guarda y devuelve si quedó vacío. `Object.keys(nuevos).length === 0` cuenta las claves del

objeto.

Los dos mensajes del correo copian la vista 2.2: si falta la @ se dice eso, y si falta lo que va después del punto, «Falta el dominio», siempre con un ejemplo.

Igual que en el Login, revisa **todos** los campos antes de responder, para mostrar todos los errores a la vez.

9.11 Validar el paso 2

screens/Registro.js — líneas 144-166

```
144  const validarPaso2 = () => {
145    const nuevos = {};
146
147    if (telefono.trim()) {
148      const digitos = telefono.replace(/\D/g, '');
149      if (digitos.length < 7 || digitos.length > 15) {
150        nuevos.telefono = 'El teléfono son entre 7 y 15 dígitos.';
151      }
152    }
153
154    if (fechaNacimiento) {
155      const edad = calcularEdad(fechaNacimiento);
156      if (edad < 14) nuevos.fecha = 'Tenés que tener al menos 14 años.';
157      if (edad > 100) nuevos.fecha = 'Revisá el año de nacimiento.';
158    }
159
160    if (!aceptaTerminos) {
161      nuevos.terminos = 'Para crear la cuenta tenés que aceptar los términos.';
162    }
163
164    setErrores(nuevos);
165    return Object.keys(nuevos).length === 0;
166  };
```

POR QUÉ ASÍ

Lo opcional se valida solo si se completó. El `if (telefono.trim())` deja pasar el campo vacío; si tiene algo, tiene que ser un teléfono válido.

`telefono.replace(/\D/g, '')` borra todo lo que no sea dígito (`\D`), en todo el texto (`g`). Así «387 000-0000» cuenta 10 dígitos aunque tenga espacios y guiones.

Los términos sí son obligatorios: sin aceptarlos no se crea la cuenta.

9.12 Avanzar, volver y elegir la fecha

screens/Registro.js — líneas 168-186

```
168  const continuar = () => {
169    if (validarPaso1()) setPaso(2);
170  };
171
172  const volver = () => {
173    if (paso === 2) {
174      setPaso(1);
175    } else {
```

```
176       navigation.goBack();
177     }
178   };
179
180   const elegirFecha = (evento, fecha) => {
181     if (Platform.OS === 'android') setMostrarCalendario(false);
182     if (evento.type === 'set' && fecha) {
183       setFechaNacimiento(fecha);
184       limpiarError('fecha');
185     }
186   };
```

POR QUÉ ASÍ

`volver` hace dos cosas según el paso: en el 2 vuelve al 1 dentro de la misma pantalla; en el 1 sale del Registro con `navigation.goBack()`.

El calendario funciona distinto en cada sistema. En Android abre un diálogo que se cierra solo al elegir o cancelar, por eso se oculta en cuanto responde. En iOS queda en la pantalla y se cierra con «Listo» (9.24). `evento.type === 'set'` indica que eligió una fecha; si canceló, vale `'dismissed'` y no se guarda nada.

Mejora pendiente: el botón atrás físico de Android, en el paso 2, cierra la pantalla en vez de volver al paso 1. Se resuelve escuchando el evento `beforeRemove` de la navegación. Es un buen ejercicio.

9.13 Errores de Firebase

screens/Registro.js líneas 188–213

```
188   const mostrarErrorDeRegistro = (error) => {
189     switch (error.code) {
190       case 'auth/email-already-in-use':
191         setCorreoYaRegistrado(true);
192         setErrores({ correo: 'Ese correo ya tiene una cuenta en Cafeto.' });
193         setPaso(1);
194         break;
195       case 'auth/invalid-email':
196         setErrores({ correo: 'El formato del correo no es válido.' });
197         setPaso(1);
198         break;
199       case 'auth/weak-password':
200         setErrores({ contrasena: 'La contraseña es demasiado débil.' });
201         setPaso(1);
202         break;
203       case 'auth/configuration-not-found':
204       case 'auth/operation-not-allowed':
205         avisar('advertencia', 'Registro no disponible', 'Falta habilitar el acceso con correo y
contraseña en Firebase.');
206         break;
207       case 'auth/network-request-failed':
208         avisar('error', 'Sin conexión', 'Revisá el wifi o los datos del teléfono.');
209         break;
210       default:
211         avisar('error', 'Error', `No pudimos crear la cuenta.\n\nCódigo: ${error.code ??
'desconocido'}`);
212     }
213   };
```

POR QUÉ ASÍ

`auth/configuration-not-found` aparece cuando el proyecto de Firebase no tiene activado Authentication o el proveedor de correo y contraseña. No es un problema del código ni de lo que escribió el usuario: hay que activarlo en la consola de Firebase (Authentication → Comenzar → Método de acceso → Correo electrónico/contraseña). Comparte mensaje con `auth/operation-not-allowed`, que es el mismo problema cuando Authentication está activo pero ese proveedor no.

Los errores del correo vuelven al paso 1. El correo se escribe en el paso 1 pero Firebase recién lo revisa al crear la cuenta en el paso 2. Por eso el error lleva al usuario de vuelta, con el campo marcado.

`auth/email-already-in-use` activa la vista 2.5. Firebase ya no permite preguntar si un correo existe antes de crear la cuenta (lo bloquea para que no se puedan averiguar correos registrados), así que la única forma de saberlo es intentarlo.

9.14 Crear la cuenta

screens/Registro.js líneas 215-237

```
215   const crearCuenta = async () => {
216     if (!validarPaso2()) return;
217
218     setCreando(true);
219     try {
220       const credencial = await createUserWithEmailAndPassword(auth, correo.trim(), contrasena);
221       await updateProfile(credencial.user, { displayName: `${nombre.trim()} ${apellido.trim()}` });
222       await AsyncStorage.setItem(
223         `cafeto:perfil:${credencial.user.uid}`,
224         JSON.stringify({
225           nombre: nombre.trim(),
226           apellido: apellido.trim(),
227           telefono: telefono.trim() ? `+54 ${telefono.trim()}` : null,
228           fechaNacimiento: fechaNacimiento ? fechaNacimiento.toISOString().slice(0, 10) : null,
229           aceptaPromociones,
230         })
231       );
232       await sendEmailVerification(credencial.user);
233     } catch (error) {
234       setCreando(false);
235       mostrarErrorDeRegistro(error);
236     }
237   };
```

Qué hace. Valida el paso 2, crea el usuario, le pone el nombre, guarda los datos opcionales en el teléfono y envía el correo de verificación.

POR QUÉ ASÍ

El orden importa. `updateProfile` y `sendEmailVerification` necesitan el usuario que devuelve `createUserWithEmailAndPassword`, así que van después y con `await`.

`JSON.stringify` convierte el objeto en texto, porque AsyncStorage solo guarda texto. Para leerlo se usa `JSON.parse`.

`toISOString().slice(0, 10)` deja la fecha como «2010-12-05», el formato que usan PostgreSQL y Django. Cuando se conecte la API, el dato ya va a tener la forma correcta.

No hay `navigate` ni `setCreando(false)` si sale bien. Apenas se crea la cuenta, la navegación pasa a Verificación (3.3) y esta pantalla se desmonta. Es el mismo razonamiento que `iniciarSesion` en el Login.

9.15 Acciones del correo ya registrado

screens/Registro.js líneas 239–260

```
239   const iniciarSesionConEseCorreo = () => {
240     navigation.popTo('Login', { correo: correo.trim() });
241   };
242
243   const recuperarContrasena = async () => {
244     try {
245       await sendPasswordResetEmail(auth, correo.trim());
246       avisar('exito', 'Revisá tu correo', `Te mandamos un enlace a ${correo.trim()} para que elijas
una contraseña nueva.`);
247     } catch (error) {
248       avisar('error', 'Error', `No pudimos enviar el correo.\n\nCódigo: ${error.code ??
'desconocido'}`);
249     }
250   };
251
252   const usarOtroCorreo = () => {
253     setCorreo('');
254     setCorreoYaRegistrado(false);
255     limpiarError('correo');
256   };
257
258   const avisarProximamente = (proveedor) => {
259     avisar('advertencia', 'Próximamente', `El registro con ${proveedor} todavía no está
disponible.`);
260   };
```

POR QUÉ ASÍ

Volver con datos a una pantalla anterior. `navigation.popTo('Login', { correo })` cierra el Registro, vuelve al Login que ya estaba debajo en el stack y le pasa un parámetro. El Login lo lee en `route.params.correo` y completa su campo (5.8).

Por qué `popTo` y no `navigate`. En React Navigation 7, `navigate('Login')` desde el Registro *apila* un Login nuevo encima, y el botón atrás volvería al Registro. `popTo` vuelve al que ya existe. Por lo mismo, «Iniciá sesión» del pie también usa `popTo`.

`usarOtroCorreo` borra el correo y apaga la vista 2.5, que es el «¿Te equivocaste? Usá otro correo» del diseño.

9.16 Fuentes y piezas reutilizables

screens/Registro.js líneas 262–286

```
  if (!fuentesCargadas) {
    return (
      <View style={[estilos.pantalla, estilos.centrado]}>
        <ActivityIndicator size="large" color={colores.marca} />
      </View>
    );
  }

  const casilla = (marcada, alTocar, contenido) => (
    <Pressable style={estilos.filaCasilla} onPress={alTocar}>
      <View style={[estilos.casilla, marcada && estilos.casillaMarcada]}>
        {marcada && <Feather name="check" size={16} color={colores.textoSobreMarca} />}
      </View>
      <Text style={estilos.textoCasilla}>{contenido}</Text>
    </Pressable>
  );

  const pie = (
    <View style={estilos.filaPie}>
      <Text style={estilos.textoPie}>¿Ya tenés cuenta? </Text>
      <Pressable onPress={() => navigation.popTo('Login')}>
        <Text style={[estilos.textoPie, estilos.enlace]}>Iniciá sesión</Text>
      </Pressable>
    </View>
  );
```

POR QUÉ ASÍ

`casilla` **es una función que devuelve JSX.** Las dos casillas del paso 2 son iguales salvo el estado, la acción y el texto. Un componente aparte también serviría; como solo se usa en este archivo y no tiene estado propio, una función interna alcanza.

`pie` **es JSX guardado en una variable.** «¿Ya tenés cuenta? Iniciá sesión» aparece en los dos pasos. Guardarlo en una constante evita escribirlo dos veces.

Las dos van **después** de la salida temprana porque no son hooks. La regla de los hooks (5.16) solo aplica a funciones que empiezan con `use`.

9.17 Contenedores, encabezado y progreso

screens/Registro.js — líneas 288–314

```
  return (
    <SafeAreaView style={estilos.pantalla}>
      <StatusBar style={colores.barraEstado} />
      <KeyboardAvoidingView
        style={estilos.pantalla}
        behavior={Platform.OS === 'ios' ? 'padding' : undefined}
      >
        <ScrollView
          contentContainerStyle={estilos.contenido}
          keyboardShouldPersistTaps="handled"
          showsVerticalScrollIndicator={false}
        >
          <View style={estilos.encabezado}>
            <Pressable onPress={volver} hitSlop={10} disabled={creando}>
              <Feather name="arrow-left" size={24} color={colores.texto} />
            </Pressable>
            <Image source={isotipo} style={estilos.isotipo} resizeMode="contain" />
            <Text style={estilos.textoPaso}>PASO {paso} DE 2</Text>
            <Pressable style={estilos.botonTema} onPress={cambiarTema}>
```

```
            <Ionicons name="contrast" size={20} color={colores.texto} />
          </Pressable>
        </View>

        <View style={estilos.progreso}>
          <View style={[estilos.segmento, estilos.segmentoActivo]} />
          <View style={[estilos.segmento, paso === 2 && estilos.segmentoActivo]} />
        </View>
```

POR QUÉ ASÍ

Las tres capas de contenedores son las mismas del Login (6.1). El texto «PASO {paso} DE 2» mezcla texto fijo y una expresión entre llaves.

Barra de progreso con dos segmentos. El primero siempre está activo; el segundo se activa con `paso === 2`. Con `flex: 1` se reparten el ancho por igual.

La flecha se deshabilita mientras se crea la cuenta, para no salir a mitad de camino.

9.18 Paso 1: nombre, apellido y correo

screens/Registro.js — líneas 316-360

```
        {paso === 1 ? (
          <>
            <Text style={estilos.titulo}>Creá tu cuenta</Text>

            <View style={estilos.fila}>
              <CampoTexto
                estiloContenedor={estilos.mitad}
                etiqueta="Nombre"
                placeholder="Juan"
                valor={nombre}
                alCambiar={alCambiar('nombre', setNombre)}
                error={errores.nombre ?? ''}
                valido={validarNombre(nombre, 'nombre') === ''}
                autoCapitalize="words"
                autoComplete="given-name"
                maxLength={50}
              />
              <CampoTexto
                estiloContenedor={estilos.mitad}
                etiqueta="Apellido"
                placeholder="Montes"
                valor={apellido}
                alCambiar={alCambiar('apellido', setApellido)}
                error={errores.apellido ?? ''}
                valido={validarNombre(apellido, 'apellido') === ''}
                autoCapitalize="words"
                autoComplete="family-name"
                maxLength={50}
              />
            </View>

            <CampoTexto
              etiqueta="Correo electrónico"
              icono="mail"
              placeholder="tunombre@correo.com"
              valor={correo}
              alCambiar={alCambiar('correo', setCorreo)}
              error={errores.correo ?? ''}
              ayudaAlEnfocar="Vas a entrar con este correo y te mandamos ahí el enlace de
```

```
verificación."
                valido={!correoYaRegistrado && FORMATO_CORREO.test(correo.trim())}
                keyboardType="email-address"
                autoCapitalize="none"
                autoCorrect={false}
                autoComplete="email"
              />
```

POR QUÉ ASÍ

Fila de dos campos. El View `fila` va en dirección horizontal con `gap: 12`, y cada campo recibe `estiloContenedor={estilos.mitad}` (`flex: 1`) para ocupar la mitad.

`errores.nombre ?? ''`. Si el objeto no tiene esa clave, da `undefined`; el `??` lo reemplaza por texto vacío, que es lo que espera `CampoTexto`.

Borde verde en vivo (vista 2.2): `valido` se calcula con la misma `validarNombre` de 9.9 y con el formato del correo. El usuario ve qué está bien antes de tocar «Continuar».

`autoCapitalize="words"` pone mayúscula a cada palabra; `autoComplete="given-name"` y `"family-name"` dejan que el teléfono sugiera los datos guardados.

La ayuda del correo aparece solo al enfocar (vista 2.6), con `ayudaAlEnfocar`.

9.19 Correo ya registrado (vista 2.5)

screens/Registro.js — líneas 362–396

```
              {correoYaRegistrado ? (
                <>
                  <View style={estilos.aviso}>
                    <Feather name="info" size={18} color={colores.texto} />
                    <View style={estilos.textosAviso}>
                      <Text style={estilos.tituloAviso}>¿Sos vos?</Text>
                      <Text style={estilos.detalleAviso}>
                        Podés entrar con esa cuenta, o recuperar la contraseña si no te acordás.
                      </Text>
                    </View>
                  </View>

                  <Pressable
                    style={({ pressed }) => [estilos.botonPrincipal, pressed &&
estilos.presionado]}
                    onPress={iniciarSesionConEseCorreo}
                  >
                    <Text style={estilos.textoBotonPrincipal}>Iniciar sesión con ese correo</Text>
                  </Pressable>
                  <Pressable
                    style={({ pressed }) => [estilos.botonSecundario, pressed &&
estilos.presionado]}
                    onPress={recuperarContrasena}
                  >
                    <Text style={estilos.textoBotonSecundario}>Recuperar contraseña</Text>
                  </Pressable>

                  <View style={estilos.espaciador} />

                  <View style={estilos.filaPie}>
                    <Text style={estilos.textoPie}>¿Te equivocaste? </Text>
```

```
                    <Pressable onPress={usarOtroCorreo}>
                      <Text style={[estilos.textoPie, estilos.enlace]}>Usá otro correo</Text>
                    </Pressable>
                  </View>
                </>
              ) : (
```

POR QUÉ ASÍ

Si `correoYaRegistrado` es verdadero, el resto del paso 1 se reemplaza por el aviso y dos acciones: entrar con ese correo o recuperar la contraseña. No tendría sentido seguir mostrando los campos de contraseña de una cuenta nueva.

El aviso usa los colores `fondoPeligro` y `peligro` que se agregaron a la paleta. El botón secundario tiene borde y fondo transparente, para que quede claro cuál es la acción principal.

9.20 Contraseña y fortaleza

screens/Registro.js líneas 397–431

```
                <>
                  <CampoTexto
                    etiqueta="Contraseña"
                    icono="lock"
                    placeholder="Mínimo 8 caracteres"
                    valor={contrasena}
                    alCambiar={alCambiar('contrasena', setContrasena)}
                    error={errores.contrasena ?? ''}
                    seguro
                    autoCapitalize="none"
                    autoComplete="new-password"
                  />
                  {contrasena !== '' && (
                    <>
                      <View style={estilos.barrasFortaleza}>
                        {[1, 2, 3, 4].map((numero) => (
                          <View
                            key={numero}
                            style={[
                              estilos.barraFortaleza,
                              numero <= fortaleza.puntaje && {
                                backgroundColor: fortaleza.segura ? colores.marca :
colores.peligro,
                              },
                            ]}
                          />
                        ))}
                      </View>
                      <View style={estilos.filaFortaleza}>
                        <Text style={estilos.textoFortaleza}>
                          Fortaleza: <Text style={estilos.nivelFortaleza}>{fortaleza.nivel}</Text>
                        </Text>
                        <Text style={estilos.textoFortaleza}>{fortaleza.consejo}</Text>
                      </View>
                    </>
                  )}
```

POR QUÉ ASÍ

`autoComplete="new-password"` le indica al teléfono que es una contraseña nueva, y los gestores de contraseñas ofrecen generar una.

Cuatro barras con `map`. `[1, 2, 3, 4].map(...)` dibuja una barra por número. Cada una se pinta si su número es menor o igual al puntaje. React exige una `key` única en cada elemento de una lista, para saber cuál es cuál al redibujar.

Color según si es segura: ámbar si ya puede avanzar, rojo si todavía no. El estilo se arma en el momento, con un objeto dentro del arreglo, porque el color depende del dato.

El medidor aparece solo cuando hay algo escrito.

9.21 Repetir contraseña y Continuar

screens/Registro.js — líneas 433-463

```
433                   <CampoTexto
434                     etiqueta="Repetir contraseña"
435                     icono="lock"
436                     placeholder="Repetí la contraseña"
437                     valor={repetir}
438                     alCambiar={alCambiar('repetir', setRepetir)}
439                     error={errores.repetir ?? ''}
440                     mensajeExito={coinciden ? 'Las contraseñas coinciden' : ''}
441                     valido={coinciden}
442                     seguro
443                     autoCapitalize="none"
444                     autoComplete="new-password"
445                     returnKeyType="next"
446                     onSubmitEditing={continuar}
447                   />
448
449                   <Pressable
450                     style={({ pressed }) => [
451                       estilos.botonPrincipal,
452                       !paso1Completo && estilos.botonDeshabilitado,
453                       pressed && estilos.presionado,
454                     ]}
455                     onPress={continuar}
456                     disabled={!paso1Completo}
457                   >
458                     <Text
459                       style={[estilos.textoBotonPrincipal, !paso1Completo &&
estilos.textoBotonDeshabilitado]}
460                     >
461                       Continuar
462                     </Text>
463                   </Pressable>
```

POR QUÉ ASÍ

`mensajeExito` y `valido` dependen de `coinciden`: borde y mensaje verdes cuando las dos son iguales. `onSubmitEditing={continuar}` hace que la tecla del teclado avance.

El botón «Continuar» usa el mismo patrón que «Iniciar sesión» en el Login (6.7): gris y deshabilitado hasta que estén los cinco campos.

9.22 Alternativas y cierre del paso 1

screens/Registro.js — líneas 465–490

```
465                   <View style={estilos.separador}>
466                     <View style={estilos.linea} />
467                     <Text style={estilos.textoSeparador}>o</Text>
468                     <View style={estilos.linea} />
469                   </View>
470
471                   <Pressable
472                     style={({ pressed }) => [estilos.botonSocial, estilos.botonGoogle, pressed &&
estilos.presionado]}
473                     onPress={() => avisarProximamente('Google')}
474                   >
475                     <FontAwesome name="google" size={20} color="#4285F4" />
476                     <Text style={[estilos.textoBotonSocial, estilos.textoGoogle]}>Continuar con
Google</Text>
477                   </Pressable>
478                   <Pressable
479                     style={({ pressed }) => [estilos.botonSocial, estilos.botonApple, pressed &&
estilos.presionado]}
480                     onPress={() => avisarProximamente('Apple')}
481                   >
482                     <FontAwesome name="apple" size={22} color="#FFFFFF" />
483                     <Text style={[estilos.textoBotonSocial, estilos.textoApple]}>Continuar con
Apple</Text>
484                   </Pressable>
485
486                   <View style={estilos.espaciador} />
487                   {pie}
488                 </>
489               )}
490             </>
```

POR QUÉ ASÍ

Separador y botones sociales idénticos a los del Login. El `espaciador` empuja el pie al fondo cuando sobra lugar; en esta versión tiene `minHeight: 16` para que nunca quede pegado. `{pie}` inserta el JSX guardado en 9.16.

9.23 Paso 2: títulos y teléfono

screens/Registro.js — líneas 491–514

```
491           ) : (
492             <>
493               <Text style={estilos.titulo}>Últimos datos</Text>
494               <Text style={estilos.subtitulo}>Opcionales, pero nos ayudan a reconocerte en el salón.
</Text>
495
496               <CampoTexto
497                 etiqueta="Teléfono"
498                 opcional
499                 icono="smartphone"
500                 prefijo={
501                   <View style={estilos.prefijo}>
502                     <Text style={estilos.pais}>AR</Text>
503                     <Text style={estilos.codigoPais}>+54</Text>
504                   </View>
505                 }
506                 placeholder="387 000 0000"
```

```
              valor={telefono}
              alCambiar={alCambiar('telefono', setTelefono)}
              error={errores.telefono ?? ''}
              ayuda="Con tu teléfono también te encontramos en la caja para sumarte puntos."
              keyboardType="phone-pad"
              autoComplete="tel"
              maxLength={20}
            />
```

POR QUÉ ASÍ

El prefijo se pasa como JSX. El «AR +54» es un View que se manda en la prop `prefijo` y `CampoTexto` lo dibuja entre el ícono y el texto (8.3).

`keyboardType="phone-pad"` abre el teclado numérico de teléfono. `opcional` agrega «· opcional» a la etiqueta.

La ayuda del diseño decía «Lo usamos para avisarte cuando tu pedido está listo». Cafeto no toma pedidos por la app, así que se cambió por el uso real: identificar al cliente en la caja para sumar puntos.

9.24 Fecha de nacimiento

screens/Registro.js — líneas 516-554

```
            <Text style={estilos.etiqueta}>
              Fecha de nacimiento<Text style={estilos.opcional}> · opcional</Text>
            </Text>
            <Pressable
              style={[estilos.selectorFecha, errores.fecha && estilos.campoConError]}
              onPress={() => setMostrarCalendario(true)}
            >
              <Feather name="calendar" size={20} color={colores.textoSecundario} />
              <Text style={[estilos.textoFecha, !fechaNacimiento && estilos.textoFechaVacia]}>
                {fechaNacimiento ? formatearFecha(fechaNacimiento) : 'Elegí tu fecha'}
              </Text>
              <Feather name="chevron-down" size={20} color={colores.textoSecundario} />
            </Pressable>
            {errores.fecha ? (
              <View style={estilos.filaMensaje}>
                <Feather name="alert-circle" size={14} color={colores.peligro} />
                <Text style={estilos.textoError}>{errores.fecha}</Text>
              </View>
            ) : (
              <Text style={estilos.ayuda}>Te regalamos un café en tu cumpleaños.</Text>
            )}

            {mostrarCalendario && (
              <DateTimePicker
                value={fechaNacimiento ?? new Date(2000, 0, 1)}
                mode="date"
                display={Platform.OS === 'ios' ? 'spinner' : 'default'}
                maximumDate={new Date()}
                minimumDate={new Date(1900, 0, 1)}
                onChange={elegirFecha}
                themeVariant={tema === 'oscuro' ? 'dark' : 'light'}
                locale="es-AR"
              />
            )}
            {mostrarCalendario && Platform.OS === 'ios' && (
              <Pressable style={estilos.botonListo} onPress={() => setMostrarCalendario(false)}>
```

```
552                 <Text style={[estilos.textoPie, estilos.enlace]}>Listo</Text>
553               </Pressable>
554             )}
```

POR QUÉ ASÍ

Un Pressable que parece un campo. La fecha no se escribe, se elige. Por eso no es un TextInput: es un botón con el mismo aspecto que abre el calendario.

Límites del calendario. `maximumDate={new Date()}` impide elegir una fecha futura y `minimumDate` no deja ir antes de 1900, igual que la restricción `ck_clientes_nacimiento` de la base del sistema de PC.

`value={fechaNacimiento ?? new Date(2000, 0, 1)}`: si todavía no eligió, el calendario se abre en el año 2000 y no en hoy, que estaría lejos de cualquier fecha de nacimiento posible.

`themeVariant` pinta el calendario de iOS en claro u oscuro según el tema de la app.

9.25 Casillas de términos y promociones

screens/Registro.js — líneas 556-579

```
556             <View style={estilos.casillas}>
557               {casilla(
558                 aceptaTerminos,
559                 () => {
560                   setAceptaTerminos(!aceptaTerminos);
561                   limpiarError('terminos');
562                 },
563                 <>
564                   Acepto los <Text style={estilos.enlace}>Términos y Condiciones</Text> y la{' '}
565                   <Text style={estilos.enlace}>Política de privacidad</Text>
566                 </>
567               )}
568               {casilla(
569                 aceptaPromociones,
570                 () => setAceptaPromociones(!aceptaPromociones),
571                 'Quiero recibir promociones y novedades por correo'
572               )}
573               {errores.terminos ? (
574                 <View style={estilos.filaMensaje}>
575                   <Feather name="alert-circle" size={14} color={colores.peligro} />
576                   <Text style={estilos.textoError}>{errores.terminos}</Text>
577                 </View>
578               ) : null}
579             </View>
```

POR QUÉ ASÍ

Se usa la función `casilla` de 9.16. El tercer argumento puede ser texto simple o JSX: la primera casilla pasa un fragmento con los enlaces subrayados; la segunda, un texto.

Al marcar «Acepto los términos» se borra su error, igual que al escribir en un campo.

`: null` al final del ternario significa «no dibujar nada». Es equivalente al `&&`, pero acá se usa ternario porque `errores.terminos` podría ser texto vacío (6.4).

9.26 Crear cuenta y cierre

screens/Registro.js — líneas 581-604

```
581              <View style={estilos.espaciador} />
582
583              <Pressable
584                style={({ pressed }) => [estilos.botonPrincipal, pressed && estilos.presionado]}
585                onPress={crearCuenta}
586                disabled={creando}
587              >
588                {creando ? (
589                  <View style={estilos.filaCargando}>
590                    <ActivityIndicator color={colores.textoSobreMarca} />
591                    <Text style={estilos.textoBotonPrincipal}>Creando cuenta…</Text>
592                  </View>
593                ) : (
594                  <Text style={estilos.textoBotonPrincipal}>Crear cuenta</Text>
595                )}
596              </Pressable>
597              {pie}
598            </>
599          )}
600        </ScrollView>
601      </KeyboardAvoidingView>
602    </SafeAreaView>
603  );
604 }
```

POR QUÉ ASÍ

Mismo botón con indicador de carga que el Login. Mientras `creando` es verdadero se deshabilita, para no crear dos cuentas con dos toques rápidos.

9.27 Estilos del Registro

screens/Registro.js — líneas 606-662

```
606 function crearEstilos(c) {
607   return StyleSheet.create({
608     pantalla: {
609       flex: 1,
610       backgroundColor: c.fondo,
611     },
612     centrado: {
613       justifyContent: 'center',
614       alignItems: 'center',
615     },
616     contenido: {
617       flexGrow: 1,
618       paddingHorizontal: 28,
619       paddingTop: 12,
620       paddingBottom: 24,
621     },
622     encabezado: {
623       flexDirection: 'row',
624       alignItems: 'center',
```

```
625       gap: 14,
626     },
627     isotipo: {
628       width: 34,
629       height: 34,
630     },
631     textoPaso: {
632       flex: 1,
633       fontFamily: 'Inter_500Medium',
634       fontSize: 12,
635       letterSpacing: 3,
636       color: c.textoSecundario,
637       textAlign: 'center',
638     },
639     botonTema: {
640       width: 40,
641       height: 40,
642       borderRadius: 20,
643       borderWidth: 1,
644       borderColor: c.borde,
645       backgroundColor: c.superficie,
646       justifyContent: 'center',
647       alignItems: 'center',
648     },
649     progreso: {
650       flexDirection: 'row',
651       gap: 10,
652       marginTop: 14,
653     },
654     segmento: {
655       flex: 1,
656       height: 4,
657       borderRadius: 2,
658       backgroundColor: c.borde,
659     },
660     segmentoActivo: {
661       backgroundColor: c.marca,
662     },
```

BASE, ENCABEZADO Y PROGRESO

`textoPaso` tiene `flex: 1` para ocupar el espacio entre el isotipo y el botón de tema, y `textAlign: 'center'` para quedar centrado en ese espacio.

screens/Registro.js líneas 663–742

```
663     titulo: {
664       fontFamily: 'Caveat_700Bold',
665       fontSize: 38,
666       color: c.titulo,
667       marginTop: 20,
668     },
669     subtitulo: {
670       fontFamily: 'Inter_400Regular',
671       fontSize: 15,
672       lineHeight: 21,
673       color: c.textoSecundario,
674     },
675     fila: {
676       flexDirection: 'row',
677       gap: 12,
678     },
679     mitad: {
680       flex: 1,
```

```
  },
  etiqueta: {
    fontFamily: 'Inter_600SemiBold',
    fontSize: 13.5,
    color: c.texto,
    marginTop: 16,
    marginBottom: 8,
  },
  opcional: {
    fontFamily: 'Inter_400Regular',
    color: c.textoTenue,
  },
  barrasFortaleza: {
    flexDirection: 'row',
    gap: 6,
    marginTop: 10,
  },
  barraFortaleza: {
    flex: 1,
    height: 4,
    borderRadius: 2,
    backgroundColor: c.borde,
  },
  filaFortaleza: {
    flexDirection: 'row',
    justifyContent: 'space-between',
    marginTop: 6,
  },
  textoFortaleza: {
    fontFamily: 'Inter_400Regular',
    fontSize: 12.5,
    color: c.textoSecundario,
  },
  nivelFortaleza: {
    fontFamily: 'Inter_600SemiBold',
    color: c.marca,
  },
  aviso: {
    flexDirection: 'row',
    gap: 12,
    padding: 16,
    marginTop: 16,
    borderRadius: 14,
    borderWidth: 1,
    borderColor: c.peligro,
    backgroundColor: c.fondoPeligro,
  },
  textosAviso: {
    flex: 1,
    gap: 4,
  },
  tituloAviso: {
    fontFamily: 'Inter_600SemiBold',
    fontSize: 14,
    color: c.texto,
  },
  detalleAviso: {
    fontFamily: 'Inter_400Regular',
    fontSize: 13.5,
    lineHeight: 19,
    color: c.textoSecundario,
  },
```

TÍTULOS, FILA, FORTALEZA Y AVISO

> Las barras de fortaleza son Views de 4 puntos de alto con `flex: 1`; sin color propio toman el del borde y se ven «apagadas».

screens/Registro.js — líneas 743-807

```
743     prefijo: {
744       flexDirection: 'row',
745       alignItems: 'center',
746       gap: 8,
747     },
748     pais: {
749       fontFamily: 'Inter_600SemiBold',
750       fontSize: 11,
751       color: c.textoSecundario,
752       paddingHorizontal: 6,
753       paddingVertical: 2,
754       borderRadius: 4,
755       backgroundColor: c.borde,
756       overflow: 'hidden',
757     },
758     codigoPais: {
759       fontFamily: 'Inter_600SemiBold',
760       fontSize: 16,
761       color: c.texto,
762     },
763     selectorFecha: {
764       flexDirection: 'row',
765       alignItems: 'center',
766       gap: 12,
767       height: 52,
768       paddingHorizontal: 16,
769       borderRadius: 14,
770       borderWidth: 1.5,
771       borderColor: c.borde,
772       backgroundColor: c.superficie,
773     },
774     campoConError: {
775       borderColor: c.peligro,
776     },
777     textoFecha: {
778       flex: 1,
779       fontFamily: 'Inter_400Regular',
780       fontSize: 16,
781       color: c.texto,
782     },
783     textoFechaVacia: {
784       color: c.textoTenue,
785     },
786     filaMensaje: {
787       flexDirection: 'row',
788       alignItems: 'center',
789       gap: 6,
790       marginTop: 6,
791     },
792     textoError: {
793       flex: 1,
794       fontFamily: 'Inter_400Regular',
795       fontSize: 12.5,
796       color: c.peligro,
797     },
798     ayuda: {
799       fontFamily: 'Inter_400Regular',
800       fontSize: 12.5,
801       color: c.textoSecundario,
802       marginTop: 6,
803     },
```

```
804     botonListo: {
805       alignSelf: 'flex-end',
806       paddingVertical: 8,
807     },
```

TELÉFONO, FECHA Y MENSAJES

`pais` lleva `overflow: 'hidden'` porque en iOS un `<Text>` con fondo no respeta el `borderRadius` sin esa propiedad. `selectorFecha` copia las medidas de `campo` para que el selector y los campos de texto se vean iguales.

screens/Registro.js líneas 808–885

```
808     casillas: {
809       gap: 14,
810       marginTop: 20,
811     },
812     filaCasilla: {
813       flexDirection: 'row',
814       alignItems: 'flex-start',
815       gap: 12,
816     },
817     casilla: {
818       width: 22,
819       height: 22,
820       borderRadius: 6,
821       borderWidth: 1.5,
822       borderColor: c.borde,
823       backgroundColor: c.superficie,
824       justifyContent: 'center',
825       alignItems: 'center',
826     },
827     casillaMarcada: {
828       borderColor: c.marca,
829       backgroundColor: c.marca,
830     },
831     textoCasilla: {
832       flex: 1,
833       fontFamily: 'Inter_400Regular',
834       fontSize: 14,
835       lineHeight: 20,
836       color: c.texto,
837     },
838     enlace: {
839       fontFamily: 'Inter_600SemiBold',
840       color: c.marca,
841       textDecorationLine: 'underline',
842     },
843     botonPrincipal: {
844       height: 54,
845       borderRadius: 14,
846       backgroundColor: c.marca,
847       justifyContent: 'center',
848       alignItems: 'center',
849       marginTop: 24,
850     },
851     botonDeshabilitado: {
852       backgroundColor: c.superficie,
853       borderWidth: 1,
854       borderColor: c.borde,
855     },
856     textoBotonPrincipal: {
857       fontFamily: 'Inter_600SemiBold',
858       fontSize: 16,
```

```
      color: c.textoSobreMarca,
    },
    textoBotonDeshabilitado: {
      color: c.textoTenue,
    },
    botonSecundario: {
      height: 54,
      borderRadius: 14,
      borderWidth: 1,
      borderColor: c.borde,
      justifyContent: 'center',
      alignItems: 'center',
      marginTop: 12,
    },
    textoBotonSecundario: {
      fontFamily: 'Inter_600SemiBold',
      fontSize: 16,
      color: c.texto,
    },
    presionado: {
      opacity: 0.85,
    },
    filaCargando: {
      flexDirection: 'row',
      alignItems: 'center',
      gap: 10,
    },
```

CASILLAS, ENLACES Y BOTONES

`filaCasilla` usa `alignItems: 'flex-start'` y no `center`: si el texto ocupa dos líneas, la casilla queda alineada con la primera.

screens/Registro.js líneas 886–946

```
    separador: {
      flexDirection: 'row',
      alignItems: 'center',
      gap: 16,
      marginVertical: 20,
    },
    linea: {
      flex: 1,
      height: 1,
      backgroundColor: c.divisor,
    },
    textoSeparador: {
      fontFamily: 'Inter_400Regular',
      fontSize: 14,
      color: c.textoSecundario,
    },
    botonSocial: {
      flexDirection: 'row',
      justifyContent: 'center',
      alignItems: 'center',
      gap: 10,
      height: 52,
      borderRadius: 14,
      borderWidth: 1,
      marginBottom: 12,
    },
    botonGoogle: {
      backgroundColor: '#FFFFFF',
      borderColor: '#E6E2D8',
```

```
    },
    botonApple: {
      backgroundColor: c.fondoApple,
      borderColor: c.bordeApple,
    },
    textoBotonSocial: {
      fontFamily: 'Inter_500Medium',
      fontSize: 16,
    },
    textoGoogle: {
      color: '#161616',
    },
    textoApple: {
      color: '#FFFFFF',
    },
    espaciador: {
      flex: 1,
      minHeight: 16,
    },
    filaPie: {
      flexDirection: 'row',
      justifyContent: 'center',
      alignItems: 'center',
      marginTop: 16,
    },
    textoPie: {
      fontFamily: 'Inter_400Regular',
      fontSize: 15,
      color: c.textoSecundario,
    },
  });
}
```

SEPARADOR, SOCIALES Y PIE

Los mismos valores del Login. Repetirlos en dos archivos es el próximo candidato a extraer en componentes, como se hizo con el campo.

10

Verificación de correo

`screens/Verificacion.js` implementa la vista **2.4** (Revisá tu correo) y la **2.8** (Cuenta verificada). La ve cualquier usuario con sesión cuyo correo todavía no está confirmado: el que acaba de registrarse y también el que se registró antes, no verificó y vuelve a iniciar sesión.

¿Y la vista 2.7, enlace vencido? Cuando el usuario toca el enlace del correo, quien lo abre es una página web de Firebase, no la app. Si el enlace venció, esa página lo informa. La app no se entera, así que no hay una pantalla para eso: el usuario vuelve y toca «Reenviar enlace». Para mostrar la vista 2.7 dentro de la app, el enlace tendría que abrir la app (deep links), igual que la vista 1.8 del Login.

10.1 Importaciones y constantes

screens/Verificacion.js — líneas 1–20

```
import React, { useEffect, useMemo, useState } from 'react';
import { ActivityIndicator, Image, Pressable, ScrollView, StyleSheet, Text, View } from 'react-native';
import { SafeAreaView } from 'react-native-safe-area-context';
import { StatusBar } from 'expo-status-bar';
import { Feather, Ionicons } from '@expo/vector-icons';
import { useFonts, Caveat_700Bold } from '@expo-google-fonts/caveat';
import { Inter_400Regular, Inter_500Medium, Inter_600SemiBold } from '@expo-google-fonts/inter';
import { deleteUser, sendEmailVerification, signOut } from 'firebase/auth';
import { auth } from '../src/config/firebaseConfig';
import { useTema } from '../src/tema/TemaContext';
import { useAviso } from '../src/aviso/AvisoContext';

const ESPERA_REENVIO = 60;
const INTERVALO_CONSULTA = 4000;

function formatearReloj(segundos) {
  const minutos = String(Math.floor(segundos / 60)).padStart(2, '0');
  const resto = String(segundos % 60).padStart(2, '0');
  return `${minutos}:${resto}`;
}
```

POR QUÉ ASÍ

Las funciones de Firebase son las que usa esta pantalla: `sendEmailVerification` para reenviar, `deleteUser` para corregir el correo y `signOut` para salir.

Números con nombre. `ESPERA_REENVIO = 60` y `INTERVALO_CONSULTA = 4000` dicen qué significan. Un `4000` suelto en medio del código obliga a adivinar si son segundos o milisegundos.

`formatearReloj` convierte 42 segundos en «00:42». `Math.floor` redondea hacia abajo y `%` da el resto de la división.

10.2 Firma, estados y correo

screens/Verificacion.js líneas 22–37

```
export default function Verificacion() {
  const [fuentesCargadas] = useFonts({
    Caveat_700Bold,
    Inter_400Regular,
    Inter_500Medium,
    Inter_600SemiBold,
  });

  const [verificado, setVerificado] = useState(false);
  const [segundos, setSegundos] = useState(ESPERA_REENVIO);
  const [entrando, setEntrando] = useState(false);

  const { colores, isotipo, cambiarTema } = useTema();
  const { avisar } = useAviso();
  const estilos = useMemo(() => crearEstilos(colores), [colores]);
  const correo = auth.currentUser?.email ?? '';
```

POR QUÉ ASÍ

La función no recibe `navigation` porque no navega: los cambios de pantalla los decide `Navigation.js`.

`auth.currentUser` es el usuario con la sesión activa. El `?.` evita un error si por un instante no hay usuario (por ejemplo, justo después de borrarlo), y `?? ''` deja texto vacío en ese caso.

`avisar` viene del contexto de avisos (3.4) y reemplaza a `Alert.alert` en toda la pantalla. `verificado` controla qué vista se ve; `segundos`, la cuenta regresiva del reenvío; `entrando`, el indicador del botón final.

10.3 Cuenta regresiva y consulta automática

screens/Verificacion.js líneas 39–56

```
  useEffect(() => {
    if (segundos === 0) return;
    const temporizador = setTimeout(() => setSegundos(segundos - 1), 1000);
    return () => clearTimeout(temporizador);
  }, [segundos]);

  useEffect(() => {
    if (verificado) return;
    const consulta = setInterval(() => {
      auth.currentUser
        .reload()
        .then(() => {
          if (auth.currentUser.emailVerified) setVerificado(true);
        })
        .catch(() => null);
    }, INTERVALO_CONSULTA);
    return () => clearInterval(consulta);
  }, [verificado]);
```

Qué hace. El primer efecto descuenta un segundo por vez hasta llegar a cero. El segundo pregunta a Firebase cada 4 segundos si el correo ya se verificó.

> POR QUÉ ASÍ
>
> **La cuenta regresiva con `setTimeout` y la dependencia `[segundos]`.** Cada vez que cambia `segundos`, el efecto programa el próximo descuento dentro de un segundo. Al llegar a cero, el `return` del principio corta la cadena. Es una forma de contar sin `setInterval`, donde habría que detener el intervalo a mano.
>
> **La función de limpieza.** Lo que devuelve un efecto se ejecuta antes del siguiente y al cerrar la pantalla. `clearTimeout` y `clearInterval` cancelan lo programado. Sin limpieza, los temporizadores seguirían corriendo con la pantalla ya cerrada.
>
> **Por qué consultar cada 4 segundos.** El usuario verifica en su aplicación de correo, fuera de Cafeto, y Firebase no avisa a la app. `reload()` pide a Firebase los datos actualizados del usuario; después, `emailVerified` ya muestra el valor nuevo. Cuando el usuario vuelve a la app, en pocos segundos la pantalla cambia sola.
>
> **`.then().catch()` y no `async/await`.** Si falla una consulta (por ejemplo, sin señal), `.catch(() ⇒ null)` la ignora y se reintenta en la siguiente vuelta. No hace falta avisar al usuario de cada intento fallido.
>
> Con `[verificado]` como dependencia, al pasar a verdadero el efecto se limpia y no vuelve a consultar.

10.4 Reenviar el enlace

screens/Verificacion.js — líneas 58-70

```
58   const reenviarEnlace = async () ⇒ {
59     try {
60       await sendEmailVerification(auth.currentUser);
61       setSegundos(ESPERA_REENVIO);
62       avisar('exito', 'Enlace enviado', `Te mandamos un nuevo enlace a ${correo}.`);
63     } catch (error) {
64       if (error.code === 'auth/too-many-requests') {
65         avisar('advertencia', 'Esperá un momento', 'Pediste varios enlaces seguidos. Probá de nuevo
en unos minutos.');
66       } else {
67         avisar('error', 'Error', `No pudimos reenviar el enlace.\n\nCódigo: ${error.code ??
'desconocido'}`);
68       }
69     }
70   };
```

> POR QUÉ ASÍ
>
> Después de reenviar, `setSegundos(ESPERA_REENVIO)` reinicia la cuenta regresiva, y el efecto de 10.3 vuelve a correr solo porque cambió su dependencia.
>
> Firebase limita cuántos correos se mandan seguidos y responde `auth/too-many-requests`. La espera de un minuto en el botón reduce ese caso, y si igual pasa, el mensaje explica qué hacer.

10.5 Corregir el correo

screens/Verificacion.js líneas 72–95

```
72   const borrarRegistro = async () => {
73     try {
74       await deleteUser(auth.currentUser);
75     } catch (error) {
76       await signOut(auth);
77       avisar(
78         'error',
79         'No pudimos borrar el registro',
80         'Iniciá sesión de nuevo con ese correo y volvé a intentarlo, o registrate con el correo
correcto.'
81       );
82     }
83   };
84
85   const corregirCorreo = () => {
86     avisar(
87       'advertencia',
88       '¿Corregir el correo?',
89       'Vamos a borrar este registro para que lo hagas de nuevo con el correo correcto.',
90       [
91         { texto: 'Cancelar', estilo: 'cancelar' },
92         { texto: 'Corregir', estilo: 'destructivo', alPresionar: borrarRegistro },
93       ]
94     );
95   };
```

POR QUÉ ASÍ

Confirmación antes de borrar. `avisar` acepta un cuarto argumento con botones (3.4). `estilo: 'cancelar'` dibuja un botón con borde, y `estilo: 'destructivo'` lo pinta de rojo para indicar que la acción no se puede deshacer. `alPresionar` es la función que se ejecuta al tocarlo.

Por qué borrar la cuenta. Si el correo está mal escrito, el enlace nunca llega y la cuenta queda inservible. Borrarla deja el correo libre y la navegación vuelve sola al Login, porque ya no hay sesión.

El plan B. Firebase solo deja borrar una cuenta si la sesión es reciente. Si pasó mucho tiempo, `deleteUser` falla; entonces se cierra la sesión y se explica qué hacer.

10.6 Entrar a la app

screens/Verificacion.js líneas 97–108

```
97   const empezar = async () => {
98     setEntrando(true);
99     await auth.currentUser.getIdToken(true);
100  };
101
102  if (!fuentesCargadas) {
103    return (
104      <View style={[estilos.pantalla, estilos.centrado]}>
105        <ActivityIndicator size="large" color={colores.marca} />
106      </View>
```

```
107     );
108   }
```

Qué hace. Al tocar «Empezar», renueva el token de la sesión. Eso dispara `onIdTokenChanged` en `Navigation.js`, que ve `emailVerified` en verdadero y muestra Home.

> POR QUÉ ASÍ
>
> `getIdToken(true)`. El `true` obliga a pedir un token nuevo aunque el actual no haya vencido. Es la pieza que conecta esta pantalla con la navegación (3.3) sin pasarle funciones ni usar `navigate`.
>
> **Por qué no entrar apenas se verifica.** Si la navegación cambiara sola, el usuario nunca vería la vista 2.8. Esperar a que toque «Empezar» le da la confirmación de que todo salió bien.
>
> Después va la salida temprana de las fuentes, igual que en las otras pantallas.

10.7 Contenedores y encabezado

screens/Verificacion.js — líneas 110-129

```
110   return (
111     <SafeAreaView style={estilos.pantalla}>
112       <StatusBar style={colores.barraEstado} />
113       <ScrollView contentContainerStyle={estilos.contenido} showsVerticalScrollIndicator={false}>
114         <View style={estilos.encabezado}>
115           {verificado ? (
116             <View style={estilos.huecoEncabezado} />
117           ) : (
118             <Pressable style={estilos.chip} onPress={() => signOut(auth)}>
119               <Text style={estilos.textoChip}>Salir</Text>
120               <Feather name="log-out" size={14} color={colores.textoSecundario} />
121             </Pressable>
122           )}
123           <Image source={isotipo} style={estilos.isotipo} resizeMode="contain" />
124           <View style={estilos.huecoEncabezado}>
125             <Pressable style={estilos.botonTema} onPress={cambiarTema}>
126               <Ionicons name="contrast" size={20} color={colores.texto} />
127             </Pressable>
128           </View>
129         </View>
```

> POR QUÉ ASÍ
>
> Sin campos de texto no hace falta `KeyboardAvoidingView`: alcanza con SafeAreaView y ScrollView.
>
> **«Salir» en lugar de «Ver la carta».** El diseño tenía un acceso a la carta, que todavía no existe. Se reemplazó por cerrar la sesión, para que nadie quede atrapado en esta pantalla si quiere entrar con otra cuenta.
>
> **Huecos del mismo ancho a los costados.** El chip y el hueco de la derecha miden 90 puntos. Así el isotipo queda centrado de verdad con `space-between`, aunque el

botón de tema sea más chico que el chip. Cuando ya está verificado, el chip se reemplaza por un hueco vacío para no mover el isotipo.

10.8 Cuenta verificada (vista 2.8)

screens/Verificacion.js líneas 131-151

```
131         {verificado ? (
132           <>
133             <View style={estilos.espaciador} />
134             <View style={[estilos.icono, estilos.iconoExito]}>
135               <Feather name="check" size={44} color={colores.exito} />
136             </View>
137             <Text style={estilos.titulo}>¡Cuenta verificada!</Text>
138             <Text style={estilos.texto}>Ya podés sumar puntos en cada compra.</Text>
139             <View style={estilos.espaciador} />
140             <Pressable
141               style={({ pressed }) => [estilos.botonPrincipal, pressed && estilos.presionado]}
142               onPress={empezar}
143               disabled={entrando}
144             >
145               {entrando ? (
146                 <ActivityIndicator color={colores.textoSobreMarca} />
147               ) : (
148                 <Text style={estilos.textoBotonPrincipal}>Empezar</Text>
149               )}
150             </Pressable>
151           </>
```

POR QUÉ ASÍ

Dos espaciadores, uno arriba y otro abajo del mensaje, lo centran en la pantalla y dejan el botón al fondo.

El diseño ofrecía «10% off en tu primera compra». Los beneficios los define el administrador en el sistema de PC, así que la app no promete uno que quizás no exista. El texto se limita a lo que sí es seguro: que ya puede sumar puntos.

10.9 Revisá tu correo (vista 2.4)

screens/Verificacion.js líneas 152-194

```
152         ) : (
153           <>
154             <View style={[estilos.icono, estilos.iconoCorreo]}>
155               <Feather name="mail" size={40} color={colores.marca} />
156             </View>
157             <Text style={estilos.titulo}>Revisá tu correo</Text>
158             <Text style={estilos.texto}>Te mandamos un enlace de verificación para activar la cuenta.
</Text>
159             <Text style={estilos.correo}>{correo}</Text>
160             <View style={estilos.filaCorregir}>
161               <Text style={estilos.textoChico}>¿Está mal escrito? </Text>
162               <Pressable onPress={corregirCorreo}>
163                 <Text style={[estilos.textoChico, estilos.enlace]}>Corregilo</Text>
164               </Pressable>
165             </View>
166
167             <View style={estilos.tarjetaExito}>
```

```
              <Feather name="check" size={18} color={colores.exito} />
              <View style={estilos.textosTarjeta}>
                <Text style={estilos.tituloTarjeta}>Cuenta creada</Text>
                <Text style={estilos.detalleTarjeta}>
                  Solo falta verificar el correo para empezar a sumar puntos.
                </Text>
              </View>
            </View>

            <Pressable
              style={({ pressed }) => [estilos.botonSecundario, pressed && estilos.presionado]}
              onPress={reenviarEnlace}
              disabled={segundos > 0}
            >
              <Text style={[estilos.textoBotonSecundario, segundos > 0 && estilos.textoEsperando]}>
                {segundos > 0 ? `Reenviar enlace en ${formatearReloj(segundos)}` : 'Reenviar
enlace'}
              </Text>
            </Pressable>

            <View style={estilos.espaciador} />
            <Text style={estilos.pie}>¿No te llegó? Revisá la carpeta de spam.</Text>
          </>
        )}
      </ScrollView>
    </SafeAreaView>
  );
}
```

POR QUÉ ASÍ

El botón de reenvío se deshabilita mientras corre la cuenta y su texto muestra el tiempo que falta. El estilo `textoEsperando` lo pinta en gris para que se note que todavía no se puede tocar.

La tarjeta verde confirma que la cuenta ya existe. Sin ella, el usuario podría pensar que el registro falló y volver a intentarlo.

10.10 Estilos

screens/Verificacion.js — líneas 196-376

```
function crearEstilos(c) {
  return StyleSheet.create({
    pantalla: {
      flex: 1,
      backgroundColor: c.fondo,
    },
    centrado: {
      justifyContent: 'center',
      alignItems: 'center',
    },
    contenido: {
      flexGrow: 1,
      paddingHorizontal: 28,
      paddingTop: 12,
      paddingBottom: 24,
    },
    encabezado: {
      flexDirection: 'row',
      justifyContent: 'space-between',
      alignItems: 'center',
```

```
  },
  huecoEncabezado: {
    width: 90,
    alignItems: 'flex-end',
  },
  chip: {
    width: 90,
    flexDirection: 'row',
    justifyContent: 'center',
    alignItems: 'center',
    gap: 6,
    height: 38,
    borderRadius: 19,
    borderWidth: 1,
    borderColor: c.borde,
    backgroundColor: c.superficie,
  },
  textoChip: {
    fontFamily: 'Inter_500Medium',
    fontSize: 14,
    color: c.textoSecundario,
  },
  isotipo: {
    width: 34,
    height: 34,
  },
  botonTema: {
    width: 40,
    height: 40,
    borderRadius: 20,
    borderWidth: 1,
    borderColor: c.borde,
    backgroundColor: c.superficie,
    justifyContent: 'center',
    alignItems: 'center',
  },
  icono: {
    alignSelf: 'center',
    justifyContent: 'center',
    alignItems: 'center',
  },
  iconoCorreo: {
    width: 88,
    height: 88,
    borderRadius: 24,
    marginTop: 40,
    backgroundColor: c.marcaSuave,
  },
  iconoExito: {
    width: 96,
    height: 96,
    borderRadius: 48,
    backgroundColor: c.fondoExito,
  },
  titulo: {
    fontFamily: 'Caveat_700Bold',
    fontSize: 38,
    color: c.titulo,
    textAlign: 'center',
    marginTop: 24,
  },
  texto: {
    fontFamily: 'Inter_400Regular',
    fontSize: 15,
    lineHeight: 22,
    color: c.textoSecundario,
    textAlign: 'center',
    marginTop: 6,
```

```
  },
  correo: {
    fontFamily: 'Inter_600SemiBold',
    fontSize: 16,
    color: c.marca,
    textAlign: 'center',
    marginTop: 12,
  },
  filaCorregir: {
    flexDirection: 'row',
    justifyContent: 'center',
    alignItems: 'center',
    marginTop: 6,
  },
  textoChico: {
    fontFamily: 'Inter_400Regular',
    fontSize: 13,
    color: c.textoSecundario,
  },
  enlace: {
    fontFamily: 'Inter_600SemiBold',
    color: c.marca,
    textDecorationLine: 'underline',
  },
  tarjetaExito: {
    flexDirection: 'row',
    gap: 12,
    padding: 16,
    marginTop: 28,
    borderRadius: 14,
    borderWidth: 1,
    borderColor: c.exito,
    backgroundColor: c.fondoExito,
  },
  textosTarjeta: {
    flex: 1,
    gap: 4,
  },
  tituloTarjeta: {
    fontFamily: 'Inter_600SemiBold',
    fontSize: 14,
    color: c.texto,
  },
  detalleTarjeta: {
    fontFamily: 'Inter_400Regular',
    fontSize: 13.5,
    lineHeight: 19,
    color: c.textoSecundario,
  },
  botonPrincipal: {
    height: 54,
    borderRadius: 14,
    backgroundColor: c.marca,
    justifyContent: 'center',
    alignItems: 'center',
  },
  textoBotonPrincipal: {
    fontFamily: 'Inter_600SemiBold',
    fontSize: 16,
    color: c.textoSobreMarca,
  },
  botonSecundario: {
    height: 54,
    borderRadius: 14,
    borderWidth: 1,
    borderColor: c.borde,
    justifyContent: 'center',
    alignItems: 'center',
```

```
      marginTop: 16,
    },
    textoBotonSecundario: {
      fontFamily: 'Inter_600SemiBold',
      fontSize: 16,
      color: c.texto,
    },
    textoEsperando: {
      color: c.textoSecundario,
    },
    presionado: {
      opacity: 0.85,
    },
    espaciador: {
      flex: 1,
      minHeight: 24,
    },
    pie: {
      fontFamily: 'Inter_400Regular',
      fontSize: 14,
      color: c.textoSecundario,
      textAlign: 'center',
    },
  });
}
```

POR QUÉ ASÍ

Estilo base más variante. `icono` centra el contenido y las variantes `iconoCorreo` (cuadrado redondeado ámbar) e `iconoExito` (círculo verde) agregan tamaño y color. Es la misma composición que `botonSocial` con `botonGoogle`.

Un círculo es un cuadrado con `borderRadius` igual a la mitad del lado (96 → 48).

11

Cómo razonar la estructura por tu cuenta

Estas son las preguntas que llevaron a cada decisión del archivo. Sirven para cualquier pantalla nueva.

1. **¿Qué cambia en esta pantalla?** Cada cosa que cambia mientras la usás (lo que se escribe, qué campo está activo, si está cargando) es un `useState`. Lo que nunca cambia va en constantes fuera del componente.
2. **¿Se puede calcular a partir de otro estado?** Entonces no es un estado: es una variable (`colores`) o un `useMemo` (`estilos`).
3. **¿Pasa algo al abrir la pantalla?** Va en un `useEffect` con `[]`.
4. **¿El usuario toca algo?** Cada acción es una función con nombre de verbo (`iniciarSesion`, `cambiarTema`). Si dos acciones repiten código, ese código pasa a una función compartida (`mostrarErrorDeFirebase`).
5. **¿Cómo se acomoda la pantalla?** Mirá el diseño en filas y columnas. Cada grupo horizontal es un View con `flexDirection: 'row'`; todo lo demás se apila en columna, que es el valor por defecto en React Native.
6. **¿Qué estilos se repiten?** Se escriben una vez y se combinan en arreglos (`botonSocial` + `botonGoogle`).
7. **¿Un bloque se repite tres veces o más?** Pasa a ser un componente con props, como `CampoTexto` (8). ¿Se repite dentro de un solo archivo y no tiene estado? Alcanza una función que devuelve JSX, como `casilla` (9.16).
8. **¿Un dato lo usan varias pantallas?** Va en un contexto por encima de todas, como el tema (3.1). Si lo usa una sola, queda en esa pantalla.
9. **¿Un formulario tiene muchos campos?** Los errores van en un solo objeto, y las funciones repetidas se fabrican con una función que devuelve funciones (9.9).
10. **¿Qué pasa en los casos difíciles?** Teclado abierto, pantalla chica, sin conexión, doble toque, fuentes sin cargar. Cada uno tiene su respuesta en el código: KeyboardAvoidingView, ScrollView, la alerta de red, `disabled`, la salida temprana.

12

Código completo

Los nueve archivos íntegros, idénticos a los del proyecto. Un script los lee directamente de la carpeta, así que coinciden línea por línea. Cada uno tiene su botón para copiarlo.

12.1 screens/Login.js

screens/Login.js · archivo completo — 640 líneas

```
import React, { useEffect, useMemo, useState } from 'react';
import {
  ActivityIndicator,
  Image,
  KeyboardAvoidingView,
  Platform,
  Pressable,
  ScrollView,
  StyleSheet,
  Text,
  TextInput,
  View,
} from 'react-native';
import { SafeAreaView } from 'react-native-safe-area-context';
import { StatusBar } from 'expo-status-bar';
import { Feather, FontAwesome, Ionicons } from '@expo/vector-icons';
import { useFonts, Caveat_700Bold } from '@expo-google-fonts/caveat';
import { Inter_400Regular, Inter_500Medium, Inter_600SemiBold } from '@expo-google-fonts/inter';
import AsyncStorage from '@react-native-async-storage/async-storage';
import { sendPasswordResetEmail, signInAnonymously, signInWithEmailAndPassword } from
'firebase/auth';
import { auth } from '../src/config/firebaseConfig';
import { useTema } from '../src/tema/TemaContext';
import { useAviso } from '../src/aviso/AvisoContext';

const CLAVE_CORREO_RECORDADO = 'cafeto:correoRecordado';
const FORMATO_CORREO = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;

export default function Login({ navigation, route }) {
  const [fuentesCargadas] = useFonts({
    Caveat_700Bold,
    Inter_400Regular,
    Inter_500Medium,
    Inter_600SemiBold,
  });

  const [correo, setCorreo] = useState('');
  const [contrasena, setContrasena] = useState('');
  const [mostrarContrasena, setMostrarContrasena] = useState(false);
  const [recordarme, setRecordarme] = useState(false);
  const [campoActivo, setCampoActivo] = useState(null);
  const [errorCorreo, setErrorCorreo] = useState('');
  const [errorContrasena, setErrorContrasena] = useState('');
  const [cargando, setCargando] = useState(false);

  const { colores, logo, cambiarTema } = useTema();
  const { avisar } = useAviso();
  const estilos = useMemo(() => crearEstilos(colores), [colores]);
  const formularioCompleto = correo.trim() !== '' && contrasena !== '';

  useEffect(() => {
```

```
    AsyncStorage.getItem(CLAVE_CORREO_RECORDADO).then((correoGuardado) => {
      if (correoGuardado) {
        setCorreo((actual) => actual || correoGuardado);
        setRecordarme(true);
      }
    });
  }, []);

  useEffect(() => {
    if (route.params?.correo) {
      setCorreo(route.params.correo);
      setErrorCorreo('');
    }
  }, [route.params?.correo]);

  const cambiarCorreo = (texto) => {
    setCorreo(texto);
    setErrorCorreo('');
  };

  const cambiarContrasena = (texto) => {
    setContrasena(texto);
    setErrorContrasena('');
  };

  const validarFormulario = () => {
    let esValido = true;

    if (!correo.trim()) {
      setErrorCorreo('Ingresá tu correo electrónico.');
      esValido = false;
    } else if (!FORMATO_CORREO.test(correo.trim())) {
      setErrorCorreo('Revisá el formato del correo.');
      esValido = false;
    }

    if (!contrasena) {
      setErrorContrasena('Ingresá tu contraseña.');
      esValido = false;
    }

    return esValido;
  };

  const guardarPreferenciaRecordarme = async () => {
    if (recordarme) {
      await AsyncStorage.setItem(CLAVE_CORREO_RECORDADO, correo.trim());
    } else {
      await AsyncStorage.removeItem(CLAVE_CORREO_RECORDADO);
    }
  };

  const mostrarErrorDeFirebase = (error) => {
    switch (error.code) {
      case 'auth/invalid-email':
        setErrorCorreo('El formato del correo no es válido.');
        break;
      case 'auth/invalid-credential':
      case 'auth/wrong-password':
      case 'auth/user-not-found':
        setErrorContrasena('La contraseña no coincide con ese correo.');
        break;
      case 'auth/too-many-requests':
        setErrorContrasena('Demasiados intentos. Esperá unos minutos y probá de nuevo.');
        break;
      case 'auth/network-request-failed':
        avisar('error', 'Sin conexión', 'Revisá el wifi o los datos del teléfono.');
        break;
```

```
119       case 'auth/configuration-not-found':
120       case 'auth/admin-restricted-operation':
121       case 'auth/operation-not-allowed':
122         avisar('advertencia', 'No disponible', 'Este método de acceso no está habilitado en
Firebase.');
123         break;
124       default:
125         avisar('error', 'Error', `No pudimos iniciar sesión.\n\nCódigo: ${error.code ??
'desconocido'}`);
126     }
127   };
128
129   const iniciarSesion = async () => {
130     if (!validarFormulario()) return;
131
132     setCargando(true);
133     try {
134       await guardarPreferenciaRecordarme();
135       await signInWithEmailAndPassword(auth, correo.trim(), contrasena);
136     } catch (error) {
137       mostrarErrorDeFirebase(error);
138       setCargando(false);
139     }
140   };
141
142   const recuperarContrasena = async () => {
143     if (!FORMATO_CORREO.test(correo.trim())) {
144       setErrorCorreo('Escribí tu correo y te mandamos un enlace para elegir una nueva.');
145       return;
146     }
147
148     try {
149       await sendPasswordResetEmail(auth, correo.trim());
150       avisar('exito', 'Revisá tu correo', `Te mandamos un enlace a ${correo.trim()} para que elijas
una contraseña nueva.`);
151     } catch (error) {
152       mostrarErrorDeFirebase(error);
153     }
154   };
155
156   const seguirSinCuenta = async () => {
157     setCargando(true);
158     try {
159       await signInAnonymously(auth);
160     } catch (error) {
161       mostrarErrorDeFirebase(error);
162       setCargando(false);
163     }
164   };
165
166   const avisarProximamente = (proveedor) => {
167     avisar('advertencia', 'Próximamente', `El ingreso con ${proveedor} todavía no está
disponible.`);
168   };
169
170   if (!fuentesCargadas) {
171     return (
172       <View style={[estilos.pantalla, estilos.centrado]}>
173         <ActivityIndicator size="large" color={colores.marca} />
174       </View>
175     );
176   }
177
178   return (
179     <SafeAreaView style={estilos.pantalla}>
180       <StatusBar style={colores.barraEstado} />
181       <KeyboardAvoidingView
182         style={estilos.pantalla}
```

```
        behavior={Platform.OS === 'ios' ? 'padding' : undefined}
      >
        <ScrollView
          contentContainerStyle={estilos.contenido}
          keyboardShouldPersistTaps="handled"
          showsVerticalScrollIndicator={false}
        >
          <View style={estilos.barraSuperior}>
            <Pressable style={estilos.chipInvitado} onPress={seguirSinCuenta} disabled={cargando}>
              <Text style={estilos.textoChip}>Seguir sin cuenta</Text>
              <Feather name="chevron-right" size={16} color={colores.textoSecundario} />
            </Pressable>
            <Pressable style={estilos.botonTema} onPress={cambiarTema}>
              <Ionicons name="contrast" size={20} color={colores.texto} />
            </Pressable>
          </View>

          <View style={estilos.marca}>
            <Image source={logo} style={estilos.logo} resizeMode="contain" />
            <Text style={estilos.lomitas}>LAS LOMITAS</Text>
          </View>

          <Text style={estilos.titulo}>Hola de nuevo</Text>
          <Text style={estilos.subtitulo}>Qué bueno verte.</Text>

          <Text style={estilos.etiqueta}>Correo electrónico</Text>
          <View
            style={[
              estilos.campo,
              campoActivo === 'correo' && estilos.campoActivo,
              errorCorreo !== '' && estilos.campoConError,
            ]}
          >
            <Feather name="mail" size={20} color={colores.textoSecundario} />
            <TextInput
              style={estilos.entrada}
              placeholder="tunombre@correo.com"
              placeholderTextColor={colores.textoTenue}
              value={correo}
              onChangeText={cambiarCorreo}
              onFocus={() => setCampoActivo('correo')}
              onBlur={() => setCampoActivo(null)}
              keyboardType="email-address"
              autoCapitalize="none"
              autoCorrect={false}
              autoComplete="email"
              returnKeyType="next"
              editable={!cargando}
            />
          </View>
          {errorCorreo !== '' && (
            <View style={estilos.filaError}>
              <Feather name="alert-circle" size={14} color={colores.peligro} />
              <Text style={estilos.textoError}>{errorCorreo}</Text>
            </View>
          )}
          {campoActivo === 'correo' && errorCorreo === '' && (
            <Text style={estilos.ayuda}>Usá el correo con el que te registraste.</Text>
          )}

          <Text style={estilos.etiqueta}>Contraseña</Text>
          <View
            style={[
              estilos.campo,
              campoActivo === 'contrasena' && estilos.campoActivo,
              errorContrasena !== '' && estilos.campoConError,
            ]}
          >
```

```
            <Feather name="lock" size={20} color={colores.textoSecundario} />
            <TextInput
              style={estilos.entrada}
              placeholder="Tu contraseña"
              placeholderTextColor={colores.textoTenue}
              value={contrasena}
              onChangeText={cambiarContrasena}
              onFocus={() => setCampoActivo('contrasena')}
              onBlur={() => setCampoActivo(null)}
              secureTextEntry={!mostrarContrasena}
              autoCapitalize="none"
              autoComplete="password"
              returnKeyType="go"
              onSubmitEditing={iniciarSesion}
              editable={!cargando}
            />
            <Pressable onPress={() => setMostrarContrasena(!mostrarContrasena)} hitSlop={10}>
              <Feather
                name={mostrarContrasena ? 'eye-off' : 'eye'}
                size={20}
                color={colores.textoSecundario}
              />
            </Pressable>
          </View>
          {errorContrasena !== '' && (
            <View style={estilos.filaError}>
              <Feather name="alert-circle" size={14} color={colores.peligro} />
              <Text style={estilos.textoError}>{errorContrasena}</Text>
            </View>
          )}

          <View style={estilos.filaOpciones}>
            <Pressable style={estilos.recordarme} onPress={() => setRecordarme(!recordarme)}>
              <View style={[estilos.casilla, recordarme && estilos.casillaMarcada]}>
                {recordarme && <Feather name="check" size={16} color={colores.textoSobreMarca} />}
              </View>
              <Text style={estilos.textoRecordarme}>Recordarme</Text>
            </Pressable>
            <Pressable onPress={recuperarContrasena}>
              <Text style={estilos.enlace}>¿Olvidaste tu contraseña?</Text>
            </Pressable>
          </View>

          <Pressable
            style={({ pressed }) => [
              estilos.botonPrincipal,
              !formularioCompleto && estilos.botonDeshabilitado,
              pressed && estilos.presionado,
            ]}
            onPress={iniciarSesion}
            disabled={cargando || !formularioCompleto}
          >
            {cargando ? (
              <View style={estilos.filaCargando}>
                <ActivityIndicator color={colores.textoSobreMarca} />
                <Text style={estilos.textoBotonPrincipal}>Ingresando...</Text>
              </View>
            ) : (
              <Text
                style={[estilos.textoBotonPrincipal, !formularioCompleto &&
estilos.textoBotonDeshabilitado]}
              >
                Iniciar sesión
              </Text>
            )}
          </Pressable>

          {cargando ? (
```

```
            <Text style={estilos.textoVerificando}>Un momento, estamos verificando tus datos.
</Text>
          ) : (
            <>
              <View style={estilos.separador}>
                <View style={estilos.linea} />
                <Text style={estilos.textoSeparador}>o</Text>
                <View style={estilos.linea} />
              </View>

              <Pressable
                style={({ pressed }) => [estilos.botonSocial, estilos.botonGoogle, pressed &&
estilos.presionado]}
                onPress={() => avisarProximamente('Google')}
              >
                <FontAwesome name="google" size={20} color="#4285F4" />
                <Text style={[estilos.textoBotonSocial, estilos.textoGoogle]}>Continuar con
Google</Text>
              </Pressable>

              <Pressable
                style={({ pressed }) => [estilos.botonSocial, estilos.botonApple, pressed &&
estilos.presionado]}
                onPress={() => avisarProximamente('Apple')}
              >
                <FontAwesome name="apple" size={22} color="#FFFFFF" />
                <Text style={[estilos.textoBotonSocial, estilos.textoApple]}>Continuar con
Apple</Text>
              </Pressable>
            </>
          )}

          <View style={estilos.espaciador} />

          <View style={estilos.filaRegistro}>
            <Text style={estilos.textoPie}>¿No tenés cuenta? </Text>
            <Pressable onPress={() => navigation.navigate('Registro')}>
              <Text style={estilos.enlaceRegistro}>Registrate</Text>
            </Pressable>
          </View>

          {!cargando && (
            <Text style={estilos.legal}>
              Al continuar aceptás los <Text style={estilos.subrayado}>Términos y Condiciones</Text>
y la{' '}
              <Text style={estilos.subrayado}>Política de privacidad</Text>.
            </Text>
          )}
        </ScrollView>
      </KeyboardAvoidingView>
    </SafeAreaView>
  );
}

function crearEstilos(c) {
  return StyleSheet.create({
    pantalla: {
      flex: 1,
      backgroundColor: c.fondo,
    },
    centrado: {
      justifyContent: 'center',
      alignItems: 'center',
    },
    contenido: {
      flexGrow: 1,
      paddingHorizontal: 28,
      paddingTop: 12,
```

```
    paddingBottom: 24,
  },
  barraSuperior: {
    flexDirection: 'row',
    justifyContent: 'space-between',
    alignItems: 'center',
  },
  chipInvitado: {
    flexDirection: 'row',
    alignItems: 'center',
    gap: 4,
    height: 38,
    paddingHorizontal: 16,
    borderRadius: 19,
    borderWidth: 1,
    borderColor: c.borde,
    backgroundColor: c.superficie,
  },
  textoChip: {
    fontFamily: 'Inter_500Medium',
    fontSize: 14,
    color: c.textoSecundario,
  },
  botonTema: {
    width: 40,
    height: 40,
    borderRadius: 20,
    borderWidth: 1,
    borderColor: c.borde,
    backgroundColor: c.superficie,
    justifyContent: 'center',
    alignItems: 'center',
  },
  marca: {
    alignItems: 'center',
    marginTop: 8,
  },
  logo: {
    width: 92,
    height: 95,
  },
  lomitas: {
    fontFamily: 'Inter_500Medium',
    fontSize: 11,
    letterSpacing: 4,
    color: c.textoSecundario,
    marginTop: 6,
  },
  titulo: {
    fontFamily: 'Caveat_700Bold',
    fontSize: 40,
    color: c.titulo,
    textAlign: 'center',
    marginTop: 24,
  },
  subtitulo: {
    fontFamily: 'Inter_400Regular',
    fontSize: 15,
    color: c.textoSecundario,
    textAlign: 'center',
    marginBottom: 8,
  },
  etiqueta: {
    fontFamily: 'Inter_600SemiBold',
    fontSize: 13.5,
    color: c.texto,
    marginTop: 16,
    marginBottom: 8,
```

```
  },
  campo: {
    flexDirection: 'row',
    alignItems: 'center',
    gap: 12,
    height: 52,
    paddingHorizontal: 16,
    borderRadius: 14,
    borderWidth: 1.5,
    borderColor: c.borde,
    backgroundColor: c.superficie,
  },
  campoActivo: {
    borderColor: c.marca,
    backgroundColor: c.superficieActiva,
  },
  campoConError: {
    borderColor: c.peligro,
  },
  entrada: {
    flex: 1,
    height: '100%',
    fontFamily: 'Inter_400Regular',
    fontSize: 16,
    color: c.texto,
  },
  filaError: {
    flexDirection: 'row',
    alignItems: 'center',
    gap: 6,
    marginTop: 6,
  },
  textoError: {
    flex: 1,
    fontFamily: 'Inter_400Regular',
    fontSize: 12.5,
    color: c.peligro,
  },
  ayuda: {
    fontFamily: 'Inter_400Regular',
    fontSize: 12.5,
    color: c.textoSecundario,
    marginTop: 6,
  },
  filaOpciones: {
    flexDirection: 'row',
    justifyContent: 'space-between',
    alignItems: 'center',
    marginTop: 16,
  },
  recordarme: {
    flexDirection: 'row',
    alignItems: 'center',
    gap: 10,
  },
  casilla: {
    width: 22,
    height: 22,
    borderRadius: 6,
    borderWidth: 1.5,
    borderColor: c.borde,
    backgroundColor: c.superficie,
    justifyContent: 'center',
    alignItems: 'center',
  },
  casillaMarcada: {
    borderColor: c.marca,
    backgroundColor: c.marca,
```

```
  },
  textoRecordarme: {
    fontFamily: 'Inter_400Regular',
    fontSize: 14,
    color: c.texto,
  },
  enlace: {
    fontFamily: 'Inter_600SemiBold',
    fontSize: 14,
    color: c.marca,
    textDecorationLine: 'underline',
  },
  botonPrincipal: {
    height: 54,
    borderRadius: 14,
    backgroundColor: c.marca,
    justifyContent: 'center',
    alignItems: 'center',
    marginTop: 24,
  },
  botonDeshabilitado: {
    backgroundColor: c.superficie,
    borderWidth: 1,
    borderColor: c.borde,
  },
  presionado: {
    opacity: 0.85,
  },
  filaCargando: {
    flexDirection: 'row',
    alignItems: 'center',
    gap: 10,
  },
  textoBotonPrincipal: {
    fontFamily: 'Inter_600SemiBold',
    fontSize: 16,
    color: c.textoSobreMarca,
  },
  textoBotonDeshabilitado: {
    color: c.textoTenue,
  },
  textoVerificando: {
    fontFamily: 'Inter_400Regular',
    fontSize: 13.5,
    color: c.textoSecundario,
    textAlign: 'center',
    marginTop: 12,
  },
  separador: {
    flexDirection: 'row',
    alignItems: 'center',
    gap: 16,
    marginVertical: 20,
  },
  linea: {
    flex: 1,
    height: 1,
    backgroundColor: c.divisor,
  },
  textoSeparador: {
    fontFamily: 'Inter_400Regular',
    fontSize: 14,
    color: c.textoSecundario,
  },
  botonSocial: {
    flexDirection: 'row',
    justifyContent: 'center',
    alignItems: 'center',
```

```
      gap: 10,
      height: 52,
      borderRadius: 14,
      borderWidth: 1,
      marginBottom: 12,
    },
    botonGoogle: {
      backgroundColor: '#FFFFFF',
      borderColor: '#E6E2D8',
    },
    botonApple: {
      backgroundColor: c.fondoApple,
      borderColor: c.bordeApple,
    },
    textoBotonSocial: {
      fontFamily: 'Inter_500Medium',
      fontSize: 16,
    },
    textoGoogle: {
      color: '#161616',
    },
    textoApple: {
      color: '#FFFFFF',
    },
    espaciador: {
      flex: 1,
    },
    filaRegistro: {
      flexDirection: 'row',
      justifyContent: 'center',
      alignItems: 'center',
      marginTop: 16,
    },
    textoPie: {
      fontFamily: 'Inter_400Regular',
      fontSize: 15,
      color: c.textoSecundario,
    },
    enlaceRegistro: {
      fontFamily: 'Inter_600SemiBold',
      fontSize: 15,
      color: c.marca,
      textDecorationLine: 'underline',
    },
    legal: {
      fontFamily: 'Inter_400Regular',
      fontSize: 12.5,
      lineHeight: 19,
      color: c.textoTenue,
      textAlign: 'center',
      marginTop: 12,
    },
    subrayado: {
      textDecorationLine: 'underline',
    },
  });
}
```

12.2 screens/Registro.js

screens/Registro.js · archivo completo — 946 líneas

```
import React, { useMemo, useState } from 'react';
import {
  ActivityIndicator,
```

```
  Image,
  KeyboardAvoidingView,
  Platform,
  Pressable,
  ScrollView,
  StyleSheet,
  Text,
  View,
} from 'react-native';
import { SafeAreaView } from 'react-native-safe-area-context';
import { StatusBar } from 'expo-status-bar';
import { Feather, FontAwesome, Ionicons } from '@expo/vector-icons';
import DateTimePicker from '@react-native-community/datetimepicker';
import { useFonts, Caveat_700Bold } from '@expo-google-fonts/caveat';
import { Inter_400Regular, Inter_500Medium, Inter_600SemiBold } from '@expo-google-fonts/inter';
import AsyncStorage from '@react-native-async-storage/async-storage';
import {
  createUserWithEmailAndPassword,
  sendEmailVerification,
  sendPasswordResetEmail,
  updateProfile,
} from 'firebase/auth';
import { auth } from '../src/config/firebaseConfig';
import { useTema } from '../src/tema/TemaContext';
import { useAviso } from '../src/aviso/AvisoContext';
import CampoTexto from '../src/componentes/CampoTexto';

const FORMATO_CORREO = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
const SOLO_LETRAS = /^[A-Za-zÁÉÍÓÚÜÑáéíóúüñ' -]+$/;
const NIVELES = ['muy débil', 'débil', 'regular', 'buena', 'fuerte'];

function medirFortaleza(clave) {
  const pruebas = [
    { cumple: clave.length >= 8, consejo: 'Usá 8 caracteres o más' },
    { cumple: /[a-z]/.test(clave) && /[A-Z]/.test(clave), consejo: 'Combiná mayúsculas y minúsculas'
},
    { cumple: /\d/.test(clave), consejo: 'Sumá un número' },
    { cumple: /[^A-Za-z0-9]/.test(clave), consejo: 'Sumá un símbolo' },
  ];
  const puntaje = pruebas.filter((prueba) => prueba.cumple).length;
  const pendiente = pruebas.find((prueba) => !prueba.cumple);

  return {
    puntaje,
    nivel: NIVELES[puntaje],
    consejo: pendiente ? pendiente.consejo : 'Excelente',
    segura: pruebas[0].cumple && pruebas[1].cumple && pruebas[2].cumple,
  };
}

function calcularEdad(nacimiento) {
  const hoy = new Date();
  let edad = hoy.getFullYear() - nacimiento.getFullYear();
  const yaCumplio =
    hoy.getMonth() > nacimiento.getMonth() ||
    (hoy.getMonth() === nacimiento.getMonth() && hoy.getDate() >= nacimiento.getDate());
  if (!yaCumplio) edad -= 1;
  return edad;
}

function formatearFecha(fecha) {
  const dia = String(fecha.getDate()).padStart(2, '0');
  const mes = String(fecha.getMonth() + 1).padStart(2, '0');
  return `${dia}/${mes}/${fecha.getFullYear()}`;
}

export default function Registro({ navigation }) {
  const [fuentesCargadas] = useFonts({
```

```
    Caveat_700Bold,
    Inter_400Regular,
    Inter_500Medium,
    Inter_600SemiBold,
  });

  const [paso, setPaso] = useState(1);
  const [nombre, setNombre] = useState('');
  const [apellido, setApellido] = useState('');
  const [correo, setCorreo] = useState('');
  const [contrasena, setContrasena] = useState('');
  const [repetir, setRepetir] = useState('');
  const [telefono, setTelefono] = useState('');
  const [fechaNacimiento, setFechaNacimiento] = useState(null);
  const [mostrarCalendario, setMostrarCalendario] = useState(false);
  const [aceptaTerminos, setAceptaTerminos] = useState(false);
  const [aceptaPromociones, setAceptaPromociones] = useState(false);
  const [errores, setErrores] = useState({});
  const [correoYaRegistrado, setCorreoYaRegistrado] = useState(false);
  const [creando, setCreando] = useState(false);

  const { tema, colores, isotipo, cambiarTema } = useTema();
  const { avisar } = useAviso();
  const estilos = useMemo(() => crearEstilos(colores), [colores]);

  const fortaleza = medirFortaleza(contrasena);
  const coinciden = repetir !== '' && repetir === contrasena;
  const paso1Completo =
    nombre.trim() !== '' && apellido.trim() !== '' && correo.trim() !== '' && contrasena !== '' &&
repetir !== '';

  const limpiarError = (campo) => {
    setErrores((anteriores) => ({ ...anteriores, [campo]: '' }));
  };

  const alCambiar = (campo, setter) => (texto) => {
    setter(texto);
    limpiarError(campo);
    if (campo === 'correo') setCorreoYaRegistrado(false);
  };

  const validarNombre = (valor, etiqueta) => {
    if (!valor.trim()) return `Ingresá tu ${etiqueta}.`;
    if (!SOLO_LETRAS.test(valor.trim())) return 'Usá solo letras.';
    return '';
  };

  const validarPaso1 = () => {
    const nuevos = {};

    const errorNombre = validarNombre(nombre, 'nombre');
    if (errorNombre) nuevos.nombre = errorNombre;

    const errorApellido = validarNombre(apellido, 'apellido');
    if (errorApellido) nuevos.apellido = errorApellido;

    if (!correo.includes('@')) {
      nuevos.correo = 'Falta la @. Ejemplo: maxi@cafeto.com.ar';
    } else if (!FORMATO_CORREO.test(correo.trim())) {
      nuevos.correo = 'Falta el dominio. Ejemplo: maxi@cafeto.com.ar';
    }

    if (!fortaleza.segura) {
      nuevos.contrasena = 'Usá 8 caracteres o más, con mayúsculas, minúsculas y un número.';
    }

    if (repetir !== contrasena) {
      nuevos.repetir = 'Las contraseñas no coinciden.';
```

```
    }

    setErrores(nuevos);
    return Object.keys(nuevos).length === 0;
  };

  const validarPaso2 = () => {
    const nuevos = {};

    if (telefono.trim()) {
      const digitos = telefono.replace(/\D/g, '');
      if (digitos.length < 7 || digitos.length > 15) {
        nuevos.telefono = 'El teléfono son entre 7 y 15 dígitos.';
      }
    }

    if (fechaNacimiento) {
      const edad = calcularEdad(fechaNacimiento);
      if (edad < 14) nuevos.fecha = 'Tenés que tener al menos 14 años.';
      if (edad > 100) nuevos.fecha = 'Revisá el año de nacimiento.';
    }

    if (!aceptaTerminos) {
      nuevos.terminos = 'Para crear la cuenta tenés que aceptar los términos.';
    }

    setErrores(nuevos);
    return Object.keys(nuevos).length === 0;
  };

  const continuar = () => {
    if (validarPaso1()) setPaso(2);
  };

  const volver = () => {
    if (paso === 2) {
      setPaso(1);
    } else {
      navigation.goBack();
    }
  };

  const elegirFecha = (evento, fecha) => {
    if (Platform.OS === 'android') setMostrarCalendario(false);
    if (evento.type === 'set' && fecha) {
      setFechaNacimiento(fecha);
      limpiarError('fecha');
    }
  };

  const mostrarErrorDeRegistro = (error) => {
    switch (error.code) {
      case 'auth/email-already-in-use':
        setCorreoYaRegistrado(true);
        setErrores({ correo: 'Ese correo ya tiene una cuenta en Cafeto.' });
        setPaso(1);
        break;
      case 'auth/invalid-email':
        setErrores({ correo: 'El formato del correo no es válido.' });
        setPaso(1);
        break;
      case 'auth/weak-password':
        setErrores({ contrasena: 'La contraseña es demasiado débil.' });
        setPaso(1);
        break;
      case 'auth/configuration-not-found':
      case 'auth/operation-not-allowed':
        avisar('advertencia', 'Registro no disponible', 'Falta habilitar el acceso con correo y
```

```
contraseña en Firebase.');
206         break;
207       case 'auth/network-request-failed':
208         avisar('error', 'Sin conexión', 'Revisá el wifi o los datos del teléfono.');
209         break;
210       default:
211         avisar('error', 'Error', `No pudimos crear la cuenta.\n\nCódigo: ${error.code ??
'desconocido'}`);
212     }
213   };
214
215   const crearCuenta = async () => {
216     if (!validarPaso2()) return;
217
218     setCreando(true);
219     try {
220       const credencial = await createUserWithEmailAndPassword(auth, correo.trim(), contrasena);
221       await updateProfile(credencial.user, { displayName: `${nombre.trim()} ${apellido.trim()}` });
222       await AsyncStorage.setItem(
223         `cafeto:perfil:${credencial.user.uid}`,
224         JSON.stringify({
225           nombre: nombre.trim(),
226           apellido: apellido.trim(),
227           telefono: telefono.trim() ? `+54 ${telefono.trim()}` : null,
228           fechaNacimiento: fechaNacimiento ? fechaNacimiento.toISOString().slice(0, 10) : null,
229           aceptaPromociones,
230         })
231       );
232       await sendEmailVerification(credencial.user);
233     } catch (error) {
234       setCreando(false);
235       mostrarErrorDeRegistro(error);
236     }
237   };
238
239   const iniciarSesionConEseCorreo = () => {
240     navigation.popTo('Login', { correo: correo.trim() });
241   };
242
243   const recuperarContrasena = async () => {
244     try {
245       await sendPasswordResetEmail(auth, correo.trim());
246       avisar('exito', 'Revisá tu correo', `Te mandamos un enlace a ${correo.trim()} para que elijas
una contraseña nueva.`);
247     } catch (error) {
248       avisar('error', 'Error', `No pudimos enviar el correo.\n\nCódigo: ${error.code ??
'desconocido'}`);
249     }
250   };
251
252   const usarOtroCorreo = () => {
253     setCorreo('');
254     setCorreoYaRegistrado(false);
255     limpiarError('correo');
256   };
257
258   const avisarProximamente = (proveedor) => {
259     avisar('advertencia', 'Próximamente', `El registro con ${proveedor} todavía no está
disponible.`);
260   };
261
262   if (!fuentesCargadas) {
263     return (
264       <View style={[estilos.pantalla, estilos.centrado]}>
265         <ActivityIndicator size="large" color={colores.marca} />
266       </View>
267     );
268   }
```

```
269
270   const casilla = (marcada, alTocar, contenido) => (
271     <Pressable style={estilos.filaCasilla} onPress={alTocar}>
272       <View style={[estilos.casilla, marcada && estilos.casillaMarcada]}>
273         {marcada && <Feather name="check" size={16} color={colores.textoSobreMarca} />}
274       </View>
275       <Text style={estilos.textoCasilla}>{contenido}</Text>
276     </Pressable>
277   );
278
279   const pie = (
280     <View style={estilos.filaPie}>
281       <Text style={estilos.textoPie}>¿Ya tenés cuenta? </Text>
282       <Pressable onPress={() => navigation.popTo('Login')}>
283         <Text style={[estilos.textoPie, estilos.enlace]}>Iniciá sesión</Text>
284       </Pressable>
285     </View>
286   );
287
288   return (
289     <SafeAreaView style={estilos.pantalla}>
290       <StatusBar style={colores.barraEstado} />
291       <KeyboardAvoidingView
292         style={estilos.pantalla}
293         behavior={Platform.OS === 'ios' ? 'padding' : undefined}
294       >
295         <ScrollView
296           contentContainerStyle={estilos.contenido}
297           keyboardShouldPersistTaps="handled"
298           showsVerticalScrollIndicator={false}
299         >
300           <View style={estilos.encabezado}>
301             <Pressable onPress={volver} hitSlop={10} disabled={creando}>
302               <Feather name="arrow-left" size={24} color={colores.texto} />
303             </Pressable>
304             <Image source={isotipo} style={estilos.isotipo} resizeMode="contain" />
305             <Text style={estilos.textoPaso}>PASO {paso} DE 2</Text>
306             <Pressable style={estilos.botonTema} onPress={cambiarTema}>
307               <Ionicons name="contrast" size={20} color={colores.texto} />
308             </Pressable>
309           </View>
310
311           <View style={estilos.progreso}>
312             <View style={[estilos.segmento, estilos.segmentoActivo]} />
313             <View style={[estilos.segmento, paso === 2 && estilos.segmentoActivo]} />
314           </View>
315
316           {paso === 1 ? (
317             <>
318               <Text style={estilos.titulo}>Creá tu cuenta</Text>
319
320               <View style={estilos.fila}>
321                 <CampoTexto
322                   estiloContenedor={estilos.mitad}
323                   etiqueta="Nombre"
324                   placeholder="Juan"
325                   valor={nombre}
326                   alCambiar={alCambiar('nombre', setNombre)}
327                   error={errores.nombre ?? ''}
328                   valido={validarNombre(nombre, 'nombre') === ''}
329                   autoCapitalize="words"
330                   autoComplete="given-name"
331                   maxLength={50}
332                 />
333                 <CampoTexto
334                   estiloContenedor={estilos.mitad}
335                   etiqueta="Apellido"
336                   placeholder="Montes"
```

```
                  valor={apellido}
                  alCambiar={alCambiar('apellido', setApellido)}
                  error={errores.apellido ?? ''}
                  valido={validarNombre(apellido, 'apellido') === ''}
                  autoCapitalize="words"
                  autoComplete="family-name"
                  maxLength={50}
                />
              </View>

              <CampoTexto
                etiqueta="Correo electrónico"
                icono="mail"
                placeholder="tunombre@correo.com"
                valor={correo}
                alCambiar={alCambiar('correo', setCorreo)}
                error={errores.correo ?? ''}
                ayudaAlEnfocar="Vas a entrar con este correo y te mandamos ahí el enlace de
verificación."
                valido={!correoYaRegistrado && FORMATO_CORREO.test(correo.trim())}
                keyboardType="email-address"
                autoCapitalize="none"
                autoCorrect={false}
                autoComplete="email"
              />

              {correoYaRegistrado ? (
                <>
                  <View style={estilos.aviso}>
                    <Feather name="info" size={18} color={colores.texto} />
                    <View style={estilos.textosAviso}>
                      <Text style={estilos.tituloAviso}>¿Sos vos?</Text>
                      <Text style={estilos.detalleAviso}>
                        Podés entrar con esa cuenta, o recuperar la contraseña si no te acordás.
                      </Text>
                    </View>
                  </View>

                  <Pressable
                    style={({ pressed }) => [estilos.botonPrincipal, pressed &&
estilos.presionado]}
                    onPress={iniciarSesionConEseCorreo}
                  >
                    <Text style={estilos.textoBotonPrincipal}>Iniciar sesión con ese correo</Text>
                  </Pressable>
                  <Pressable
                    style={({ pressed }) => [estilos.botonSecundario, pressed &&
estilos.presionado]}
                    onPress={recuperarContrasena}
                  >
                    <Text style={estilos.textoBotonSecundario}>Recuperar contraseña</Text>
                  </Pressable>

                  <View style={estilos.espaciador} />

                  <View style={estilos.filaPie}>
                    <Text style={estilos.textoPie}>¿Te equivocaste? </Text>
                    <Pressable onPress={usarOtroCorreo}>
                      <Text style={[estilos.textoPie, estilos.enlace]}>Usá otro correo</Text>
                    </Pressable>
                  </View>
                </>
              ) : (
                <>
                  <CampoTexto
                    etiqueta="Contraseña"
                    icono="lock"
                    placeholder="Mínimo 8 caracteres"
```

```
                    valor={contrasena}
                    alCambiar={alCambiar('contrasena', setContrasena)}
                    error={errores.contrasena ?? ''}
                    seguro
                    autoCapitalize="none"
                    autoComplete="new-password"
                  />
                  {contrasena !== '' && (
                    <>
                      <View style={estilos.barrasFortaleza}>
                        {[1, 2, 3, 4].map((numero) => (
                          <View
                            key={numero}
                            style={[
                              estilos.barraFortaleza,
                              numero <= fortaleza.puntaje && {
                                backgroundColor: fortaleza.segura ? colores.marca :
colores.peligro,
                              },
                            ]}
                          />
                        ))}
                      </View>
                      <View style={estilos.filaFortaleza}>
                        <Text style={estilos.textoFortaleza}>
                          Fortaleza: <Text style={estilos.nivelFortaleza}>{fortaleza.nivel}</Text>
                        </Text>
                        <Text style={estilos.textoFortaleza}>{fortaleza.consejo}</Text>
                      </View>
                    </>
                  )}

                  <CampoTexto
                    etiqueta="Repetir contraseña"
                    icono="lock"
                    placeholder="Repetí la contraseña"
                    valor={repetir}
                    alCambiar={alCambiar('repetir', setRepetir)}
                    error={errores.repetir ?? ''}
                    mensajeExito={coinciden ? 'Las contraseñas coinciden' : ''}
                    valido={coinciden}
                    seguro
                    autoCapitalize="none"
                    autoComplete="new-password"
                    returnKeyType="next"
                    onSubmitEditing={continuar}
                  />

                  <Pressable
                    style={({ pressed }) => [
                      estilos.botonPrincipal,
                      !paso1Completo && estilos.botonDeshabilitado,
                      pressed && estilos.presionado,
                    ]}
                    onPress={continuar}
                    disabled={!paso1Completo}
                  >
                    <Text
                      style={[estilos.textoBotonPrincipal, !paso1Completo &&
estilos.textoBotonDeshabilitado]}
                    >
                      Continuar
                    </Text>
                  </Pressable>

                  <View style={estilos.separador}>
                    <View style={estilos.linea} />
                    <Text style={estilos.textoSeparador}>o</Text>
```

```
                    <View style={estilos.linea} />
                  </View>

                  <Pressable
                    style={({ pressed }) => [estilos.botonSocial, estilos.botonGoogle, pressed &&
estilos.presionado]}
                    onPress={() => avisarProximamente('Google')}
                  >
                    <FontAwesome name="google" size={20} color="#4285F4" />
                    <Text style={[estilos.textoBotonSocial, estilos.textoGoogle]}>Continuar con
Google</Text>
                  </Pressable>
                  <Pressable
                    style={({ pressed }) => [estilos.botonSocial, estilos.botonApple, pressed &&
estilos.presionado]}
                    onPress={() => avisarProximamente('Apple')}
                  >
                    <FontAwesome name="apple" size={22} color="#FFFFFF" />
                    <Text style={[estilos.textoBotonSocial, estilos.textoApple]}>Continuar con
Apple</Text>
                  </Pressable>

                  <View style={estilos.espaciador} />
                  {pie}
                </>
              )}
            </>
          ) : (
            <>
              <Text style={estilos.titulo}>Últimos datos</Text>
              <Text style={estilos.subtitulo}>Opcionales, pero nos ayudan a reconocerte en el salón.
</Text>

              <CampoTexto
                etiqueta="Teléfono"
                opcional
                icono="smartphone"
                prefijo={
                  <View style={estilos.prefijo}>
                    <Text style={estilos.pais}>AR</Text>
                    <Text style={estilos.codigoPais}>+54</Text>
                  </View>
                }
                placeholder="387 000 0000"
                valor={telefono}
                alCambiar={alCambiar('telefono', setTelefono)}
                error={errores.telefono ?? ''}
                ayuda="Con tu teléfono también te encontramos en la caja para sumarte puntos."
                keyboardType="phone-pad"
                autoComplete="tel"
                maxLength={20}
              />

              <Text style={estilos.etiqueta}>
                Fecha de nacimiento<Text style={estilos.opcional}> · opcional</Text>
              </Text>
              <Pressable
                style={[estilos.selectorFecha, errores.fecha && estilos.campoConError]}
                onPress={() => setMostrarCalendario(true)}
              >
                <Feather name="calendar" size={20} color={colores.textoSecundario} />
                <Text style={[estilos.textoFecha, !fechaNacimiento && estilos.textoFechaVacia]}>
                  {fechaNacimiento ? formatearFecha(fechaNacimiento) : 'Elegí tu fecha'}
                </Text>
                <Feather name="chevron-down" size={20} color={colores.textoSecundario} />
              </Pressable>
              {errores.fecha ? (
                <View style={estilos.filaMensaje}>
```

```
                <Feather name="alert-circle" size={14} color={colores.peligro} />
                <Text style={estilos.textoError}>{errores.fecha}</Text>
              </View>
            ) : (
              <Text style={estilos.ayuda}>Te regalamos un café en tu cumpleaños.</Text>
            )}

            {mostrarCalendario && (
              <DateTimePicker
                value={fechaNacimiento ?? new Date(2000, 0, 1)}
                mode="date"
                display={Platform.OS === 'ios' ? 'spinner' : 'default'}
                maximumDate={new Date()}
                minimumDate={new Date(1900, 0, 1)}
                onChange={elegirFecha}
                themeVariant={tema === 'oscuro' ? 'dark' : 'light'}
                locale="es-AR"
              />
            )}
            {mostrarCalendario && Platform.OS === 'ios' && (
              <Pressable style={estilos.botonListo} onPress={() => setMostrarCalendario(false)}>
                <Text style={[estilos.textoPie, estilos.enlace]}>Listo</Text>
              </Pressable>
            )}

            <View style={estilos.casillas}>
              {casilla(
                aceptaTerminos,
                () => {
                  setAceptaTerminos(!aceptaTerminos);
                  limpiarError('terminos');
                },
                <>
                  Acepto los <Text style={estilos.enlace}>Términos y Condiciones</Text> y la{' '}
                  <Text style={estilos.enlace}>Política de privacidad</Text>
                </>
              )}
              {casilla(
                aceptaPromociones,
                () => setAceptaPromociones(!aceptaPromociones),
                'Quiero recibir promociones y novedades por correo'
              )}
              {errores.terminos ? (
                <View style={estilos.filaMensaje}>
                  <Feather name="alert-circle" size={14} color={colores.peligro} />
                  <Text style={estilos.textoError}>{errores.terminos}</Text>
                </View>
              ) : null}
            </View>

            <View style={estilos.espaciador} />

            <Pressable
              style={({ pressed }) => [estilos.botonPrincipal, pressed && estilos.presionado]}
              onPress={crearCuenta}
              disabled={creando}
            >
              {creando ? (
                <View style={estilos.filaCargando}>
                  <ActivityIndicator color={colores.textoSobreMarca} />
                  <Text style={estilos.textoBotonPrincipal}>Creando cuenta…</Text>
                </View>
              ) : (
                <Text style={estilos.textoBotonPrincipal}>Crear cuenta</Text>
              )}
            </Pressable>
            {pie}
          </>
```

```
          )}
        </ScrollView>
      </KeyboardAvoidingView>
    </SafeAreaView>
  );
}

function crearEstilos(c) {
  return StyleSheet.create({
    pantalla: {
      flex: 1,
      backgroundColor: c.fondo,
    },
    centrado: {
      justifyContent: 'center',
      alignItems: 'center',
    },
    contenido: {
      flexGrow: 1,
      paddingHorizontal: 28,
      paddingTop: 12,
      paddingBottom: 24,
    },
    encabezado: {
      flexDirection: 'row',
      alignItems: 'center',
      gap: 14,
    },
    isotipo: {
      width: 34,
      height: 34,
    },
    textoPaso: {
      flex: 1,
      fontFamily: 'Inter_500Medium',
      fontSize: 12,
      letterSpacing: 3,
      color: c.textoSecundario,
      textAlign: 'center',
    },
    botonTema: {
      width: 40,
      height: 40,
      borderRadius: 20,
      borderWidth: 1,
      borderColor: c.borde,
      backgroundColor: c.superficie,
      justifyContent: 'center',
      alignItems: 'center',
    },
    progreso: {
      flexDirection: 'row',
      gap: 10,
      marginTop: 14,
    },
    segmento: {
      flex: 1,
      height: 4,
      borderRadius: 2,
      backgroundColor: c.borde,
    },
    segmentoActivo: {
      backgroundColor: c.marca,
    },
    titulo: {
      fontFamily: 'Caveat_700Bold',
      fontSize: 38,
      color: c.titulo,
```

```
    marginTop: 20,
  },
  subtitulo: {
    fontFamily: 'Inter_400Regular',
    fontSize: 15,
    lineHeight: 21,
    color: c.textoSecundario,
  },
  fila: {
    flexDirection: 'row',
    gap: 12,
  },
  mitad: {
    flex: 1,
  },
  etiqueta: {
    fontFamily: 'Inter_600SemiBold',
    fontSize: 13.5,
    color: c.texto,
    marginTop: 16,
    marginBottom: 8,
  },
  opcional: {
    fontFamily: 'Inter_400Regular',
    color: c.textoTenue,
  },
  barrasFortaleza: {
    flexDirection: 'row',
    gap: 6,
    marginTop: 10,
  },
  barraFortaleza: {
    flex: 1,
    height: 4,
    borderRadius: 2,
    backgroundColor: c.borde,
  },
  filaFortaleza: {
    flexDirection: 'row',
    justifyContent: 'space-between',
    marginTop: 6,
  },
  textoFortaleza: {
    fontFamily: 'Inter_400Regular',
    fontSize: 12.5,
    color: c.textoSecundario,
  },
  nivelFortaleza: {
    fontFamily: 'Inter_600SemiBold',
    color: c.marca,
  },
  aviso: {
    flexDirection: 'row',
    gap: 12,
    padding: 16,
    marginTop: 16,
    borderRadius: 14,
    borderWidth: 1,
    borderColor: c.peligro,
    backgroundColor: c.fondoPeligro,
  },
  textosAviso: {
    flex: 1,
    gap: 4,
  },
  tituloAviso: {
    fontFamily: 'Inter_600SemiBold',
    fontSize: 14,
```

```
    color: c.texto,
  },
  detalleAviso: {
    fontFamily: 'Inter_400Regular',
    fontSize: 13.5,
    lineHeight: 19,
    color: c.textoSecundario,
  },
  prefijo: {
    flexDirection: 'row',
    alignItems: 'center',
    gap: 8,
  },
  pais: {
    fontFamily: 'Inter_600SemiBold',
    fontSize: 11,
    color: c.textoSecundario,
    paddingHorizontal: 6,
    paddingVertical: 2,
    borderRadius: 4,
    backgroundColor: c.borde,
    overflow: 'hidden',
  },
  codigoPais: {
    fontFamily: 'Inter_600SemiBold',
    fontSize: 16,
    color: c.texto,
  },
  selectorFecha: {
    flexDirection: 'row',
    alignItems: 'center',
    gap: 12,
    height: 52,
    paddingHorizontal: 16,
    borderRadius: 14,
    borderWidth: 1.5,
    borderColor: c.borde,
    backgroundColor: c.superficie,
  },
  campoConError: {
    borderColor: c.peligro,
  },
  textoFecha: {
    flex: 1,
    fontFamily: 'Inter_400Regular',
    fontSize: 16,
    color: c.texto,
  },
  textoFechaVacia: {
    color: c.textoTenue,
  },
  filaMensaje: {
    flexDirection: 'row',
    alignItems: 'center',
    gap: 6,
    marginTop: 6,
  },
  textoError: {
    flex: 1,
    fontFamily: 'Inter_400Regular',
    fontSize: 12.5,
    color: c.peligro,
  },
  ayuda: {
    fontFamily: 'Inter_400Regular',
    fontSize: 12.5,
    color: c.textoSecundario,
    marginTop: 6,
```

```
  },
  botonListo: {
    alignSelf: 'flex-end',
    paddingVertical: 8,
  },
  casillas: {
    gap: 14,
    marginTop: 20,
  },
  filaCasilla: {
    flexDirection: 'row',
    alignItems: 'flex-start',
    gap: 12,
  },
  casilla: {
    width: 22,
    height: 22,
    borderRadius: 6,
    borderWidth: 1.5,
    borderColor: c.borde,
    backgroundColor: c.superficie,
    justifyContent: 'center',
    alignItems: 'center',
  },
  casillaMarcada: {
    borderColor: c.marca,
    backgroundColor: c.marca,
  },
  textoCasilla: {
    flex: 1,
    fontFamily: 'Inter_400Regular',
    fontSize: 14,
    lineHeight: 20,
    color: c.texto,
  },
  enlace: {
    fontFamily: 'Inter_600SemiBold',
    color: c.marca,
    textDecorationLine: 'underline',
  },
  botonPrincipal: {
    height: 54,
    borderRadius: 14,
    backgroundColor: c.marca,
    justifyContent: 'center',
    alignItems: 'center',
    marginTop: 24,
  },
  botonDeshabilitado: {
    backgroundColor: c.superficie,
    borderWidth: 1,
    borderColor: c.borde,
  },
  textoBotonPrincipal: {
    fontFamily: 'Inter_600SemiBold',
    fontSize: 16,
    color: c.textoSobreMarca,
  },
  textoBotonDeshabilitado: {
    color: c.textoTenue,
  },
  botonSecundario: {
    height: 54,
    borderRadius: 14,
    borderWidth: 1,
    borderColor: c.borde,
    justifyContent: 'center',
    alignItems: 'center',
```

```
    marginTop: 12,
  },
  textoBotonSecundario: {
    fontFamily: 'Inter_600SemiBold',
    fontSize: 16,
    color: c.texto,
  },
  presionado: {
    opacity: 0.85,
  },
  filaCargando: {
    flexDirection: 'row',
    alignItems: 'center',
    gap: 10,
  },
  separador: {
    flexDirection: 'row',
    alignItems: 'center',
    gap: 16,
    marginVertical: 20,
  },
  linea: {
    flex: 1,
    height: 1,
    backgroundColor: c.divisor,
  },
  textoSeparador: {
    fontFamily: 'Inter_400Regular',
    fontSize: 14,
    color: c.textoSecundario,
  },
  botonSocial: {
    flexDirection: 'row',
    justifyContent: 'center',
    alignItems: 'center',
    gap: 10,
    height: 52,
    borderRadius: 14,
    borderWidth: 1,
    marginBottom: 12,
  },
  botonGoogle: {
    backgroundColor: '#FFFFFF',
    borderColor: '#E6E2D8',
  },
  botonApple: {
    backgroundColor: c.fondoApple,
    borderColor: c.bordeApple,
  },
  textoBotonSocial: {
    fontFamily: 'Inter_500Medium',
    fontSize: 16,
  },
  textoGoogle: {
    color: '#161616',
  },
  textoApple: {
    color: '#FFFFFF',
  },
  espaciador: {
    flex: 1,
    minHeight: 16,
  },
  filaPie: {
    flexDirection: 'row',
    justifyContent: 'center',
    alignItems: 'center',
    marginTop: 16,
```

```
    },
    textoPie: {
      fontFamily: 'Inter_400Regular',
      fontSize: 15,
      color: c.textoSecundario,
    },
  });
}
```

12.3 screens/Verificacion.js

screens/Verificacion.js · archivo completo — 376 líneas

```
import React, { useEffect, useMemo, useState } from 'react';
import { ActivityIndicator, Image, Pressable, ScrollView, StyleSheet, Text, View } from 'react-native';
import { SafeAreaView } from 'react-native-safe-area-context';
import { StatusBar } from 'expo-status-bar';
import { Feather, Ionicons } from '@expo/vector-icons';
import { useFonts, Caveat_700Bold } from '@expo-google-fonts/caveat';
import { Inter_400Regular, Inter_500Medium, Inter_600SemiBold } from '@expo-google-fonts/inter';
import { deleteUser, sendEmailVerification, signOut } from 'firebase/auth';
import { auth } from '../src/config/firebaseConfig';
import { useTema } from '../src/tema/TemaContext';
import { useAviso } from '../src/aviso/AvisoContext';

const ESPERA_REENVIO = 60;
const INTERVALO_CONSULTA = 4000;

function formatearReloj(segundos) {
  const minutos = String(Math.floor(segundos / 60)).padStart(2, '0');
  const resto = String(segundos % 60).padStart(2, '0');
  return `${minutos}:${resto}`;
}

export default function Verificacion() {
  const [fuentesCargadas] = useFonts({
    Caveat_700Bold,
    Inter_400Regular,
    Inter_500Medium,
    Inter_600SemiBold,
  });

  const [verificado, setVerificado] = useState(false);
  const [segundos, setSegundos] = useState(ESPERA_REENVIO);
  const [entrando, setEntrando] = useState(false);

  const { colores, isotipo, cambiarTema } = useTema();
  const { avisar } = useAviso();
  const estilos = useMemo(() => crearEstilos(colores), [colores]);
  const correo = auth.currentUser?.email ?? '';

  useEffect(() => {
    if (segundos === 0) return;
    const temporizador = setTimeout(() => setSegundos(segundos - 1), 1000);
    return () => clearTimeout(temporizador);
  }, [segundos]);

  useEffect(() => {
    if (verificado) return;
    const consulta = setInterval(() => {
      auth.currentUser
        .reload()
        .then(() => {
          if (auth.currentUser.emailVerified) setVerificado(true);
```

```
        })
        .catch(() => null);
    }, INTERVALO_CONSULTA);
    return () => clearInterval(consulta);
  }, [verificado]);

  const reenviarEnlace = async () => {
    try {
      await sendEmailVerification(auth.currentUser);
      setSegundos(ESPERA_REENVIO);
      avisar('exito', 'Enlace enviado', `Te mandamos un nuevo enlace a ${correo}.`);
    } catch (error) {
      if (error.code === 'auth/too-many-requests') {
        avisar('advertencia', 'Esperá un momento', 'Pediste varios enlaces seguidos. Probá de nuevo en unos minutos.');
      } else {
        avisar('error', 'Error', `No pudimos reenviar el enlace.\n\nCódigo: ${error.code ?? 'desconocido'}`);
      }
    }
  };

  const borrarRegistro = async () => {
    try {
      await deleteUser(auth.currentUser);
    } catch (error) {
      await signOut(auth);
      avisar(
        'error',
        'No pudimos borrar el registro',
        'Iniciá sesión de nuevo con ese correo y volvé a intentarlo, o registrate con el correo correcto.'
      );
    }
  };

  const corregirCorreo = () => {
    avisar(
      'advertencia',
      '¿Corregir el correo?',
      'Vamos a borrar este registro para que lo hagas de nuevo con el correo correcto.',
      [
        { texto: 'Cancelar', estilo: 'cancelar' },
        { texto: 'Corregir', estilo: 'destructivo', alPresionar: borrarRegistro },
      ]
    );
  };

  const empezar = async () => {
    setEntrando(true);
    await auth.currentUser.getIdToken(true);
  };

  if (!fuentesCargadas) {
    return (
      <View style={[estilos.pantalla, estilos.centrado]}>
        <ActivityIndicator size="large" color={colores.marca} />
      </View>
    );
  }

  return (
    <SafeAreaView style={estilos.pantalla}>
      <StatusBar style={colores.barraEstado} />
      <ScrollView contentContainerStyle={estilos.contenido} showsVerticalScrollIndicator={false}>
        <View style={estilos.encabezado}>
          {verificado ? (
            <View style={estilos.huecoEncabezado} />
```

```
          ) : (
            <Pressable style={estilos.chip} onPress={() => signOut(auth)}>
              <Text style={estilos.textoChip}>Salir</Text>
              <Feather name="log-out" size={14} color={colores.textoSecundario} />
            </Pressable>
          )}
          <Image source={isotipo} style={estilos.isotipo} resizeMode="contain" />
          <View style={estilos.huecoEncabezado}>
            <Pressable style={estilos.botonTema} onPress={cambiarTema}>
              <Ionicons name="contrast" size={20} color={colores.texto} />
            </Pressable>
          </View>
        </View>

        {verificado ? (
          <>
            <View style={estilos.espaciador} />
            <View style={[estilos.icono, estilos.iconoExito]}>
              <Feather name="check" size={44} color={colores.exito} />
            </View>
            <Text style={estilos.titulo}>¡Cuenta verificada!</Text>
            <Text style={estilos.texto}>Ya podés sumar puntos en cada compra.</Text>
            <View style={estilos.espaciador} />
            <Pressable
              style={({ pressed }) => [estilos.botonPrincipal, pressed && estilos.presionado]}
              onPress={empezar}
              disabled={entrando}
            >
              {entrando ? (
                <ActivityIndicator color={colores.textoSobreMarca} />
              ) : (
                <Text style={estilos.textoBotonPrincipal}>Empezar</Text>
              )}
            </Pressable>
          </>
        ) : (
          <>
            <View style={[estilos.icono, estilos.iconoCorreo]}>
              <Feather name="mail" size={40} color={colores.marca} />
            </View>
            <Text style={estilos.titulo}>Revisá tu correo</Text>
            <Text style={estilos.texto}>Te mandamos un enlace de verificación para activar la cuenta.
</Text>
            <Text style={estilos.correo}>{correo}</Text>
            <View style={estilos.filaCorregir}>
              <Text style={estilos.textoChico}>¿Está mal escrito? </Text>
              <Pressable onPress={corregirCorreo}>
                <Text style={[estilos.textoChico, estilos.enlace]}>Corregilo</Text>
              </Pressable>
            </View>

            <View style={estilos.tarjetaExito}>
              <Feather name="check" size={18} color={colores.exito} />
              <View style={estilos.textosTarjeta}>
                <Text style={estilos.tituloTarjeta}>Cuenta creada</Text>
                <Text style={estilos.detalleTarjeta}>
                  Solo falta verificar el correo para empezar a sumar puntos.
                </Text>
              </View>
            </View>

            <Pressable
              style={({ pressed }) => [estilos.botonSecundario, pressed && estilos.presionado]}
              onPress={reenviarEnlace}
              disabled={segundos > 0}
            >
              <Text style={[estilos.textoBotonSecundario, segundos > 0 && estilos.textoEsperando]}>
                {segundos > 0 ? `Reenviar enlace en ${formatearReloj(segundos)}` : 'Reenviar
```

```
enlace'}
              </Text>
            </Pressable>

            <View style={estilos.espaciador} />
            <Text style={estilos.pie}>¿No te llegó? Revisá la carpeta de spam.</Text>
          </>
        )}
      </ScrollView>
    </SafeAreaView>
  );
}

function crearEstilos(c) {
  return StyleSheet.create({
    pantalla: {
      flex: 1,
      backgroundColor: c.fondo,
    },
    centrado: {
      justifyContent: 'center',
      alignItems: 'center',
    },
    contenido: {
      flexGrow: 1,
      paddingHorizontal: 28,
      paddingTop: 12,
      paddingBottom: 24,
    },
    encabezado: {
      flexDirection: 'row',
      justifyContent: 'space-between',
      alignItems: 'center',
    },
    huecoEncabezado: {
      width: 90,
      alignItems: 'flex-end',
    },
    chip: {
      width: 90,
      flexDirection: 'row',
      justifyContent: 'center',
      alignItems: 'center',
      gap: 6,
      height: 38,
      borderRadius: 19,
      borderWidth: 1,
      borderColor: c.borde,
      backgroundColor: c.superficie,
    },
    textoChip: {
      fontFamily: 'Inter_500Medium',
      fontSize: 14,
      color: c.textoSecundario,
    },
    isotipo: {
      width: 34,
      height: 34,
    },
    botonTema: {
      width: 40,
      height: 40,
      borderRadius: 20,
      borderWidth: 1,
      borderColor: c.borde,
      backgroundColor: c.superficie,
      justifyContent: 'center',
      alignItems: 'center',
```

```
  },
  icono: {
    alignSelf: 'center',
    justifyContent: 'center',
    alignItems: 'center',
  },
  iconoCorreo: {
    width: 88,
    height: 88,
    borderRadius: 24,
    marginTop: 40,
    backgroundColor: c.marcaSuave,
  },
  iconoExito: {
    width: 96,
    height: 96,
    borderRadius: 48,
    backgroundColor: c.fondoExito,
  },
  titulo: {
    fontFamily: 'Caveat_700Bold',
    fontSize: 38,
    color: c.titulo,
    textAlign: 'center',
    marginTop: 24,
  },
  texto: {
    fontFamily: 'Inter_400Regular',
    fontSize: 15,
    lineHeight: 22,
    color: c.textoSecundario,
    textAlign: 'center',
    marginTop: 6,
  },
  correo: {
    fontFamily: 'Inter_600SemiBold',
    fontSize: 16,
    color: c.marca,
    textAlign: 'center',
    marginTop: 12,
  },
  filaCorregir: {
    flexDirection: 'row',
    justifyContent: 'center',
    alignItems: 'center',
    marginTop: 6,
  },
  textoChico: {
    fontFamily: 'Inter_400Regular',
    fontSize: 13,
    color: c.textoSecundario,
  },
  enlace: {
    fontFamily: 'Inter_600SemiBold',
    color: c.marca,
    textDecorationLine: 'underline',
  },
  tarjetaExito: {
    flexDirection: 'row',
    gap: 12,
    padding: 16,
    marginTop: 28,
    borderRadius: 14,
    borderWidth: 1,
    borderColor: c.exito,
    backgroundColor: c.fondoExito,
  },
  textosTarjeta: {
```

```
      flex: 1,
      gap: 4,
    },
    tituloTarjeta: {
      fontFamily: 'Inter_600SemiBold',
      fontSize: 14,
      color: c.texto,
    },
    detalleTarjeta: {
      fontFamily: 'Inter_400Regular',
      fontSize: 13.5,
      lineHeight: 19,
      color: c.textoSecundario,
    },
    botonPrincipal: {
      height: 54,
      borderRadius: 14,
      backgroundColor: c.marca,
      justifyContent: 'center',
      alignItems: 'center',
    },
    textoBotonPrincipal: {
      fontFamily: 'Inter_600SemiBold',
      fontSize: 16,
      color: c.textoSobreMarca,
    },
    botonSecundario: {
      height: 54,
      borderRadius: 14,
      borderWidth: 1,
      borderColor: c.borde,
      justifyContent: 'center',
      alignItems: 'center',
      marginTop: 16,
    },
    textoBotonSecundario: {
      fontFamily: 'Inter_600SemiBold',
      fontSize: 16,
      color: c.texto,
    },
    textoEsperando: {
      color: c.textoSecundario,
    },
    presionado: {
      opacity: 0.85,
    },
    espaciador: {
      flex: 1,
      minHeight: 24,
    },
    pie: {
      fontFamily: 'Inter_400Regular',
      fontSize: 14,
      color: c.textoSecundario,
      textAlign: 'center',
    },
  });
}
```

12.4 src/componentes/CampoTexto.js

src/componentes/CampoTexto.js · archivo completo — 146 líneas

```
import React, { useMemo, useState } from 'react';
import { Pressable, StyleSheet, Text, TextInput, View } from 'react-native';
```

```
import { Feather } from '@expo/vector-icons';
import { useTema } from '../tema/TemaContext';

export default function CampoTexto({
  etiqueta,
  opcional = false,
  icono,
  prefijo,
  valor,
  alCambiar,
  error = '',
  mensajeExito = '',
  ayuda = '',
  ayudaAlEnfocar = '',
  valido = false,
  seguro = false,
  estiloContenedor,
  ...propsEntrada
}) {
  const { colores } = useTema();
  const estilos = useMemo(() => crearEstilos(colores), [colores]);
  const [enfocado, setEnfocado] = useState(false);
  const [mostrar, setMostrar] = useState(false);

  let mensaje = null;
  if (error !== '') {
    mensaje = { texto: error, icono: 'alert-circle', estilo: estilos.textoError, color:
colores.peligro };
  } else if (mensajeExito !== '') {
    mensaje = { texto: mensajeExito, icono: 'check-circle', estilo: estilos.textoExito, color:
colores.exito };
  } else if (enfocado && ayudaAlEnfocar !== '') {
    mensaje = { texto: ayudaAlEnfocar, estilo: estilos.textoAyuda };
  } else if (ayuda !== '') {
    mensaje = { texto: ayuda, estilo: estilos.textoAyuda };
  }

  return (
    <View style={estiloContenedor}>
      <Text style={estilos.etiqueta}>
        {etiqueta}
        {opcional && <Text style={estilos.opcional}> · opcional</Text>}
      </Text>
      <View
        style={[
          estilos.campo,
          valido && estilos.campoValido,
          enfocado && estilos.campoActivo,
          error !== '' && estilos.campoConError,
        ]}
      >
        {icono && <Feather name={icono} size={20} color={colores.textoSecundario} />}
        {prefijo}
        <TextInput
          style={estilos.entrada}
          placeholderTextColor={colores.textoTenue}
          value={valor}
          onChangeText={alCambiar}
          onFocus={() => setEnfocado(true)}
          onBlur={() => setEnfocado(false)}
          secureTextEntry={seguro && !mostrar}
          {...propsEntrada}
        />
        {seguro && (
          <Pressable onPress={() => setMostrar(!mostrar)} hitSlop={10}>
            <Feather name={mostrar ? 'eye-off' : 'eye'} size={20} color={colores.textoSecundario}
/>
          </Pressable>
```

```
        )}
      </View>
      {mensaje && (
        <View style={estilos.filaMensaje}>
          {mensaje.icono && <Feather name={mensaje.icono} size={14} color={mensaje.color} />}
          <Text style={mensaje.estilo}>{mensaje.texto}</Text>
        </View>
      )}
    </View>
  );
}

function crearEstilos(c) {
  return StyleSheet.create({
    etiqueta: {
      fontFamily: 'Inter_600SemiBold',
      fontSize: 13.5,
      color: c.texto,
      marginTop: 16,
      marginBottom: 8,
    },
    opcional: {
      fontFamily: 'Inter_400Regular',
      color: c.textoTenue,
    },
    campo: {
      flexDirection: 'row',
      alignItems: 'center',
      gap: 12,
      height: 52,
      paddingHorizontal: 16,
      borderRadius: 14,
      borderWidth: 1.5,
      borderColor: c.borde,
      backgroundColor: c.superficie,
    },
    campoValido: {
      borderColor: c.exito,
    },
    campoActivo: {
      borderColor: c.marca,
      backgroundColor: c.superficieActiva,
    },
    campoConError: {
      borderColor: c.peligro,
    },
    entrada: {
      flex: 1,
      height: '100%',
      fontFamily: 'Inter_400Regular',
      fontSize: 16,
      color: c.texto,
    },
    filaMensaje: {
      flexDirection: 'row',
      alignItems: 'center',
      gap: 6,
      marginTop: 6,
    },
    textoError: {
      flex: 1,
      fontFamily: 'Inter_400Regular',
      fontSize: 12.5,
      color: c.peligro,
    },
    textoExito: {
      flex: 1,
      fontFamily: 'Inter_400Regular',
```

```
      fontSize: 12.5,
      color: c.exito,
    },
    textoAyuda: {
      flex: 1,
      fontFamily: 'Inter_400Regular',
      fontSize: 12.5,
      color: c.textoSecundario,
    },
  });
}
```

12.5 src/tema/temas.js

src/tema/temas.js · archivo completo — 64 líneas

```
export const temas = {
  oscuro: {
    fondo: '#121128',
    superficie: '#1B1B3A',
    superficieActiva: '#22223F',
    borde: '#2E2E5C',
    divisor: '#2E2E52',
    texto: '#E2E8F0',
    textoSecundario: '#A6B0C3',
    textoTenue: '#6E7899',
    titulo: '#FDE1A5',
    marca: '#FAB41F',
    marcaSuave: '#2A2410',
    textoSobreMarca: '#161616',
    peligro: '#F0564A',
    fondoPeligro: '#2A1B22',
    exito: '#35D97C',
    fondoExito: '#152A24',
    bordeExito: '#2F6B55',
    bordePeligro: '#6B4A52',
    fondoAdvertencia: '#2A2410',
    bordeAdvertencia: '#6B5A2A',
    velo: 'rgba(8, 8, 20, 0.72)',
    fondoApple: '#000000',
    bordeApple: '#2E2E5C',
    barraEstado: 'light',
  },
  claro: {
    fondo: '#FFFFFF',
    superficie: '#F4F1EA',
    superficieActiva: '#FFFFFF',
    borde: '#E6E2D8',
    divisor: '#EAECF1',
    texto: '#161616',
    textoSecundario: '#5B6078',
    textoTenue: '#8A8F9C',
    titulo: '#161616',
    marca: '#FAB41F',
    marcaSuave: '#FFF3D6',
    textoSobreMarca: '#161616',
    peligro: '#F0564A',
    fondoPeligro: '#FDECEC',
    exito: '#1F9D57',
    fondoExito: '#E9F8F0',
    bordeExito: '#A8DFC0',
    bordePeligro: '#F3B8B3',
    fondoAdvertencia: '#FFF3D6',
    bordeAdvertencia: '#F2D38A',
    velo: 'rgba(22, 22, 22, 0.45)',
```

```
    fondoApple: '#000000',
    bordeApple: '#000000',
    barraEstado: 'dark',
  },
};

export const logos = {
  oscuro: require('../../assets/cafeto-isologotipo-negativo@512.png'),
  claro: require('../../assets/cafeto-isologotipo-positivo@512.png'),
};

export const isotipos = {
  oscuro: require('../../assets/cafeto-isotipo-blanco@512.png'),
  claro: require('../../assets/cafeto-isotipo-negro@512.png'),
};
```

12.6 src/tema/TemaContext.js

src/tema/TemaContext.js · archivo completo — 25 líneas

```
import React, { createContext, useContext, useMemo, useState } from 'react';
import { isotipos, logos, temas } from './temas';

const TemaContext = createContext(null);

export function TemaProvider({ children }) {
  const [tema, setTema] = useState('oscuro');

  const valor = useMemo(
    () => ({
      tema,
      colores: temas[tema],
      logo: logos[tema],
      isotipo: isotipos[tema],
      cambiarTema: () => setTema((actual) => (actual === 'oscuro' ? 'claro' : 'oscuro')),
    }),
    [tema]
  );

  return <TemaContext.Provider value={valor}>{children}</TemaContext.Provider>;
}

export function useTema() {
  return useContext(TemaContext);
}
```

12.7 App.js

App.js · archivo completo — 15 líneas

```
import React from 'react';
import 'react-native-gesture-handler';
import Navigation from './navigation/Navigation';
import { TemaProvider } from './src/tema/TemaContext';
import { AvisoProvider } from './src/aviso/AvisoContext';

export default function App() {
  return (
    <TemaProvider>
      <AvisoProvider>
        <Navigation />
      </AvisoProvider>
    </TemaProvider>
```

```
  );
}
```

12.8 navigation/Navigation.js

navigation/Navigation.js · archivo completo — 61 líneas

```
import React, { useEffect, useState } from 'react';
import { ActivityIndicator, StyleSheet, Text, View } from 'react-native';
import { NavigationContainer } from '@react-navigation/native';
import { createStackNavigator } from '@react-navigation/stack';
import { onIdTokenChanged } from 'firebase/auth';
import { auth, isFirebaseConfigured } from '../src/config/firebaseConfig';
import Login from '../screens/Login';
import Registro from '../screens/Registro';
import Verificacion from '../screens/Verificacion';
import Home from '../screens/Home';

const Stack = createStackNavigator();

export default function Navigation() {
  const [user, setUser] = useState(undefined);
  const [verificado, setVerificado] = useState(false);

  useEffect(() => {
    if (!auth) return;
    return onIdTokenChanged(auth, (usuario) => {
      setUser(usuario);
      setVerificado(Boolean(usuario && (usuario.emailVerified || usuario.isAnonymous)));
    });
  }, []);

  if (!isFirebaseConfigured) {
    return (
      <View style={styles.center}>
        <Text style={styles.message}>
          Falta configurar Firebase. Copia .env.example como .env y completa los datos de tu
proyecto.
        </Text>
      </View>
    );
  }

  if (user === undefined) {
    return <View style={styles.center}><ActivityIndicator size="large" /></View>;
  }

  return (
    <NavigationContainer>
      <Stack.Navigator screenOptions={{ headerShown: false }}>
        {!user ? (
          <>
            <Stack.Screen name="Login" component={Login} />
            <Stack.Screen name="Registro" component={Registro} />
          </>
        ) : !verificado ? (
          <Stack.Screen name="Verificacion" component={Verificacion} />
        ) : (
          <Stack.Screen name="Home" component={Home} options={{ headerShown: true, title: 'Inicio' }}
/>
        )}
      </Stack.Navigator>
    </NavigationContainer>
  );
}
```

```

const styles = StyleSheet.create({
  center: { flex: 1, justifyContent: 'center', alignItems: 'center', padding: 24 },
  message: { textAlign: 'center', fontSize: 16 },
});
```

12.9 src/aviso/AvisoContext.js

src/aviso/AvisoContext.js · archivo completo — 175 líneas

```
import React, { createContext, useContext, useMemo, useState } from 'react';
import { Modal, Pressable, StyleSheet, Text, View } from 'react-native';
import { Feather } from '@expo/vector-icons';
import { useTema } from '../tema/TemaContext';

const AvisoContext = createContext(null);

const ICONOS = {
  error: 'alert-circle',
  advertencia: 'alert-triangle',
  exito: 'check-circle',
};

const ESTILOS_BOTON = {
  cancelar: ['botonCancelar', 'textoCancelar'],
  destructivo: ['botonDestructivo', 'textoDestructivo'],
};

export function AvisoProvider({ children }) {
  const { colores } = useTema();
  const estilos = useMemo(() => crearEstilos(colores), [colores]);
  const [aviso, setAviso] = useState(null);

  const valor = useMemo(
    () => ({
      avisar: (tipo, titulo, mensaje, botones = [{ texto: 'Entendido' }]) => {
        setAviso({ tipo, titulo, mensaje, botones });
      },
    }),
    []
  );

  const colorDelIcono = {
    error: colores.peligro,
    advertencia: colores.marca,
    exito: colores.exito,
  };

  const tocarBoton = (boton) => {
    setAviso(null);
    if (boton.alPresionar) boton.alPresionar();
  };

  return (
    <AvisoContext.Provider value={valor}>
      {children}
      <Modal
        visible={aviso !== null}
        transparent
        animationType="fade"
        statusBarTranslucent
        onRequestClose={() => setAviso(null)}
      >
        <View style={estilos.velo}>
          {aviso && (
```

```
            <View style={[estilos.tarjeta, estilos[aviso.tipo]]}>
              <View style={estilos.cabecera}>
                <Feather name={ICONOS[aviso.tipo]} size={22} color={colorDelIcono[aviso.tipo]} />
                <Text style={estilos.titulo}>{aviso.titulo}</Text>
              </View>
              <Text style={estilos.mensaje}>{aviso.mensaje}</Text>
              <View style={estilos.botones}>
                {aviso.botones.map((boton) => {
                  const [estiloBoton, estiloTexto] = ESTILOS_BOTON[boton.estilo] ?? [
                    'botonPrincipal',
                    'textoPrincipal',
                  ];
                  return (
                    <Pressable
                      key={boton.texto}
                      style={({ pressed }) => [estilos.boton, estilos[estiloBoton], pressed &&
estilos.presionado]}
                      onPress={() => tocarBoton(boton)}
                    >
                      <Text style={[estilos.textoBoton, estilos[estiloTexto]]}>{boton.texto}</Text>
                    </Pressable>
                  );
                })}
              </View>
            </View>
          )}
        </View>
      </Modal>
    </AvisoContext.Provider>
  );
}

export function useAviso() {
  return useContext(AvisoContext);
}

function crearEstilos(c) {
  return StyleSheet.create({
    velo: {
      flex: 1,
      justifyContent: 'center',
      paddingHorizontal: 28,
      backgroundColor: c.velo,
    },
    tarjeta: {
      padding: 20,
      borderRadius: 18,
      borderWidth: 1,
    },
    error: {
      backgroundColor: c.fondoPeligro,
      borderColor: c.bordePeligro,
    },
    advertencia: {
      backgroundColor: c.fondoAdvertencia,
      borderColor: c.bordeAdvertencia,
    },
    exito: {
      backgroundColor: c.fondoExito,
      borderColor: c.bordeExito,
    },
    cabecera: {
      flexDirection: 'row',
      alignItems: 'center',
      gap: 10,
    },
    titulo: {
      flex: 1,
```

```
      fontFamily: 'Inter_600SemiBold',
      fontSize: 16,
      color: c.texto,
    },
    mensaje: {
      fontFamily: 'Inter_400Regular',
      fontSize: 14.5,
      lineHeight: 21,
      color: c.textoSecundario,
      marginTop: 8,
      marginLeft: 32,
    },
    botones: {
      flexDirection: 'row',
      justifyContent: 'flex-end',
      gap: 10,
      marginTop: 20,
    },
    boton: {
      height: 44,
      paddingHorizontal: 18,
      borderRadius: 12,
      justifyContent: 'center',
      alignItems: 'center',
    },
    botonPrincipal: {
      backgroundColor: c.marca,
    },
    botonCancelar: {
      borderWidth: 1,
      borderColor: c.borde,
    },
    botonDestructivo: {
      backgroundColor: c.peligro,
    },
    presionado: {
      opacity: 0.85,
    },
    textoBoton: {
      fontFamily: 'Inter_600SemiBold',
      fontSize: 15,
    },
    textoPrincipal: {
      color: c.textoSobreMarca,
    },
    textoCancelar: {
      color: c.texto,
    },
    textoDestructivo: {
      color: '#FFFFFF',
    },
  });
}
```

13

Autoevaluación

Intentá responder sin mirar el código. Si dudás, volvé al bloque indicado.

- ¿Por qué `logos` usa dos `require` en lugar de armar la ruta con el nombre del tema? (5.4)
- ¿Qué problema resolvió mover el tema a un contexto? (3)
- ¿Por qué la navegación usa `onIdTokenChanged` y un estado `verificado` aparte? (3.3)
- ¿Por qué `AvisoProvider` tiene que ir dentro de `TemaProvider`? (3.2)
- ¿Por qué se reemplazó `Alert.alert` por un `Modal` propio? (3.4)
- ¿Qué pasaría si el `if (!fuentesCargadas)` estuviera antes de los `useState`? (5.16)
- ¿Por qué no hay `navigation.navigate('Home')` en ningún lado? (1 y 5.13)
- ¿Qué diferencia hay entre `style` y `contentContainerStyle` en un ScrollView? (6.1)
- ¿Por qué el borde del campo está en un View y no en el TextInput? (6.4)
- ¿Por qué `onPress={() ⇒ avisarProximamente('Google')}` lleva flecha? (5.15)
- Si el botón ya está deshabilitado con los campos vacíos, ¿por qué `validarFormulario` sigue revisando que no estén vacíos? (5.10)
- ¿Para qué sirve el fragmento `<> … </>` en el bloque de carga? (6.8)
- ¿Qué hace el espaciador y por qué solo se nota mientras carga? (6.9)
- ¿Por qué se guarda el correo y no la contraseña en AsyncStorage? (5.8)
- ¿Cuál es la diferencia entre `flex: 1` y `flexGrow: 1` en el contenido del ScrollView? (7.1)
- ¿Qué estado guarda `CampoTexto` y cuál deja en manos de la pantalla? ¿Por qué? (8.2)
- ¿Qué devuelve `alCambiar('correo', setCorreo)`? (9.9)
- ¿Por qué se exigen solo tres de las cuatro pruebas de fortaleza para avanzar? (9.5)
- ¿Por qué el error de correo ya registrado aparece en el paso 1 si se detecta en el paso 2? (9.13)
- ¿Por qué el Registro usa `popTo` y no `navigate` para volver al Login? (9.15)
- ¿Cómo se entera la app de que el usuario verificó su correo en otra aplicación? (10.3)
- ¿Qué hace `getIdToken(true)` y por qué cambia la pantalla? (10.6)

Ejercicio de memoria. Cerrá esta guía, abrí un archivo vacío y reconstruí el Login siguiendo solo los títulos de la sección 4. Después hacé lo mismo con el Registro y los pasos de 9.2. Al terminar, compará con la sección 12.

Ejercicios para practicar. 1) Pasá los dos campos del Login a `CampoTexto`. 2) Hacé que el botón atrás de Android vuelva al paso 1 del Registro (9.12). 3) Extraé los botones de Google y Apple a un componente `BotonesSociales` que usen el Login y el Registro.

14

La rueda flotante de Expo Go

Al probar en el celular aparece una rueda de configuración que se puede arrastrar y que puede quedar encima del botón de contraste. **Esa rueda no es parte de la app.** La dibuja Expo Go: es el botón de herramientas de desarrollo y abre el menú de desarrollador (recargar, inspector, rendimiento, etc.). Por eso no aparece en `Login.js` y no se puede borrar desde el código.

Solo existe mientras desarrollás. Cuando la app se compila para instalarse (un APK o una build de la tienda), la rueda no está.

Cómo sacarla del medio

1. **Moverla.** Se puede arrastrar justamente para eso: dejala en un borde donde no tape nada, por ejemplo abajo a la derecha.
2. **Ocultarla desde el menú.** Tocá la rueda para abrir el menú de desarrollador y buscá la opción para ocultar el botón de herramientas. El nombre exacto cambia según la versión de Expo Go, así que revisá las opciones del menú.
3. **Abrir el menú sin la rueda.** Si la ocultás, el menú se sigue abriendo sacudiendo el celular o presionando `m` en la terminal donde corre `npm start`.

¿Hay que reemplazarla por algo en la app?

No. El botón de contraste es de la app y queda donde está. La rueda es una herramienta tuya como desarrollador, no del usuario, así que no necesita un reemplazo en la pantalla.

Si más adelante el proyecto pasa de Expo Go a una *development build* (con `expo-dev-client`), ahí sí se puede ocultar por código o configuración: la librería trae `setToolsButtonVisible(false)` y la opción `toolsButton`. En Expo Go no aplican.

15

Qué falta del Login y el Registro

El Login funciona completo: valida, inicia sesión con Firebase, recuerda el correo, recupera la contraseña por correo y deja entrar como invitado. El Registro crea la cuenta, guarda los datos y verifica el correo. Lo que falta son algunas **vistas y estados** del diseño (carpeta `Vistas-Móvil-Cafeto`, resúmenes `Resumen_1_Login.png` y `Resumen_2_Registro.png`).

VISTA	ESTADO	QUÉ FALTA Y CÓMO ENCARARLO
1.1 Login inicial	Hecha	Nada.
1.2 Login con error	Parcial	El campo en rojo y el mensaje ya están. Falta la tarjeta «Revisá las mayúsculas · Varios intentos fallidos activan una pausa» y el borde verde del correo válido. Se resuelve con un estado `intentosFallidos` que sube en cada error de contraseña, y la tarjeta se muestra cuando vale 1 o más.
1.3 Login escribiendo	Hecha	Borde ámbar al enfocar, ayuda debajo del correo y botón gris deshabilitado mientras falte un campo (6.4 y 6.7).
1.4 Recuperar contraseña	Resuelta de otra forma	Hoy se envía el enlace desde el mismo Login. El diseño pide una pantalla propia con su campo, el botón «Enviando…», «Volver a iniciar sesión» y la ayuda por WhatsApp. Es un archivo nuevo `screens/RecuperarContrasena.js` agregado al stack y abierto con `navigation.navigate`. Reutiliza casi todo lo de esta guía.
1.5 Ingresando	Hecha	Botón con indicador, aviso «Un momento, estamos verificando tus datos.», separador, sociales y texto legal ocultos, y el registro al fondo (6.7 a 6.9).
1.6 Demasiados intentos	Parcial	Se muestra el mensaje. Falta la tarjeta con cuenta regresiva (0:30), el botón deshabilitado durante la pausa y el enlace «Recuperala ahora». La cuenta regresiva se hace con un estado de segundos y un `setInterval` dentro de un `useEffect` que se limpia al llegar a cero.
1.7 Sin conexión	Falta	Hoy solo aparece una alerta al fallar. La vista completa necesita saber si hay red: se instala `@react-native-community/netinfo` con `npx expo install` y se muestra la pantalla con «Reintentar» cuando no hay conexión.
1.8 Contraseña	Falta	Firebase, por defecto, deja elegir la contraseña nueva en una página web suya que se abre desde el correo. Hacerlo dentro de

VISTA	ESTADO	QUÉ FALTA Y CÓMO ENCARARLO
nueva		la app requiere que el enlace abra la app (deep links) y usar `confirmPasswordReset`. Es lo más complejo de la lista; conviene dejarlo para el final.
Google y Apple	Falta	Hoy muestran «Próximamente», en el Login y en el Registro. Requieren configurar cada proveedor en Firebase y en la app; con Expo Go tienen limitaciones.
2.1 Registro, paso 1	Hecha	Sección 9.
2.2 Registro, validación	Hecha	Bordes verdes, errores por campo, medidor de fortaleza y coincidencia de contraseñas.
2.3 Registro, paso 2	Hecha	Teléfono y fecha de nacimiento se guardan en el teléfono hasta conectar la API de Django (9.1).
2.4 Verificación de correo	Hecha	Sección 10, con consulta automática y reenvío.
2.5 Correo ya registrado	Hecha	Se detecta al crear la cuenta y vuelve al paso 1 (9.13).
2.6 Teclado abierto	Hecha	Ayuda debajo del correo al enfocarlo y contenido desplazable.
2.7 Enlace vencido	Resuelta de otra forma	El enlace lo abre una página de Firebase, que avisa si venció; en la app se pide uno nuevo con «Reenviar enlace» (10).
2.8 Cuenta verificada	Hecha	Con «Empezar» en lugar de «Ver la carta», que todavía no existe.

Orden sugerido para lo que queda: 1.2 y 1.6 (agregan el contador de intentos), luego 1.4 (pantalla nueva), 1.7 y por último 1.8.

Cuando se conecte Django. El Registro y el Login dejarán de llamar a Firebase y llamarán a la API del sistema de PC. La interfaz, las validaciones y `CampoTexto` quedan igual; cambian las funciones `crearCuenta`, `iniciarSesion` y la consulta de verificación. Los datos que hoy van a AsyncStorage pasarán a la tabla `clientes`.

Pantalla de Inicio. `screens/Home.js` sigue siendo la del proyecto del profesor: usa `Alert.alert` y no tiene el diseño de Cafeto. Se va a rehacer con las vistas de la sección de clientes.

Para probar: en la terminal, dentro de `sistema-cafeto-movil`, ejecutá `npm start` y escaneá el QR con Expo Go. Para «Seguir sin cuenta», habilitá Authentication → Método de acceso → Anónimo en la consola de Firebase.